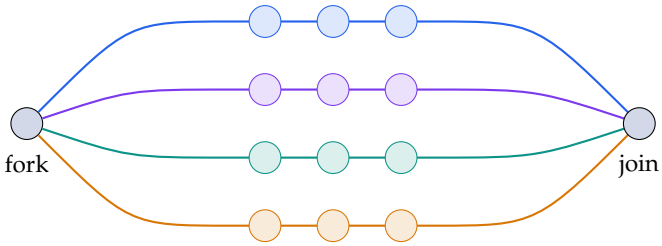
CS4533
Parallel and Concurrent Programming

Study Notes - Part 1



University of Moratuwa
Semester 7

Lectures 01–09

Introduction · How much parallelism · The concurrent programming abstraction
Shared-memory programming · Mutual exclusion · Basic constructs · Accelerators · Semaphores

Concepts, proofs and worked examples for shared-memory concurrency, with the reasoning behind each step.

Contents

1 Introduction to Concurrent Programming **1**

1.1 The Hardware Motivation . . . 1

1.1.1 Moore's Law and the End of the Free Ride . . . 1

1.1.2 Migration in Architecture . . . 2

1.1.3 Three Scaling Processes . . . 2

1.2 Defining the Field . . . 3

1.2.1 Concurrent vs Parallel vs Distributed . . . 3

1.2.2 Sequential vs Concurrent Computation . . . 4

1.2.3 What This Course Covers . . . 4

1.3 Asynchrony . . . 4

1.4 Why Parallel/Concurrent Computing? . . . 5

1.5 Flynn's Taxonomy . . . 5

1.6 Road Map of the Course . . . 6

1.7 Case Study: Parallel Primality Testing . . . 6

1.7.1 Attempt 1: Static Load Balancing . . . 6

1.7.2 Attempt 2: Shared Counter . . . 7

1.7.3 What the Counter Really Does . . . 7

1.7.4 The Race Condition . . . 8

1.7.5 The Fix: Atomicity . . . 8

1.8 The Alice and Bob Fable . . . 9

1.8.1 Formalizing the Problem . . . 9

1.8.2 Three Failed Protocols . . . 9

1.8.3 Working Protocol: Flags . . . 10

1.9 Producer/Consumer . . . 11

1.9.1 The Solution: A Single Can . . . 11

1.10 Readers/Writers . . . 12

1.11 Summary . . . 13

2 How Much Parallelism? **14**

2.1 Amdahl's Law . . . 14

2.1.1 Definition of Speedup . . . 14

2.1.2 The Law . . . 14

2.1.3 Worked Examples (all with 10 processors) . . . 15

2.1.4 The Asymptotic Limit . . . 15
2.2 Costs and Complexity of Parallelism . . . 16
2.2.1 Costs . . . 16
2.2.2 Complexity . . . 17
2.3 Case Study: Fibonacci . . . 17
2.3.1 Serial Version . . . 17
2.3.2 Parallel Version with Dynamic Multithreading . . . 18
2.4 The Computation DAG Model . . . 18
2.5 Performance Measures: Work and Span . . . 18
2.5.1 Notation . . . 19
2.6 Speedup, Efficiency, and Parallelism . . . 19
2.6.1 Parallelism . . . 20
2.7 Throughput, Latency, and Resources . . . 21
2.8 Scalability . . . 21
2.9 Measuring Performance . . . 22
2.9.1 Statistical Basis . . . 22
2.9.2 How Many Samples? . . . 23
2.10 Data Presentation . . . 23
2.11 Summary . . . 24

3 Concurrent Programming Abstraction 26
Part 1: The Critical-Section Problem . . . 26
3.1 Race Conditions . . . 26
3.2 The Critical-Section Problem . . . 26
3.2.1 General Structure of a Process . . . 27
3.3 Requirements for a Correct Solution . . . 27
3.4 Deadlock and Starvation . . . 27
3.4.1 Deadlock . . . 28
3.4.2 Starvation . . . 28
3.5 Methods for Synchronization . . . 29
3.6 Classical Problems of Synchronization . . . 29
Part 2: Concurrent Programming Abstraction . . . 29
3.7 The Abstraction . . . 30
3.8 Events, Threads, and States . . . 30
3.8.1 Threads are State Machines . . . 30
3.9 Concurrency and Interleavings . . . 31
3.10 Intervals and Precedence . . . 31

3.11 Interleaving . . . 32
3.11.1 Counting Interleavings . . . 32
3.11.2 Interleaving Changes the Meaning of a Program . . . 33
3.12 Atomic Instructions . . . 34
3.12.1 The Granularity Example . . . 34
3.12.2 Exercise: Possible Final Values . . . 35
3.13 Correctness of Concurrent Programs . . . 35
3.14 Summary . . . 36

4 Shared Memory Programming: Pthreads & OpenMP 38
4.1 The Shared Memory System . . . 38
Part 1: POSIX Threads (Pthreads) . . . 38
4.2 What Pthreads Is . . . 38
4.2.1 Hello World . . . 39
4.2.2 Compiling and Running . . . 39
4.3 The Fork/Join Model . . . 40
4.4 Starting and Stopping Threads . . . 40
4.4.1 `pthread_create` . . . 40
4.4.2 `pthread_join` . . . 41
4.5 Example: Matrix-Vector Multiplication . . . 41
4.5.1 Serial Version . . . 41
4.5.2 Parallel Decomposition . . . 42
4.6 Example: Estimating π (and How It Breaks) . . . 42
4.6.1 Fix Attempt 1: Busy-Waiting . . . 43
4.7 Mutexes . . . 43
4.7.1 Global Sum Using a Mutex . . . 44
4.7.2 Busy-Waiting vs Mutex . . . 45
4.8 Read-Write Locks . . . 45
4.8.1 Motivating Example: A Shared Sorted Linked List . . . 45
4.8.2 Solution 1: One Mutex for the Whole List . . . 47
4.8.3 Solution 2: One Mutex per Node . . . 47
4.8.4 Solution 3: Read-Write Locks . . . 48
4.8.5 Performance Comparison . . . 49
Part 2: OpenMP . . . 49
4.9 What OpenMP Is . . . 49
4.9.1 Hello World in OpenMP . . . 50
4.10 OpenMP Pragmas and Clauses . . . 51

4.10.1 The Parallel Directive . . . 51
4.10.2 Clauses . . . 51
4.10.3 Mutual Exclusion in OpenMP . . . 51
4.11 Example: The Trapezoidal Rule . . . 52
4.11.1 Serial Algorithm . . . 52
4.11.2 OpenMP Version . . . 52
4.12 Summary . . . 53

5 The Mutual Exclusion Problem 55
5.1 Statement of the Problem . . . 55
5.2 Formal Requirements for *N* Processes . . . 55
5.2.1 Ground Rules for Two Processes . . . 56
5.3 First Attempt: Strict Alternation . . . 56
5.4 Second Attempt: Test Then Set Own Flag . . . 57
5.5 Third Attempt: Set Own Flag Then Test . . . 58
5.6 Fourth Attempt: Back Off . . . 59
5.7 A Correct Solution: Dekker's Algorithm . . . 61
5.8 Peterson's Algorithm . . . 62
5.9 Summary Table of All Attempts . . . 63
5.10 Exercise . . . 64
5.11 Summary . . . 65

6 Basic Constructs 66
6.1 Why Concurrent Programming is Hard . . . 66
6.1.1 How Things Can Go Wrong . . . 66
6.2 Mutual Exclusion vs Synchronization . . . 67
6.3 Deadlocks . . . 68
6.3.1 The Four Conditions for Deadlock (Coffman conditions) . . . 68
6.4 Livelock and Starvation . . . 68
6.5 The Lock Interface . . . 69
6.5.1 Using Locks . . . 69
6.5.2 Example: A Correct Counter . . . 69
6.6 Exercise: Lock Ordering . . . 70
6.7 Building a Lock: 2-Thread Solutions First . . . 71
6.7.1 LockOne . . . 71
6.7.2 LockTwo . . . 72
6.8 Peterson's Algorithm . . . 72
6.9 The Filter Algorithm for *n* Threads . . . 73

6.9.1 The Code . . . 74
6.10 Bounded Waiting . . . 75
6.11 The Bakery Algorithm . . . 75
6.11.1 The Code . . . 75
6.11.2 Properties . . . 76
6.12 Comparison of All Lock Implementations . . . 77
6.13 Summary . . . 77

7 Introduction to Accelerators 79
7.1 What is a GPU? . . . 79
7.1.1 Why GPUs Exist . . . 79
7.2 Example Specifications . . . 80
7.3 CPU vs GPU Architecture . . . 80
7.3.1 FLOPS and GFLOPS . . . 81
7.4 A Simple Performance Test . . . 81
7.5 Programming GPUs . . . 82
7.6 CUDA Architecture . . . 83
7.6.1 Grid, Block, Thread . . . 83
7.6.2 Hello World . . . 84
7.7 CUDA Device Memory Model . . . 84
7.7.1 Minimal Memory Example . . . 85
7.7.2 Vector Addition — One Block per Element . . . 85
7.8 Case Study: Matrix Multiplication in CUDA . . . 86
7.8.1 The Original CPU Kernel . . . 86
7.8.2 Mapping Loops onto the Grid . . . 86
7.8.3 The GPU Kernel . . . 87
7.8.4 The Full Host Program . . . 87
7.9 CUDA Language Reference . . . 88
7.9.1 Function Type Qualifiers . . . 89
7.9.2 Variable Type Qualifiers . . . 89
7.9.3 Execution Configuration . . . 89
7.9.4 Built-in Variables . . . 90
7.9.5 Built-in Vector Types . . . 90
7.9.6 Intrinsic Functions . . . 90
7.10 Synchronization, Fencing and Atomics . . . 91
7.10.1 Atomic Functions . . . 91
7.11 Device and Memory Management APIs . . . 92

7.11.1 Error Handling . . . 93
7.12 Intel Xeon Phi . . . 94
7.12.1 Knights Landing (KNL) . . . 94
7.12.2 Architecture . . . 94
7.12.3 Memory . . . 95
7.13 GPU vs Xeon Phi at a Glance . . . 96
7.14 Summary . . . 96

8 Semaphores **98**
8.1 The Railway Metaphor . . . 98
8.2 Definition of a Semaphore . . . 99
8.2.1 The Semaphore Invariants . . . 100
8.3 Mutual Exclusion using a Semaphore . . . 100
8.3.1 Proof of Mutual Exclusion . . . 101
8.3.2 Proof of Deadlock Freedom . . . 101
8.3.3 Proof of No Individual Starvation (for $N = 2$) . . . 101
8.3.4 No Starvation in the Absence of Contention . . . 102
8.4 Semaphore Definitions . . . 102
8.4.1 Blocked-Set Semaphore . . . 102
8.4.2 Blocked-Queue Semaphore . . . 102
8.4.3 Busy-Wait Semaphore . . . 102
8.4.4 Which Definitions Starve? . . . 103
8.5 Exercise: Bill and Ben in the Cafe . . . 104
8.6 The Producer–Consumer Problem . . . 105
8.6.1 Infinite Buffer — First Attempt . . . 106
8.6.2 Infinite Buffer — With a Semaphore . . . 106
8.6.3 Bounded (Circular) Buffer . . . 107
8.7 The Dining Philosophers . . . 108
8.8 The Readers and Writers Problem . . . 109
8.8.1 An Over-Constrained Solution . . . 109
8.8.2 The Proper Solution . . . 110
8.9 The Sleeping Barber . . . 111
8.10 Exercise: The Baboon Crossing Problem . . . 112
8.11 Comparison of the Classic Problems . . . 113
8.12 Summary . . . 114

9 Basic Constructs II: Mutex & Monitors **116**
9.1 Issues with Busy Waiting . . . 116

9.2 Semaphores, Recapped . . . 117
9.3 Mutex . . . 117
9.4 Why Semaphores Are Not Enough . . . 117
9.5 Monitor Definition . . . 118
9.5.1 Condition Variables . . . 118
9.5.2 The Two Essential Capabilities . . . 119
9.6 Producer–Consumer with a Monitor . . . 119
9.7 Monitor Signalling Disciplines . . . 120
9.7.1 Signal and Exit . . . 121
9.7.2 Signal and Continue . . . 121
9.7.3 Signal and Wait . . . 121
9.7.4 The Three Disciplines Side by Side . . . 122
9.8 Monitors in Java . . . 122
9.8.1 The Object Lock . . . 123
9.8.2 Condition Synchronization in Java . . . 123
9.9 Producer–Consumer in Java — and the Barging Problem . . . 124
9.10 Multiplex . . . 125
9.11 Barriers . . . 126
9.12 The Construct Ladder . . . 127
9.13 Summary . . . 127

A Quick Reference **129**
A.1 Formula Sheet . . . 129
A.2 Correctness Properties at a Glance . . . 129
A.3 Optional Web Figures . . . 130
A.4 Building These Notes . . . 130

List of Figures

1.1 Illustrative trend (not exact data): transistor counts keep growing exponentially while clock frequency stalled; the extra transistors became extra cores. . . . 1

1.2 From a single CPU with private memory to many CPUs with private caches sharing one memory over a bus. . . . 2

1.3 Real multicore speedup falls far short of the ideal. . . . 3

1.4 Concurrency is about structure (time-slicing on one core); parallelism is about execution (simultaneous on many cores). . . . 3

1.5 Every difficulty in this course flows from the right-hand picture. . . . 4

1.6 Asynchrony: any thread can stall at any moment for an unpredictable length of time. 4

1.7 Flynn's taxonomy. I = instruction stream, D = data stream, PU = processing unit. . . 5

1.8 Static partitioning: work per range is uneven, so most threads idle while the last one finishes. . . . 6

1.9 Dynamic load balancing with a shared counter. . . . 7

1.10 Two increments overlap; both threads receive the same number and one update is lost. 8

1.11 The Flag Principle: whoever looks last must see both flags raised, so they cannot both enter. . . . 10

1.12 Producer and consumer strictly take turns on a single reusable bit. . . . 12

1.13 A reader that overlaps a tile-by-tile write sees a message nobody wrote. . . . 12

2.1 Lecture outline. . . . 14

2.2 Only the parallel part shrinks when processors are added; the serial part stays. . . . 15

2.3 Amdahl's Law: every curve flattens at its ceiling $1/(1-p)$ no matter how many processors are added. . . . 16

2.4 Real parallel execution interleaves useful work with overheads. . . . 17

2.5 Recursion tree of FIB(5). The two subtrees of each node are independent and could run in parallel. . . . 17

2.6 Computation dag of P-FIB(4). Each non-leaf call has 3 strands (before spawn, between spawn and sync, after sync). Work $= 17$, span $= 8$. . . . 19

2.7 For P-FIB(4), speedup is capped by both P (Work Law) and the parallelism $17/8$ (Span Law). . . . 20

2.8 Task-level parallelism shortens one job; data-level parallelism pushes more jobs through. . . . 21

2.9 A cost vs problem size graph: scalable systems grow cost roughly in proportion to problem size. . . . 22

2.10 Distribution of the sample mean ($\sigma = 5$): the standard error $\sigma/\sqrt{n}$ shrinks as n grows. 23

2.11 Error bars communicate measurement uncertainty; box plots summarise median, quartiles and outliers. . . . 24

3.1 Resource-allocation graph for the semaphore deadlock: a cycle of "holds" and "waits for" edges. . . . 28
3.2 The dining philosophers: five philosophers, five forks. . . . 29
3.3 A thread as a state machine: events are transitions between states. . . . 30
3.4 Two overlapping thread timelines and the single interleaved sequence of their events. 31
3.5 Two intervals are either disjoint (ordered by precedence) or overlapping (unordered). 31
3.6 Interleavings as lattice paths: moving right executes the next P1 action, moving up the next P2 action. . . . 33
3.7 The lost-update interleaving on a load-store machine: final `n == 1`, not 2. . . . 34
3.8 A safety violation can be observed at a point; a liveness violation never can. . . . 36

4.1 A shared-memory system: every core can reach every memory location. . . . 38
4.2 Fork/join: threads are created, run concurrently, and are joined back into the main thread. . . . 40
4.3 Block-row decomposition of $y = Ax$ for 3 threads. Every thread reads all of x but writes only its own y entries. . . . 42
4.4 A thread's life around a mutex. Blocked threads sleep instead of spinning. . . . 44
4.5 Run-times, $n = 10^8$ terms, 2×4-core processors. . . . 45
4.6 A sorted linked list of ints. . . . 45
4.7 Insert splices a new node between `pred_p` and `curr_p`. . . . 46
4.8 Thread A deletes node 5 while Thread B is reading it: a use-after-free. . . . 47
4.9 Hand-over-hand (lock-coupling) traversal with a mutex per node (red = locked). . . . 47
4.10 Readers overlap freely; a writer waits for all readers to leave and then excludes everyone. . . . 48
4.11 Linked-list run-times (log scale). The RW lock only wins when reads dominate. . . . 49
4.12 A parallel region: the team is created, every thread runs the block, and an implicit barrier ends the region. . . . 51
4.13 $n = 24$ trapezoids split into 4 contiguous blocks of 6, one block per thread. . . . 52

5.1 Strict alternation forces P1, P2, P1, P2…; if P2 stops wanting the CS, P1 is blocked forever. . . . 56
5.2 Attempt 2: both processes pass the test before either sets its flag. . . . 58
5.3 Attempt 3 deadlock: each waits for a flag the other will only lower after its CS. . . . 59
5.4 Livelock: the two flags toggle in perfect lock-step, so each process always sees the other's flag down. . . . 61
5.5 Dekker's pre-protocol for process i (the other process is j). . . . 62
5.6 Peterson: whoever writes `turn` last waits; the other proceeds. When the winner lowers its flag, the loser enters. . . . 63

5.7 The narrative arc of the four attempts. 64

6.1 Check-then-act on a bank balance. 67
6.2 A cycle in the resource-allocation graph is the circular-wait condition made visible. . 68
6.3 Case B: inconsistent lock order creates a cycle. 70
6.4 Peterson's lock for thread i. 73
6.5 The Filter lock as a funnel: each level filters out at least one contender. 74
6.6 Bakery: among interested threads, the smallest (label, id) pair goes first. 75
6.7 First-come-first-served: if A's doorway finishes before B's starts, A enters first. 76
6.8 The progression of lock algorithms. 77

7.1 An accelerator system: a host CPU and a GPU card, each with its own memory, joined by PCIe. 79
7.2 Where the transistors go: a few big, cache-backed cores vs. many rows of small ALUs with tiny control and cache. 80
7.3 GTX 480 vs. Core i7 BLAS: speedup rises then plateaus around 220–260×. 82
7.4 The CUDA software stack. 83
7.5 A kernel launch is a grid of blocks; each block is a group of threads with its own shared memory. 83
7.6 Mapping (block, thread) to a global index with `blockDim.x = 6`. 84
7.7 The host–device round trip for every kernel. 85
7.8 `add<<<N,1>>>`: one block (with a single thread) per vector element. 86
7.9 Row-major storage: rows are laid out one after another. 86
7.10 Each thread computes one element $a_{j,i}$; a block covers 32 consecutive columns of one row (schematic). 87
7.11 The launch returns at once; the host blocks only when it synchronises. 88
7.12 CUDA memory spaces: per-thread registers, per-block shared memory, grid-wide global and constant memory. 89
7.13 A barrier orders execution: no thread passes until every thread in the block arrives. . 91
7.14 Pageable vs. pinned transfers. 93
7.15 Knights Landing: tiles of 2 cores, each core with 2 AVX-512 VPUs and 4 hardware threads. 95
7.16 MCDRAM modes, selected at boot time in the BIOS. 95

8.1 A railway semaphore guarding a single section of track: a train may enter only when the arm signals clear. 98
8.2 The two atomic semaphore operations. 99
8.3 The invariant $\#CS + S = 1$ holds throughout an execution. 101
8.4 How waiting processes are held under the three semaphore definitions. 103
8.5 Rendezvous: whoever finishes first signals and then waits; both deliver together. . . 105

8.6 Asynchronous communication through a buffer. 106
8.7 A circular buffer of $N = 8$ slots. The two semaphores count occupied and empty slots. 107
8.8 Asymmetric fork ordering: everybody acquires forks in increasing index order. . . . 109
8.9 The lightswitch pattern: the first reader in turns the lock on, the last reader out turns it off. 110
8.10 The barber shop: chairs buffer *processes* rather than data. 111
8.11 Baboon crossing: one direction at a time, at most five on the rope. 112

9.1 Busy waiting + strict priorities: the spinner monopolises the CPU the lock holder needs. 116
9.2 From semaphores to monitors. 118
9.3 Anatomy of a monitor: a service queue to get in, one active process inside, and a FIFO queue per condition variable. 119
9.4 Who runs after B executes `signal(Q)` under each discipline (A waits on Q, C waits on the service queue). 122
9.5 Java thread states around an object's monitor. 123
9.6 The barging problem under signal-and-continue. 124
9.7 A multiplex: a counting semaphore initialised to N. 125
9.8 A barrier releases every process only when the last one arrives. 126
9.9 The ladder of synchronization constructs built in Lectures 5–9. 127

LECTURE 1

Introduction to Concurrent Programming

Overview

Why concurrency became unavoidable, what distinguishes *concurrent* from *parallel* and *distributed* computing, and the fundamental coordination problems (mutual exclusion, producer/consumer, readers/writers) that the rest of the course solves. Slides adapted from *The Art of Multiprocessor Programming* by Maurice Herlihy and Nir Shavit.

1.1 The Hardware Motivation

1.1.1 Moore's Law and the End of the Free Ride

What is still true and what broke

- **Transistor count** on a chip still roughly **doubles every 2 years**. This part of Moore's Law holds.
- **Clock speed has flattened sharply.** Power and heat limits stopped the frequency race (around 2004–2005).
- Manufacturers spend the extra transistors on **more cores**, not faster cores.

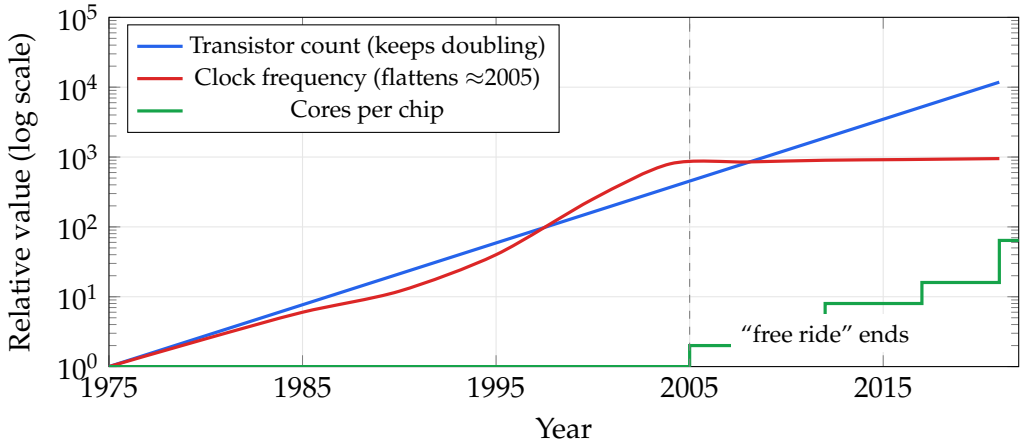


Figure 1.1: Illustrative trend (not exact data): transistor counts keep growing exponentially while clock frequency stalled; the extra transistors became extra cores.

Why do we care?

Time no longer cures software bloat. The **"free ride" is over**. If your program's path length doubles, you can no longer just wait 6 months for faster hardware. Your software must somehow **exploit twice as much concurrency**.

1.1.2 Migration in Architecture

	Uniprocessor	Shared Memory Multiprocessor (SMP)
CPUs	1	Many
Cache	1 cache	1 cache per CPU
Interconnect	Direct	Shared **bus**
Memory	Private	**Shared memory**

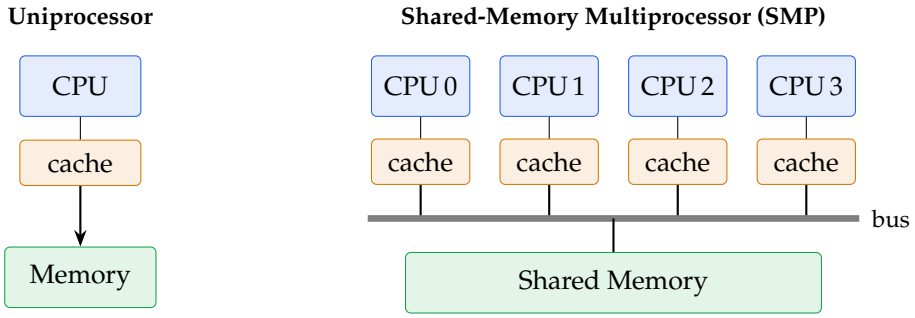


Figure 1.2: From a single CPU with private memory to many CPUs with private caches sharing one memory over a bus.

The shift

Multiple CPUs, each with a private cache, all connected via a bus to one shared memory. The shared memory is exactly what makes coordination necessary and hard.

1.1.3 Three Scaling Processes

Speedup over successive hardware generations

Scenario	Gen 1	Gen 2	Gen 3
Traditional (uniprocessor)	1.8x	3.6x	7x
Multicore (ideal)	1.8x	3.6x	7x
Multicore (real world)	1.8x	2x	2.9x

The ideal multicore column is a fantasy. In reality **parallelization and synchronization require great care**, and the gains collapse.

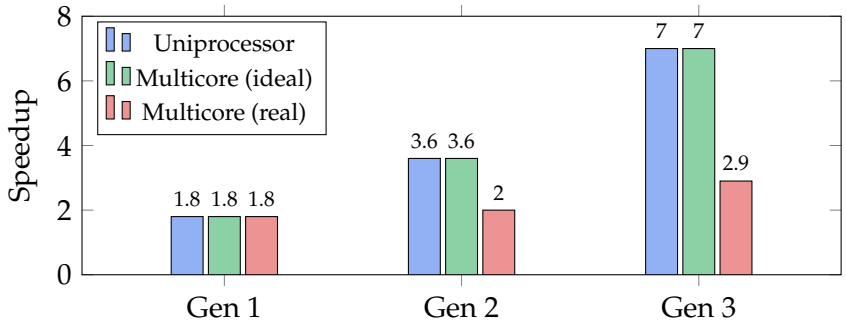


Figure 1.3: Real multicore speedup falls far short of the ideal.

1.2 Defining the Field

Definition

Parallel and concurrent computing "is a form of computation in which many calculations are carried out simultaneously by many threads of execution."

1.2.1 Concurrent vs Parallel vs Distributed

Term	Definition
Concurrent computing	Multiple tasks can be **in progress** at any instant in a program. They need not run at the same physical moment.
Parallel computing	Multiple tasks **cooperate closely** to solve a problem within a program, genuinely at the same time.
Distributed computing	A program needs to cooperate with **other programs** to solve a problem.

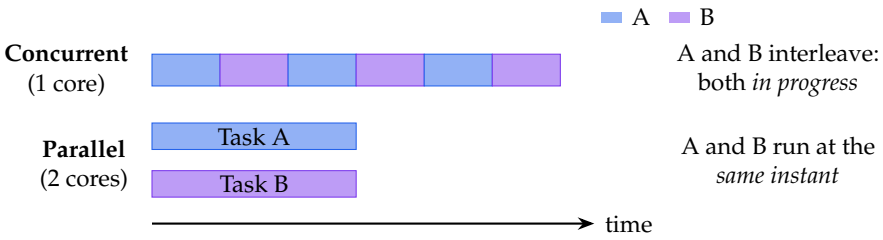


Figure 1.4: Concurrency is about structure (time-slicing on one core); parallelism is about execution (simultaneous on many cores).

Mental model

Concurrency is about structure (dealing with many things at once). **Parallelism is about execution** (doing many things at once). A single core can be concurrent but not parallel.

1.2.2 Sequential vs Concurrent Computation

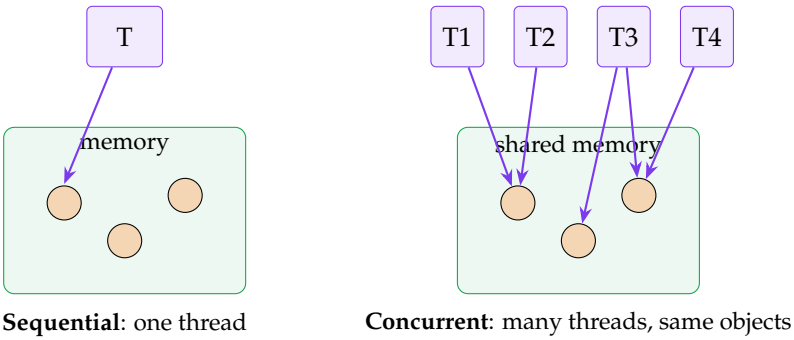


Figure 1.5: Every difficulty in this course flows from the right-hand picture.

1.2.3 What This Course Covers

- Theory around parallelism
- Tools and tricks
- Use cases and potential applications
- Common solution patterns

1.3 Asynchrony

Threads suffer sudden unpredictable delays

Cause	Typical magnitude
Cache miss	Short
Page fault	Long
Scheduling quantum used up	Really long

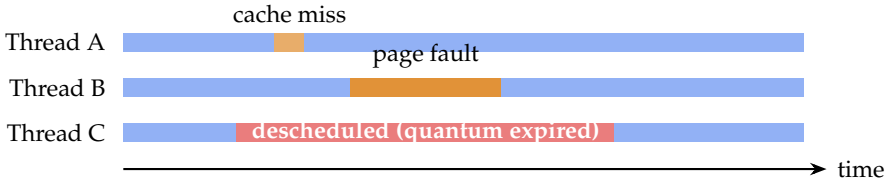


Figure 1.6: Asynchrony: any thread can stall at any moment for an unpredictable length of time.

Consequences you must design around

- Threads are scheduled by the **OS and thread library**, not by you.
- Execution can be **stopped at any time**, and another thread may be started.
- **No timing guarantees are given.** Algorithms must not make *any* assumption about speed of execution.

An algorithm that is only correct "because that instruction is fast" is not correct.

1.4 Why Parallel/Concurrent Computing?

Four motivations

1. **Only way to solve some problems.** Rendering animations, weather prediction, Big Bang simulation. *Ratatouille took 1532 CPU-years to render.*
2. **Most real-world problems are inherently parallel.** Weather, galaxies, bee/ant colonies, traffic.
3. **To fully utilize your computer.** Computers often spend time waiting on I/O, which takes far longer than CPU processing. Unless your application is pure number crunching, parallelism gives large gains.
4. **To make use of the cloud.** The cloud places hundreds to thousands of machines at ordinary people's fingertips.

1.5 Flynn's Taxonomy

Flynn classifies machines by the number of concurrent **instruction** streams and **data** streams (S = single, M = multiple).

Class	Meaning	Examples
SISD	Single Instruction, Single Data	Normal sequential programs, uniprocessor
SIMD	Single Instruction, Multiple Data	**Data parallelism**, GPUs, vector processors
MISD	Multiple Instruction, Single Data	Fault-tolerant schemes
MIMD	Multiple Instruction, Multiple Data	**Most parallel programs**, multi-core

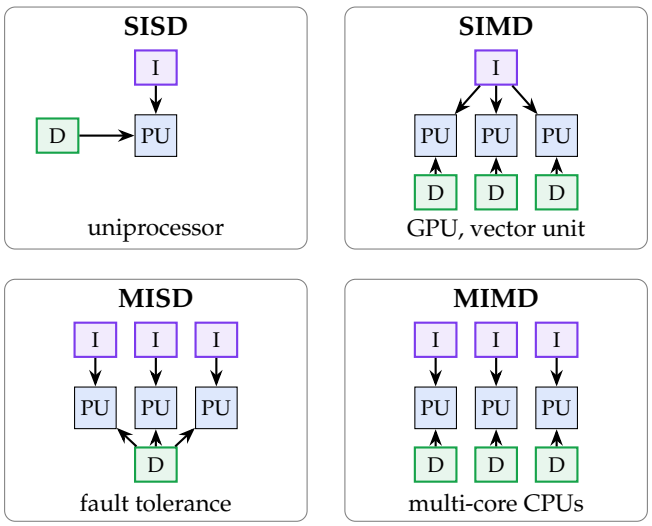


Figure 1.7: Flynn's taxonomy. I = instruction stream, D = data stream, PU = processing unit.

Remember

Your multi-core laptop running threads is **MIMD**. Your GPU shader is **SIMD**. MISD is the odd one out and exists mainly for redundancy and fault tolerance.

1.6 Road Map of the Course

Principles first, then practice

- Start with **idealized models**
- Look at **simplistic problems**
- Emphasize **correctness over pragmatism**

The toy problems are deliberate. If you cannot get a counter right, you cannot get a database right.

1.7 Case Study: Parallel Primality Testing

Challenge

Print all primes from 1 to 10^{10}**.** Given: a 10-processor multiprocessor, 1 thread per processor. Goal: get a 10-fold speedup, or close to it.

1.7.1 Attempt 1: Static Load Balancing

Split the range evenly. Each thread tests a range of 10^9 numbers:

$$\text{Thread } i \text{ handles } [\, i \cdot 10^9 + 1,\ (i+1) \cdot 10^9 \,]$$

```
void primePrint {
  int i = ThreadID.get();                    // IDs in {0..9}
  for (j = i*1000000000+1; j < (i+1)*1000000000; j++) {
    if (isPrime(j))
      print(j);
  }
}
```

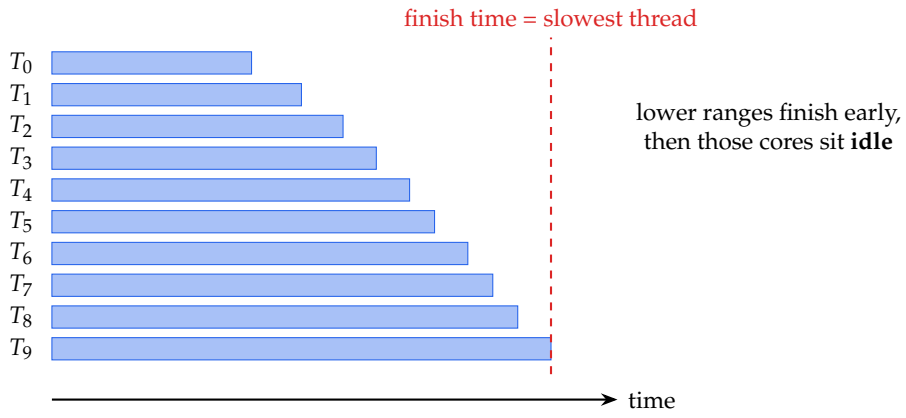


Figure 1.8: Static partitioning: work per range is uneven, so most threads idle while the last one finishes.

Why this fails

- Higher ranges have **fewer primes**.
- Yet larger numbers are **harder to test**.
- Thread workloads are therefore **uneven**.
- We need **dynamic load balancing**.

1.7.2 Attempt 2: Shared Counter

Each thread repeatedly "takes a number" from a shared counter, like a queue ticket dispenser.

```
Counter counter = new Counter(1);        // shared counter object

void primePrint {
  long j = 0;
  while (j < 10000000000L) {             // stop when every value taken
    j = counter.getAndIncrement();       // increment & return each new value
    if (isPrime(j))
      print(j);
  }
}
```

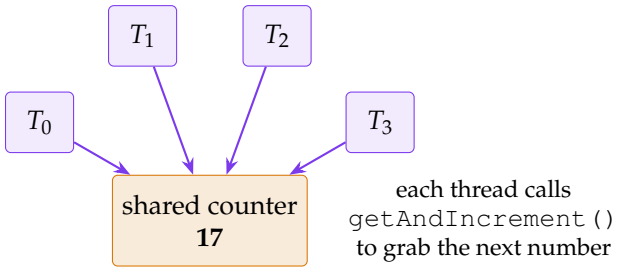


Figure 1.9: Dynamic load balancing with a shared counter.

1.7.3 What the Counter Really Does

```
public class Counter {
  private long value;

  public long getAndIncrement() {
    return value++;
  }
}
```

value++ is not one operation

The compiler expands it into three separate steps:

```
temp  = value;
value = temp + 1;
return temp;
```

Any thread can be preempted between any two of these steps.

1.7.4 The Race Condition

Step	Thread A	Thread B	`value`
1	read 1		1
2		read 1	1
3	write 2		2
4		write 2	2

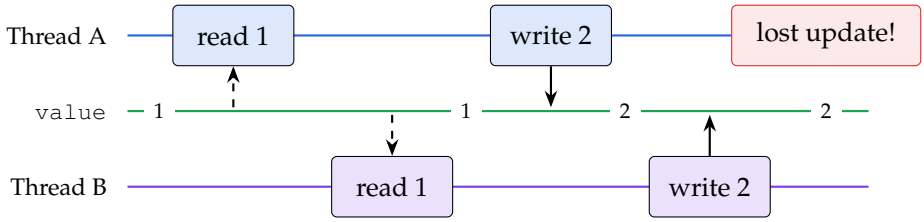


Figure 1.10: Two increments overlap; both threads receive the same number and one update is lost.

Not so good

Both threads receive **the same number**. The sequence produced is 1, 2, 3, 2 instead of 1, 2, 3, 4. **An update is silently lost.**

Is the problem inherent?

Yes. Whenever a **read** and a **write** from different threads overlap, the outcome depends on ordering. Everyday analogies: two people walking onto a narrow passage from opposite ends, or two fielders both running to catch the same ball in cricket.

1.7.5 The Fix: Atomicity

Requirement

Make these steps **atomic** (indivisible): `temp = value; value = temp + 1; return temp;`

Two ways to obtain atomicity:

Hardware solution

A single **ReadModifyWrite()** machine instruction performs the read, the modify, and the write as one indivisible operation.

Software/language solution (Java)

```
public class Counter {
  private long value;

  public long getAndIncrement() {
    synchronized {
```

```
        temp = value;
        value = temp + 1;
      }
      return temp;
    }
  }
```

The **synchronized block** provides **mutual exclusion**: at most one thread inside at a time.

1.8 The Alice and Bob Fable

Setup

Alice and Bob share a pond. Alice has a pet, Bob has a pet, and **the pets do not get along**. The pond must never contain both pets at once. This is **mutual exclusion** in disguise.

1.8.1 Formalizing the Problem

Property	Slogan	Meaning	Examples
Safety	"Nothing bad happens **ever**"	Consistency; an invariant that always holds	Mutual exclusion
Liveness	"Something good happens **eventually**"	Progress is made	No deadlock, no starvation

Efficiency matters too, and solutions are graded on all three.

Our specific requirements

- **Mutual Exclusion (safety):** both pets are never in the pond simultaneously.
- **No Deadlock (liveness):** if only one wants in, it gets in. If both want in, one gets in.

1.8.2 Three Failed Protocols

Protocol 1 — Just look at the pond

Idea: look at the pond before releasing your pet. **Gotcha:** trees obscure the view. **Interpretation:** threads cannot "see" what other threads are doing. **Explicit communication is required** for coordination.

Protocol 2 — Cell phone

Idea: Bob calls Alice, or vice versa. **Gotcha:** Bob is in the shower, Alice's battery is charging, Bob is out shopping for pet food. **Interpretation: message passing does not work** here. The recipient might not be listening, or might not be there at all. Communication must be **persistent**

(like writing), not **transient** (like speaking).

Protocol 3 — Cans and strings

Idea: cans sit on Alice's window sill with strings leading to Bob's house. Bob pulls a string to knock over a can. Alice can look at the cans whenever she wants. **Gotcha:** cans **cannot be reused**, and Bob runs out of cans because the window sill has limited space. **Interpretation:** you **cannot solve mutual exclusion with interrupts**. A sender setting a fixed bit in the receiver's space, with the receiver resetting it when ready, would require an **unbounded number of interrupt bits**.

1.8.3 Working Protocol: Flags

Alice	Bob
Raise flag	Raise flag
Wait until Bob's flag is down	**While** Alice's flag is up: lower own flag, wait for Alice's flag to go down, raise own flag
Unleash pet	Unleash pet
Lower flag when pet returns	Lower flag when pet returns

Bob defers to Alice. The asymmetry is what breaks the tie.

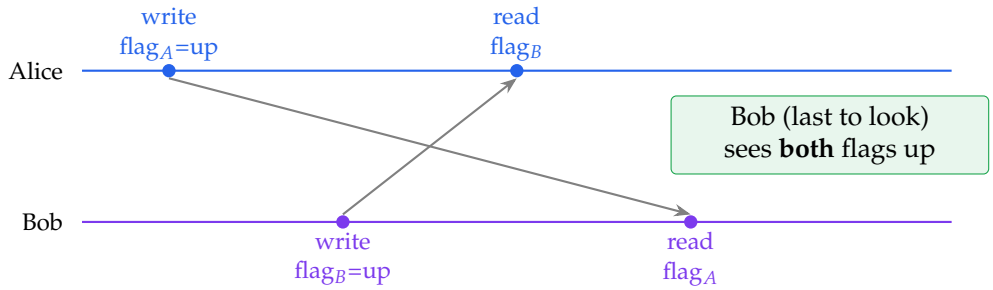


Figure 1.11: The Flag Principle: whoever looks last must see both flags raised, so they cannot both enter.

The Flag Principle

1. Raise your flag.
2. Look at the other's flag.

If each raises and then looks, the last one to look must see both flags up. From this you can prove:

- **Mutual exclusion** is satisfied (by deriving a contradiction).
- **No deadlock** occurs (if Bob sees Alice's flag, he gives her priority).

Remaining flaws

- The protocol is **unfair**: Bob's pet might never get in.
- The protocol **uses waiting**: if Bob is eaten by his pet, Alice's pet might never get in.

Waiting is problematic in general. **If one participant is delayed, so is everyone else**, and delays are common and unpredictable.

Moral of the story

Mutual exclusion **cannot** be solved by message passing (transient communication, e.g. cell phones) or by interrupts (tin cans). It **can** be solved by **one-bit shared variables that can be read or written**, that is, a flag.

1.9 Producer/Consumer

The fable continues

Alice and Bob fall in love, marry, then divorce. She gets the pets, he has to feed them. They cannot meet because of restraining orders. So Bob puts food in the pond, and later Alice releases the pets.

What must be avoided

- Releasing pets when there is **no food**.
- Putting out food when **uneaten food remains**.

Needed mechanism: Bob lets Alice know when food has been put out, and Alice lets Bob know when to put out more.

1.9.1 The Solution: A Single Can

```
// Bob's code (producer)
while (true) {
  while (can.isDown()) {};      // wait until pets have eaten
  pond.stockWithFood();
  can.knockOver();              // signal: food is ready
}

// Alice's code (consumer)
while (true) {
  while (can.isUp()) {};        // wait until food is ready
  pet.release();
  pet.recapture();
  can.reset();                  // signal: pond is empty
}
```

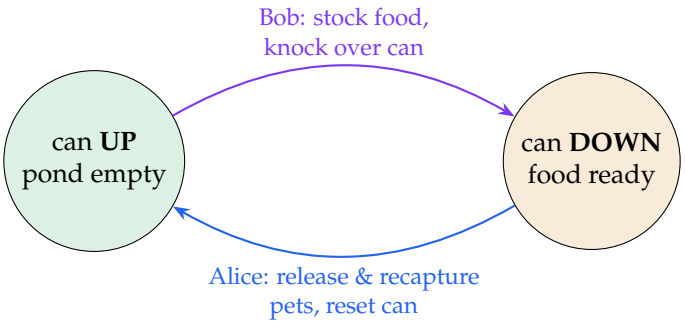


Figure 1.12: Producer and consumer strictly take turns on a single reusable bit.

Correctness properties

Property	Statement	Type
Mutual Exclusion	Pets and Bob are never together in the pond	Safety
No Starvation	If Bob is always willing to feed and the pets are always famished, the pets eat infinitely often	Liveness
Producer/Consumer	Pets never enter the pond unless there is food, and Bob never provides food if unconsumed food remains	Safety

But did we not say interrupts cannot solve mutual exclusion?

This problem is **different**:

- Alice and Bob **need to cooperate**. If not, the pets die and Bob goes to jail. Therefore **resetting the interrupt bit (the tin can) cannot be postponed**.
- Producer and consumer **take turns**, exactly like setting and resetting a single interrupt bit.

One reusable bit is enough here, whereas mutual exclusion needed unboundedly many.

1.10 Readers/Writers

The fable drags on

Bob and Alice still need to communicate, so they use billboards made of Scrabble letter tiles. The billboard is large, so only **one letter can be written at a time**.

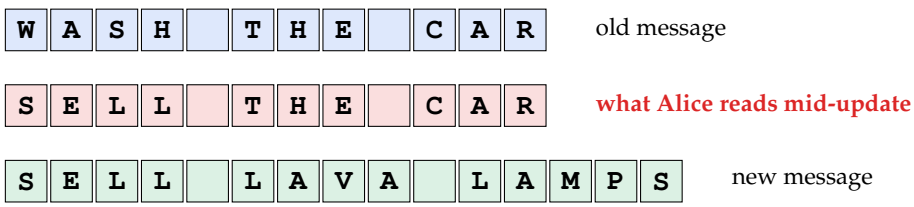


Figure 1.13: A reader that overlaps a tile-by-tile write sees a message nobody wrote.

The problem

Bob writes `WASH THE CAR` one tile at a time. Later he writes `SELL LAVA LAMPS` over it, again one tile at a time. Alice reads mid-update and sees `SELL THE CAR`. **A mixed message that was never intended by anyone.**

Requirement

Devise a protocol so that the **writer** writes one letter at a time, the **reader** reads one letter at a time, and the reader sees **either the old message or the new message**, never a mixture.

The tension

This is easy with mutual exclusion, but mutual exclusion requires **waiting**: one waits for the other and everything executes sequentially. Remarkably, **R/W can be solved without mutual exclusion**. That result comes later in the course.

1.11 Summary

Lecture Summary

- **Why concurrent programming:** clock speeds stopped rising, so performance now comes from exploiting more cores.
- **Concurrent vs parallel vs distributed:** in progress together, cooperating together, and cooperating across programs.
- **Asynchrony:** no timing assumptions are ever safe.
- **Two fundamental interactions:** mutual exclusion and cooperation.
- **Requirements to prove:** Safety, Liveness, and Efficiency.
- **What to avoid:** Race Conditions, Deadlocks, and Starvation.

Self-check Questions

1. Which half of Moore's Law still holds, and which half broke?
2. Give a one-line distinction between concurrent, parallel, and distributed computing.
3. Why does static range splitting fail for the primality problem?
4. Expand `value++` into its three underlying steps and construct an interleaving that loses an update.
5. Classify each as safety or liveness: mutual exclusion, no deadlock, no starvation, producer/consumer ordering.
6. Why does the cell phone protocol fail, and what property must communication have instead?
7. State the Flag Principle and explain why it implies mutual exclusion.
8. Why can a single tin can solve producer/consumer when tin cans cannot solve mutual exclusion?

LECTURE 2

How Much Parallelism?

Overview

How much faster can a parallel program actually get? This lecture covers Amdahl's Law, the cost and complexity of parallelism, the work/span model for multithreaded computation, and the metrics (speedup, efficiency, parallelism, throughput, latency, scalability) used to answer that question rigorously.



Figure 2.1: Lecture outline.

2.1 Amdahl's Law

Why we care

We want as much of the code as possible to execute concurrently. A **larger sequential part implies reduced performance**, and Amdahl's Law (1967) shows this relation is **not linear**.

2.1.1 Definition of Speedup

$$\text{Speedup} = \frac{\text{OldExecutionTime}}{\text{NewExecutionTime}}$$

This is the speedup of a computation given n CPUs instead of 1.

2.1.2 The Law

Amdahl's Law

$$\text{Speedup} = \frac{1}{(1-p) + \frac{p}{n}}$$

p	**parallel fraction** of the computation
$1-p$	**sequential fraction** of the computation
n	**number of processors**

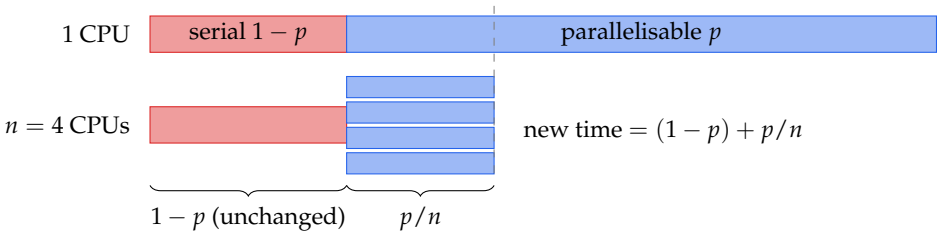


Figure 2.2: Only the parallel part shrinks when processors are added; the serial part stays.

2.1.3 Worked Examples (all with 10 processors)

Four parallel fractions

$$p = 0.60: \quad \frac{1}{1 - 0.6 + \frac{0.6}{10}} = \frac{1}{0.46} = 2.17$$

$$p = 0.80: \quad \frac{1}{1 - 0.8 + \frac{0.8}{10}} = \frac{1}{0.28} = 3.57$$

$$p = 0.90: \quad \frac{1}{1 - 0.9 + \frac{0.9}{10}} = \frac{1}{0.19} = 5.26$$

$$p = 0.99: \quad \frac{1}{1 - 0.99 + \frac{0.99}{10}} = \frac{1}{0.109} = 9.17$$

Parallel fraction p	Sequential fraction	Speedup on 10 CPUs	% of ideal 10x
0.60	0.40	2.17	22%
0.80	0.20	3.57	36%
0.90	0.10	5.26	53%
0.99	0.01	9.17	92%

Notice how brutally the last few percent of serial code punish you.

2.1.4 The Asymptotic Limit

Maximum speedup with infinite processors

As $n \to \infty$, the $\frac{p}{n}$ term vanishes:

$$\text{Speedup}_{\max} = \frac{1}{1-p}$$

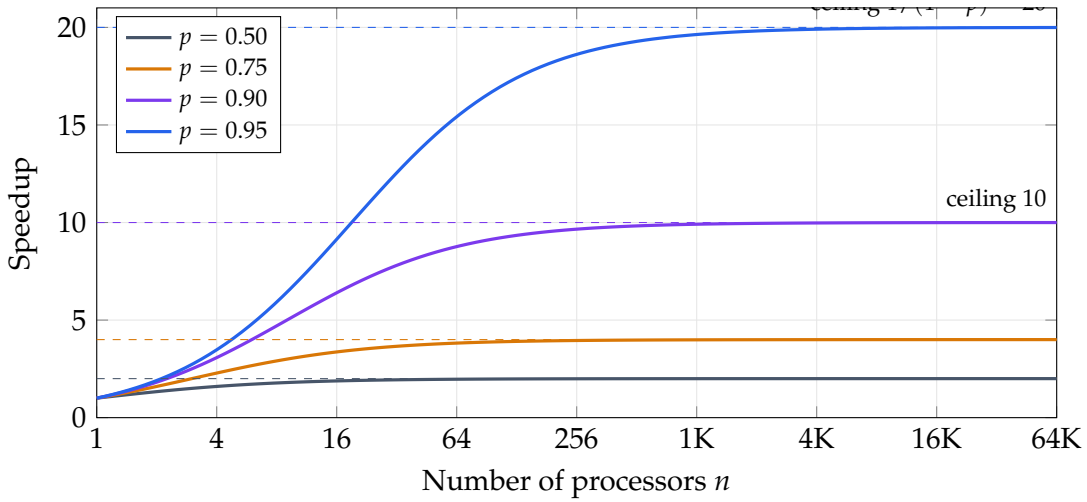


Figure 2.3: Amdahl's Law: every curve flattens at its ceiling $1/(1-p)$ no matter how many processors are added.

The hard ceiling

Even with an **unlimited** number of processors, maximum speedup is limited to $\frac{1}{1-p}$. For example, with only **5% of the computation being serial**, the maximum speedup is **20**, no matter how many processors you throw at it.

The moral — what good use of multiple cores means

- Find ways to **effectively parallelize** your code.
- **Minimize sequential parts.** It is worth the effort to parallelize even the last 10% of serial code.
- **Reduce idle time** in which threads wait without executing.

This is what the course is about: the percentage that is **not easy** to make concurrent yet has a **large impact** on overall speedup.

2.2 Costs and Complexity of Parallelism

2.2.1 Costs

Overheads that Amdahl's Law does not even include

- Task **start-up** time
- **Synchronizations**
- **Data communications**
- Software overhead imposed by parallel compilers, libraries, tools, operating system
- Task **termination** time



Figure 2.4: Real parallel execution interleaves useful work with overheads.

The efficiency trade-off

Parallel programs have **efficiency** < 1, which means they **waste resources**. For small programs the additional cost is prohibitive. **Parallel programming lets us get faster results at the cost of efficiency.** It lets us do a 1-CPU-year problem within a day by using more CPUs.

2.2.2 Complexity

Parallel programs are more complex than their serial counterparts

Complexity is measured in **programmer time** across the lifecycle: Design → Coding → Debugging → Tuning → Maintenance. They must yield **significant** improvement to justify the costs. **Using parallelism to achieve a 10 to 20% gain is not useful.**

2.3 Case Study: Fibonacci

2.3.1 Serial Version

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i \geq 2, \qquad F_0 = 0,\ F_1 = 1$$

```
FIB(n)
    if n <= 1 return n
    else
        x ← FIB(n - 1)
        y ← FIB(n - 2)
        return x + y
```

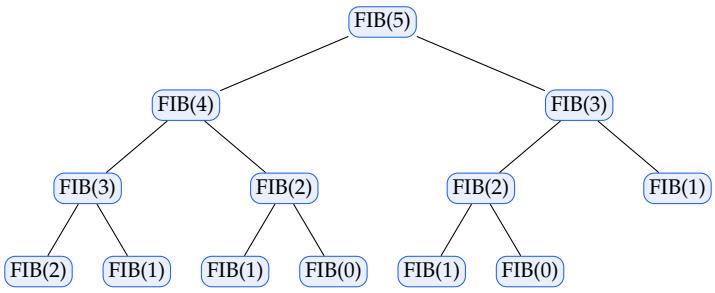


Figure 2.5: Recursion tree of FIB(5). The two subtrees of each node are independent and could run in parallel.

Can we parallelize the computation of F_{10000}?

The recursion tree shows that `FIB(n-1)` and `FIB(n-2)` are independent subproblems, so they *can* run at the same time. Whether that helps is the real question.

2.3.2 Parallel Version with Dynamic Multithreading

```
P-FIB(n)
    if n <= 1 return n
    else
        x ← spawn P-FIB(n - 1)
        y ← P-FIB(n - 2)
        sync
        return x + y
```

Keywords

- **spawn**: the child procedure may execute **in parallel** with the parent. It does not force parallel execution, it only permits it.
- **sync**: the parent waits until all its spawned children have completed.

2.4 The Computation DAG Model

Model for multithreaded execution

Think of a multithreaded computation as a **computation dag** (directed acyclic graph):

Element	Meaning
Vertex	A **strand**: a chain of instructions with no parallel control
Edge (u, v)	Strand u must execute before v
Strand with 2 successors	One of them must have been **spawned**
Strand with multiple predecessors	Predecessors joined because of a **sync**

Series and parallel

If a computation dag has a **directed path** from strand u to strand v, then u and v are **logically in series**. Otherwise they are **logically in parallel**.

Idealized machine assumptions

A set of processors of **equal power**; **consistent shared memory**; **ignore** the cost of scheduling.

2.5 Performance Measures: Work and Span

The two fundamental quantities

Work — total time to execute on **1 processor**. The sum of the times taken by each strand, i.e. the **number of vertices in the DAG**.
Span — longest time to execute strands along **any path** in the DAG. If each strand takes unit time, the span is the **number of vertices along the critical path**.

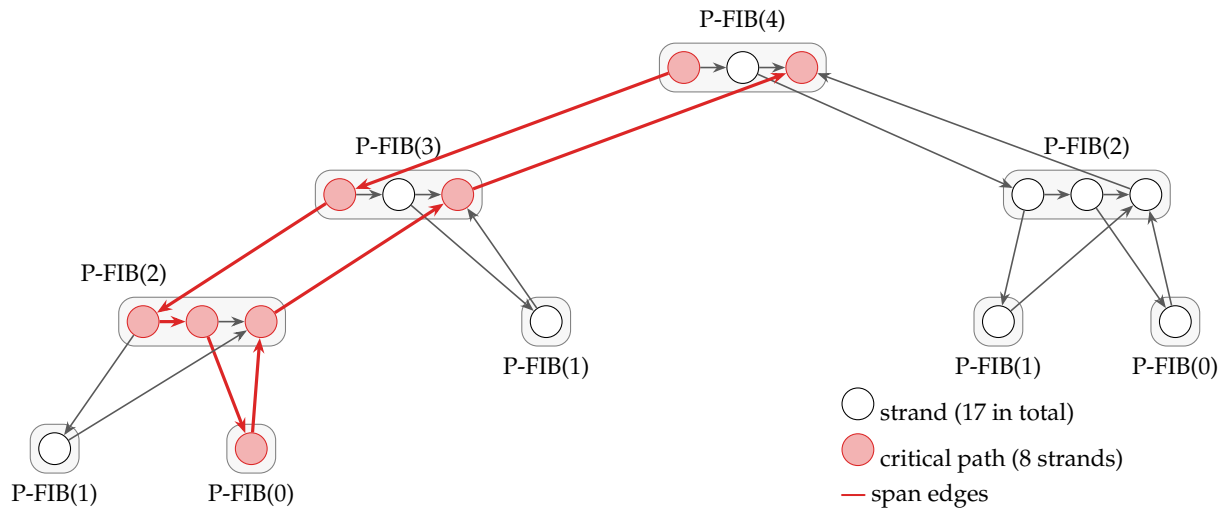


Figure 2.6: Computation dag of P-FIB(4). Each non-leaf call has 3 strands (before spawn, between spawn and sync, after sync). Work = 17, span = 8.

Worked example: P-FIB(4) dag

- Total **17 vertices** ⇒ **Work = 17** time units. (4 internal calls × 3 strands + 5 leaf calls × 1 strand = 17.)
- **8 vertices** on the critical path ⇒ **Span = 8** time units.

Actual running time also depends on the number of processors available and on how the scheduler allocates strands to processors.

2.5.1 Notation

Symbol	Meaning
T_P	Running time on P processors
T_1	Running time on 1 processor = **Work** (time of the best serial algorithm)
T_∞	Running time if each strand can run on its own processor = **Span** (infinite processors)

We want $T_P \leq T_1$. Work and span provide **lower bounds** on the running time T_P of a multithreaded computation on P processors.

2.6 Speedup, Efficiency, and Parallelism

Speedup

$$\text{Speedup} = \frac{T_1}{T_P}$$

How many times faster the computation is on P processors than on 1 processor. Speedup is **at most** P: $\frac{T_1}{T_P} \leq P$.

$T_1/T_P = \Theta(P)$ **linear speedup**

$T_1/T_P = P$ **perfect linear speedup**

Efficiency

$$\text{Efficiency} = \frac{T_1}{P \cdot T_P}, \qquad 0 < \text{Efficiency} \leq 1$$

Efficiency measures **how much we lost by parallelizing**, for example due to communication cost.

The Work Law

- In 1 step, an ideal computer with P processors can do at most P units of work.
- In T_P time it can do at most $P \times T_P$ work.
- As the total work to do is T_1: $P \times T_P \geq T_1$, hence

$$\boxed{T_P \geq \frac{T_1}{P}}$$

Similarly the **Span Law**: $T_P \geq T_\infty$, because P processors can never beat infinitely many.

2.6.1 Parallelism

Parallelism

$$\text{Parallelism} = \frac{T_1}{T_\infty} = \frac{\text{Work}}{\text{Span}}$$

Three ways to interpret it

- **As a ratio:** the average amount of work that can be performed in parallel for each step along the critical path.
- **As an upper bound:** the maximum possible speedup on **any** number of processors.
- **As a limit on scaling:** if the number of processors **exceeds** the parallelism, you cannot possibly achieve perfect linear speedup.

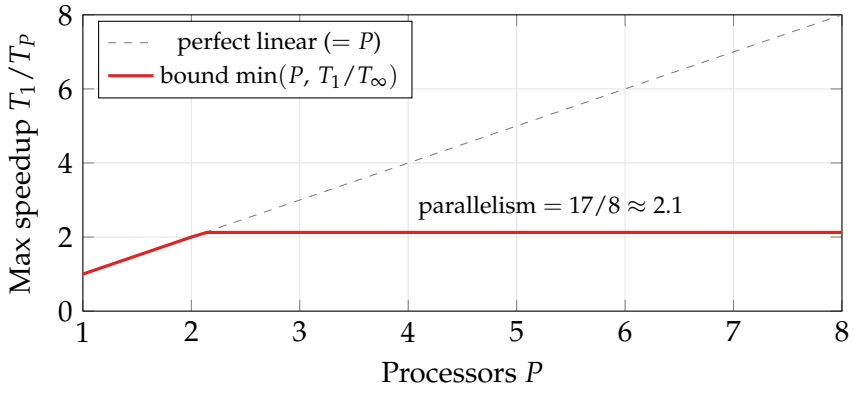


Figure 2.7: For P-FIB(4), speedup is capped by both P (Work Law) and the parallelism $17/8$ (Span Law).

Applying it to P-FIB(4)

Work $= 17$, Span $= 8$:

$$\text{Parallelism} = \frac{17}{8} = 2.125$$

Interpretation: no matter how many processors are used for the computation of `P-FIB(4)`, the speedup will not be more than about 2.

2.7 Throughput, Latency, and Resources

Situation	Latency	Throughput
Cost of a problem ↑	↑	↓
Problem parallelizable at **task level**, resources ↑	↓	↑
Problem parallelizable at **data level**, resources ↑	unchanged	↑

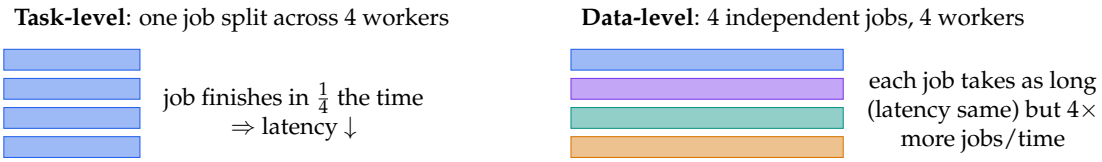


Figure 2.8: Task-level parallelism shortens one job; data-level parallelism pushes more jobs through.

Reading this table

Task-level parallelism speeds up a *single* job, so latency falls. Data-level parallelism lets you push *more* jobs through, so only throughput rises.

2.8 Scalability

Definition

Say there is a solution A for a problem of size k. Typically, to solve a larger version of A, we add more resources such as more memory or more CPU. Most problems **do not** behave that way, and when they do it is a very important property called **scalability**. Scalability is measured through a **cost vs problem size** graph.

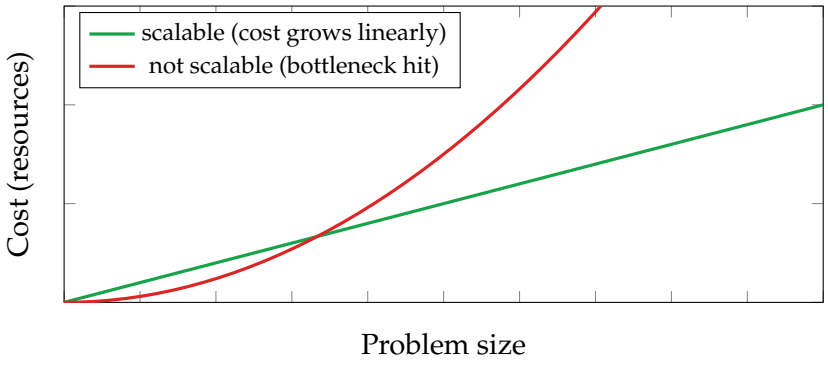


Figure 2.9: A cost vs problem size graph: scalable systems grow cost roughly in proportion to problem size.

Scalability is about bottlenecks

Common bottlenecks: CPU; memory–CPU **bus bandwidth** on an SMP machine; communications **network bandwidth**; available **memory** on any given machine or set of machines. We push the limits imposed by bottlenecks through architecture and design, but **when one bottleneck goes away, another takes its place**.

2.9 Measuring Performance

The honest position

We can **never** measure the real performance of a system. Yet we still try. To understand a system, **two readings** are required: **Latency** — time to finish **1 instance** of the problem; **Throughput** — number of instances that can be finished in **unit time**.

Which one should you measure?

Measure latency when: you have only 1 instance to run; the operation has a **user waiting** on it; **time-sensitive deadlines** are involved (e.g. predicting a storm as soon as possible).
Measure throughput when: latency is not important and **overall utilization** is more crucial. Sometimes you need **both**.

2.9.1 Statistical Basis

When you measure a system, you are taking a sample

Central Limit Theorem. When we draw n samples from a distribution with mean μ and variance σ^2, as n increases the distribution of the sample average approaches a **normal distribution** with mean μ and variance $\frac{\sigma^2}{n}$, **irrespective of the shape** of the original distribution.
This justifies reporting **confidence intervals and error bars**. More readings means a tighter confidence interval.

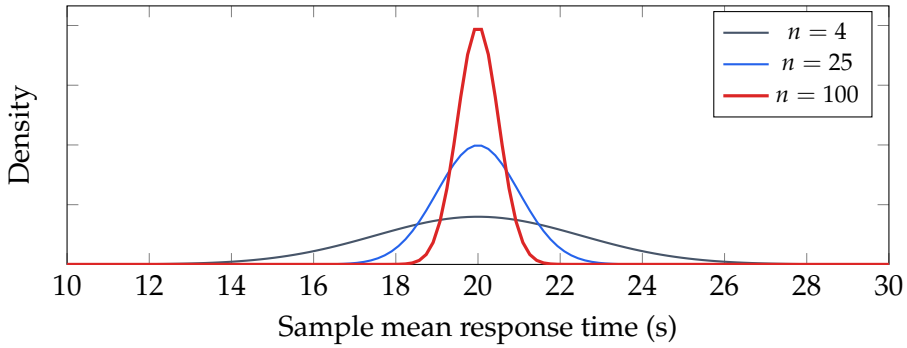


Figure 2.10: Distribution of the sample mean ($\sigma = 5$): the standard error $\sigma/\sqrt{n}$ shrinks as n grows.

2.9.2 How Many Samples?

Required number of observations

To get an accuracy of $\pm r\%$ at a confidence level of $100(1-\alpha)\%$:

$$n = \left(\frac{100 \cdot z \cdot s}{r \cdot \bar{x}}\right)^2$$

where $\bar{x}$ is the sample mean, s the sample standard deviation, and z the normal quantile ($z_{1-\alpha/2}$).

Worked example

Sample mean response time $\bar{x} = 20\,\text{s}$, sample standard deviation $s = 5$. How many repetitions are needed for response time accurate to within **1 second** at **95% confidence**?

- Required accuracy: $r = \frac{1}{20} = 5\%$
- $z = 1.960$ for 95% confidence

$$n = \left(\frac{100 \times 1.960 \times 5}{5 \times 20}\right)^2 = (9.8)^2 \approx 96$$

About **96 repetitions** are needed.

2.10 Data Presentation

Form	Strength
Numbers	Average, standard deviation, min, max, percentiles
Tables	Enable comparisons
Graphs	Easy to see trends, enable more complex comparisons

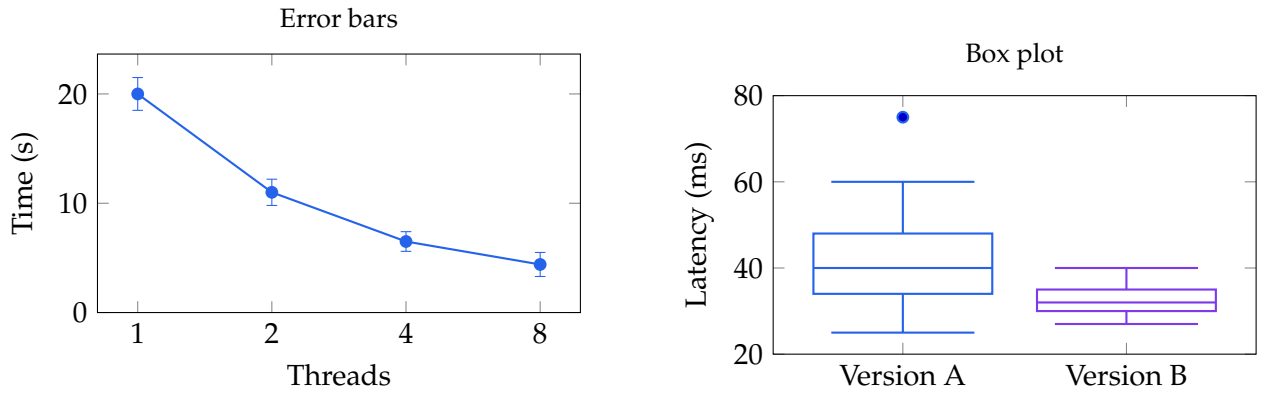


Figure 2.11: Error bars communicate measurement uncertainty; box plots summarise median, quartiles and outliers.

Graph rules

- Use a **suitable graph type** for the case under analysis and the data.
- Should have a **title or caption**.
- Axes properly titled **with units**: the **independent variable** always goes on the x-axis; **time** always on the x-axis; ranges may differ; tics large enough to cover the needed range without lots of extra space; **no need to start at zero**.
- Use a **key** to explain colours or symbols.
- Graph should **fill the available space**.
- **Error bars are encouraged** to indicate uncertainty in a measurement.

2.11 Summary

Lecture Summary

- **Amdahl's Law:** Speedup $= \frac{1}{(1-p)+p/n}$, with a hard ceiling of $\frac{1}{1-p}$.
- The **sequential fraction dominates**. 1% serial code caps you at 100x however many cores you own.
- Parallelism has real **costs** (startup, sync, communication, termination) and real **complexity** costs in programmer time.
- **Work** T_1 and **Span** T_∞ define a computation dag. **Parallelism** $= T_1/T_\infty$ upper-bounds any achievable speedup.
- **Work Law:** $T_P \geq T_1/P$. **Efficiency** $= T_1/(P \cdot T_P) \leq 1$.
- **Scalability is about bottlenecks**, and removing one exposes the next.
- Measure both **latency** and **throughput**, report **confidence intervals**, and plot honestly.

Self-check Questions

1. A program is 95% parallel. What is its speedup on 16 processors, and what is its maximum speedup ever?

2. Why does Amdahl's Law give an **optimistic** estimate in practice?
3. Define work and span for a computation dag, and compute both for a dag you draw for `P-FIB(5)`.
4. Derive the Work Law $T_P \geq T_1/P$.
5. If a computation has Work $= 40$ and Span $= 5$, what is the largest number of processors worth using?
6. Distinguish linear speedup from perfect linear speedup.
7. When would you optimise for throughput at the expense of latency?
8. State four common scalability bottlenecks.

LECTURE 3

Concurrent Programming Abstraction

Overview

The formal model underlying everything else in the course. **Part 1** covers the critical-section problem, deadlock and starvation, and the classical synchronization problems. **Part 2** builds the abstraction itself: events, threads as state machines, intervals, the precedence relation, interleaving, atomicity, and the two classes of correctness properties. Companion slides for *The Art of Multiprocessor Programming* by Maurice Herlihy and Nir Shavit.

Part 1: The Critical-Section Problem

3.1 Race Conditions

Definition

A **race condition** is the situation where several processes access and manipulate shared data concurrently. **The final value of the shared data depends upon which process finishes last.**

The remedy

To prevent race conditions, concurrent processes must be **synchronized**.

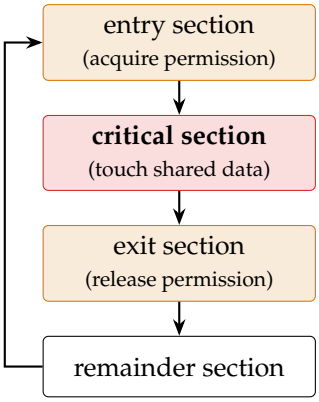
3.2 The Critical-Section Problem

Statement of the problem

- n processes all competing to use some **shared data**.
- Each process has a code segment, called the **critical section**, in which the shared data is accessed.
- **Problem:** ensure that when one process is executing in its critical section, **no other process is allowed to execute in its critical section**.

3.2.1 General Structure of a Process

```
// General structure of process
   Pi
do {
    entry section
        critical section
    exit section
        remainder section
} while (1);
```



Note

Processes may share some common variables to synchronize their actions. The entry and exit sections are exactly where that synchronization code lives.

3.3 Requirements for a Correct Solution

Three conditions any solution must satisfy

1. Mutual Exclusion. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. Progress. If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next **cannot be postponed indefinitely**.
3. Bounded Waiting. A bound must exist on the number of times that **other** processes are allowed to enter their critical sections **after** a process has made a request to enter its critical section and **before** that request is granted.

Assumptions

Each process executes at a **nonzero speed**. **No assumption** is made concerning the **relative speed** of the n processes.

How to remember the three

- **Mutual exclusion** stops the bad thing (a safety property).
- **Progress** stops the system from stalling when the section is free (a liveness property).
- **Bounded waiting** stops any individual from being starved forever (a liveness property, and stronger than progress).

3.4 Deadlock and Starvation

3.4.1 Deadlock

Definition

Deadlock occurs when two or more processes are waiting **indefinitely** for an event that only one of the waiting processes can cause.

Classic semaphore deadlock

Let S and Q be two semaphores initialised to 1.

Process P_0	Process P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
⋮	⋮
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

P_0 holds S and waits for Q. P_1 holds Q and waits for S. **Neither can ever proceed.** The cause is **inconsistent lock ordering**.

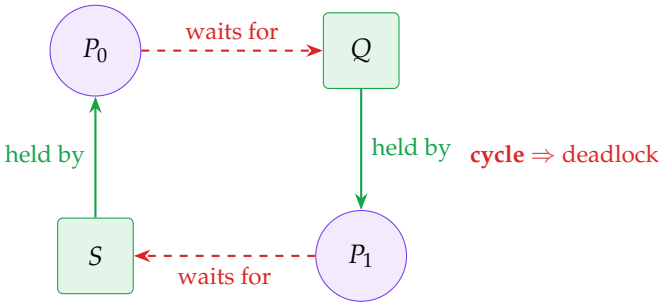


Figure 3.1: Resource-allocation graph for the semaphore deadlock: a cycle of "holds" and "waits for" edges.

3.4.2 Starvation

Definition

Starvation is indefinite blocking. A process is **perpetually denied** the necessary resources to process its work.

	Deadlock	Starvation
Who is stuck	**All** involved processes	**One** unlucky process
System progress	None at all	The system continues fine
Typical cause	Circular wait	Unfair scheduling or priority

3.5 Methods for Synchronization

Three mechanisms covered in this course

Locks, **Semaphores**, **Monitors**.

3.6 Classical Problems of Synchronization

Problem	Core difficulty
Bounded-Buffer (Producer–Consumer)	Producer must not overfill a finite buffer, consumer must not read from an empty one
Readers and Writers	Many readers may share, but a writer must be exclusive
Dining Philosophers	Acquiring multiple resources without circular wait

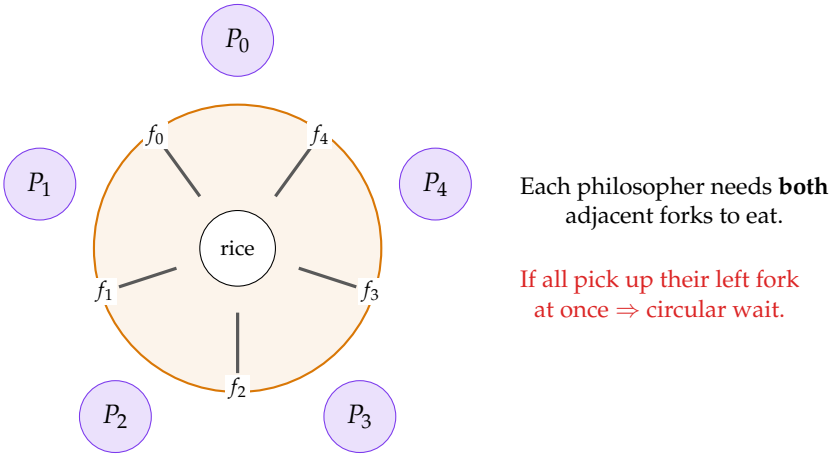


Figure 3.2: The dining philosophers: five philosophers, five forks.

Dining philosophers

Philosophers alternate between **eating** and **thinking**. Eating needs **2 forks**. A philosopher can pick up **only one fork at a time**. If every philosopher simultaneously picks up their left fork, all five hold one fork and wait forever for the second. **This is a textbook circular-wait deadlock.**

Part 2: Concurrent Programming Abstraction

3.7 The Abstraction

Core definition

The execution of a concurrent program consists of **multiple processes active at the same time**. Each process is the execution of a **sequential program**.

- If the computer has **multiple PEs** (processing elements), there is **parallel** or **real concurrent** execution.
- With a **single PE**, the processor is **switched among** the processes. This is called **pseudo-concurrent execution**, or **interleaving**.

The key simplification

Without loss of generality, we **always model concurrent execution as interleaved**, whether or not the implementation runs on multiple PEs. This single decision is what makes concurrency reasoning tractable. One model covers both hardware cases.

Only two interactions to consider

Each process is considered to be operating on **its own processor**, executing **its own program**. The only two possible interactions are: **Contention** — competing for a **shared resource**; and **Communication** — synchronizing, i.e. agreeing that a certain event has taken place.

3.8 Events, Threads, and States

Events

An event a_0 of thread A is **instantaneous**. There are **no simultaneous events** (ties are broken arbitrarily).

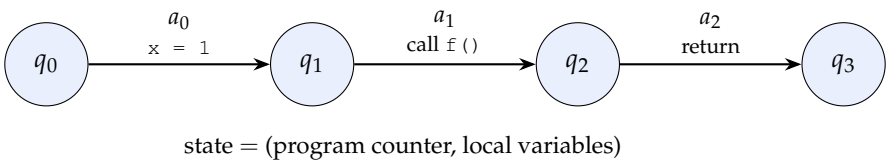
Formal definition of a thread

A thread A is a **sequence** of events $a_0, a_1, a_2, \ldots$ Notation: $a_0 \to a_1$ indicates **order**.

Example thread events

Assign to a **shared** variable; assign to a **local** variable; **invoke** a method; **return** from a method; lots of other things.

3.8.1 Threads are State Machines



state = (program counter, local variables)

Figure 3.3: A thread as a state machine: events are transitions between states.

Level	State consists of
Thread state	Program counter, local variables
System state	Object fields (shared variables), plus the **union of all thread states**

3.9 Concurrency and Interleavings

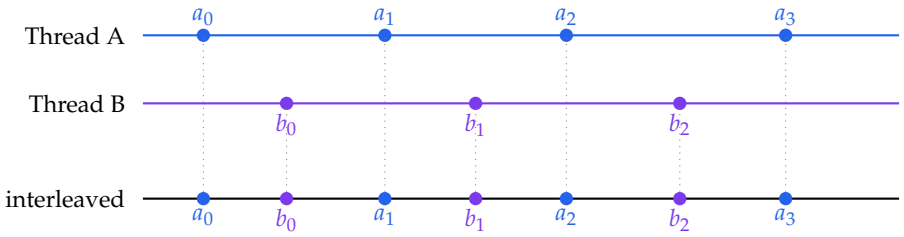


Figure 3.4: Two overlapping thread timelines and the single interleaved sequence of their events.

The picture

Thread A runs along its own timeline; thread B runs along its own timeline. Concurrency means those timelines **overlap in real time**. **Interleaving** means the events of two or more threads are woven into a single sequence, and those events are **not necessarily independent**.

3.10 Intervals and Precedence

Interval

An interval $A_0 = (a_0, a_1)$ is the **time between events** a_0 **and** a_1.



Figure 3.5: Two intervals are either disjoint (ordered by precedence) or overlapping (unordered).

The precedence relation

Interval A_0 **precedes** interval B_0, written $A_0 \to B_0$, if the **end event of** A_0 occurs before the **start event of** B_0. Also called **"happens before"**.

Intuition

$A_0 \to B_0$ is just like saying 1066 AD $\to$ 1492 AD. It is ordinary chronological ordering of non-overlapping time spans.

Properties of precedence ordering

- **Irreflexive:** it is never true that $A \rightarrow A$.
- **Antisymmetric:** if $A \rightarrow B$ then it is **not** true that $B \rightarrow A$.
- **Transitive:** if $A \rightarrow B$ and $B \rightarrow C$ then $A \rightarrow C$.

It is only a partial order

Overlapping intervals are **unordered**. Neither $A \rightarrow B$ nor $B \rightarrow A$ holds when they overlap. This is exactly why concurrency is hard: **there is no single global order of events to reason about.**

3.11 Interleaving

The central definition

Concurrent programming abstraction is the study of interleaved execution of the atomic instructions of sequential processes.

Rules of the model

- The **absolute time** taken by an atomic instruction is **ignored**.
- **Arbitrary interleavings are allowed.**
- The interleaving must be **fair**, that is no process is deferred forever.
- A concurrent program is required to be **correct under all fair interleavings**.

Consequence for debugging

Conventional **debugging to detect and correct errors is impossible**. You cannot test your way to correctness, because you cannot reproduce or enumerate the interleavings. You must **prove** correctness instead.

3.11.1 Counting Interleavings

Two processes, two actions each

```
process P1      process P2
  p1a;            p2a;
  p1b;            p2b;
end P1;         end P2;
```

All 6 possible interleavings (also called a **history** or **trace**):

```
p1a; p1b; p2a; p2b;          p2a; p2b; p1a; p1b;
p1a; p2a; p1b; p2b;          p2a; p1a; p2b; p1b;
p1a; p2a; p2b; p1b;          p2a; p1a; p1b; p2b;
```

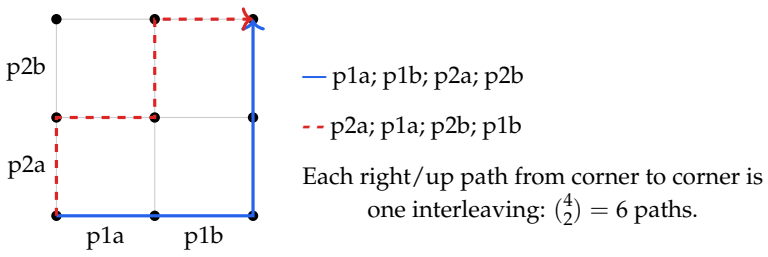


Figure 3.6: Interleavings as lattice paths: moving right executes the next P1 action, moving up the next P2 action.

General formula

With n processes each executing a sequence of m atomic actions, the number of different interleavings is

$$\frac{(n \cdot m)!}{(m!)^n}$$

Applying the formula

Two processes, two actions each: $\frac{(2 \cdot 2)!}{(2!)^2} = \frac{24}{4} = 6$ (matches the list above).

Three processes, two atomic actions each: $\frac{(3 \cdot 2)!}{(2!)^3} = \frac{720}{8} = 90$.

Growth: 4 processes $\times$ 3 actions gives $\frac{12!}{(3!)^4} = 369{,}600$ — far too many to test exhaustively.

3.11.2 Interleaving Changes the Meaning of a Program

Worked example

```
int c1 = 2;
int c2 = 3;

process P1                  process P2
p1:  c1 = c1 * c2;          p2:  c1 = c1 + c2;
end P1;                     end P2;
```

Order	Result
`p1` before `p2`	$c_1 = 2 \cdot 3 = 6$, then $6 + 3$: `c1 == 9`, `c2 == 3`
`p2` before `p1`	$c_1 = 2 + 3 = 5$, then $5 \cdot 3$: `c1 == 15`, `c2 == 3`

The program has **no single meaning**. This is the essence of a race condition.

3.12 Atomic Instructions

Why this matters

It is **extremely important** to define exactly which instructions are being interleaved, that is **which instructions are atomic**.

Definition

An **atomic action** makes an **indivisible state transition**.

3.12.1 The Granularity Example

Consider this program on a 2-processor computer, where each PE has its own set of registers.

```
int n = 0;

process P1              process P2
p1: n = n + 1;          p2: n = n + 1;
end P1;                 end P2;
```

Case A: single INC instruction

If the compiler translates `n = n + 1` into a **single INC machine instruction**, any interleaving gives the correct result `n == 2`.

Case B: load-store architecture

Values are loaded into registers, operated on, then stored back. Some interleavings now give an **incorrect** result:

```
p1: load n into R          (R1 = 0)
p2: load n into R          (R2 = 0)
p1: R = R + 1              (R1 = 1)
p2: R = R + 1              (R2 = 1)
p1: store R into n         (n  = 1)
p2: store R into n         (n  = 1)    → n == 1
```

One increment is lost.

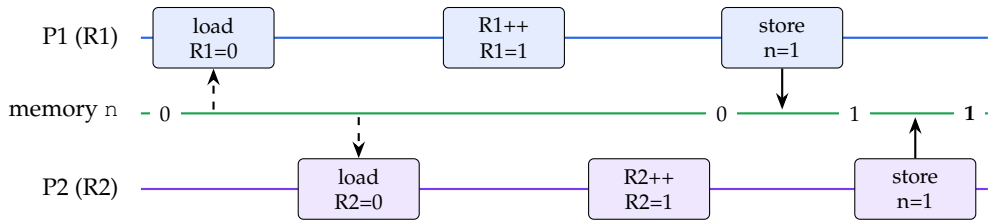


Figure 3.7: The lost-update interleaving on a load-store machine: final `n == 1`, not 2.

The lost update problem

The previous example illustrates the **lost update problem**, an example of a **race condition**. **Race conditions occur when two or more processes share data and the final result, which**

may be erroneous, depends on the interleaving of the processes' atomic actions.

The lesson

The *same source code* is correct on one architecture and broken on another, purely because of what the hardware treats as atomic. Never reason about correctness at the level of source statements. Reason at the level of **atomic actions**.

3.12.2 Exercise: Possible Final Values

Assuming a load-store architecture, what are the possible final values of `x`?

```
int y = 0;
int z = 0;

process P1          process P2
1a: x = y + z;      2a: y = 1;
end P1;             2b: z = 2;
                    end P2;
```

Solution

On a load-store machine, `1a` decomposes into: load `y` into a register, load `z` into a register, add, store into `x`. `P2`'s two assignments can be interleaved between those loads.

Interleaving	`y` read	`z` read	Final `x`
`1a` completes entirely before `2a`	0	0	**0**
`2a`, then `1a` completes, then `2b`	1	0	**1**
load `y`, then `2a` and `2b`, then load `z`	0	2	**2**
`2a` and `2b` both complete before `1a`	1	2	**3**

Possible final values: **0, 1, 2 and 3**. The value **2** is the surprising one, and it is only reachable because `x = y + z` is **not** atomic.

3.13 Correctness of Concurrent Programs

How correctness is defined

The correctness of concurrent programs is defined in terms of **properties of execution sequences**. There are **two types** of correctness properties: **safety** and **liveness**.

Safety: "nothing bad will ever happen"

The property must **always** be true.

- **Mutual exclusion:** processes may not interleave certain sub-sequences of instructions, e.g. at most one process may access a printer at any one instant.

- **Absence of deadlock:** deadlock occurs when all processes are blocked and are unable to proceed.

Liveness: "something good will eventually happen"

The property must **eventually** be true.

- **Absence of starvation:** e.g. if a process posts a request to print, eventually it will be assigned a printer.

	Safety	Liveness
Slogan	Nothing bad happens **ever**	Something good happens **eventually**
Quantifier	Always true	Eventually true
Violated by	A **finite** prefix of an execution	Only an **infinite** execution
Examples	Mutual exclusion, absence of deadlock	Absence of starvation, progress, bounded waiting

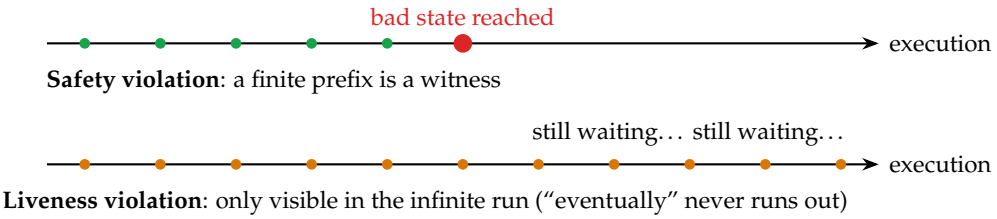


Figure 3.8: A safety violation can be observed at a point; a liveness violation never can.

Why the distinction is practical

A safety violation can be **caught** by observing a finite run, because you can point at the exact moment it broke. A liveness violation can **never** be caught by observation, because "eventually" has not run out yet. That asymmetry shapes how each is tested and proved.

3.14 Summary

Lecture Summary

- A **race condition** exists when the final value of shared data depends on which process finishes last. The cure is **synchronization**.
- The **critical-section problem** requires **mutual exclusion**, **progress**, and **bounded waiting**.
- **Deadlock** freezes all involved processes (circular wait). **Starvation** freezes one process (unfairness).
- Synchronization mechanisms: **locks, semaphores, monitors**. Classical problems: **bounded-buffer, readers/writers, dining philosophers**.
- Concurrent execution is **always modelled as interleaving**, arbitrary but fair, and correctness is required under all of them.

- The number of interleavings is $\frac{(n \cdot m)!}{(m!)^n}$, which rules out testing as a correctness strategy.
- **Precedence** $\rightarrow$ is irreflexive, antisymmetric and transitive, but only a **partial** order because intervals can overlap.
- **Atomicity granularity determines correctness.** `n = n + 1` is safe as a single INC and unsafe on a load-store machine.
- Correctness splits into **safety** (always) and **liveness** (eventually).

Self-check Questions

1. State the three requirements for a solution to the critical-section problem, and classify each as safety or liveness.
2. Why is bounded waiting a stronger requirement than progress?
3. Construct a two-semaphore deadlock and explain the ordering rule that would prevent it.
4. How many interleavings does a program with 4 processes of 3 atomic actions each have?
5. Why is conventional debugging inadequate for concurrent programs?
6. Show why $\rightarrow$ is a partial and not a total order on intervals.
7. Give an interleaving of two `n = n + 1` operations on a load-store machine that produces `n == 1`.
8. In the `x = y + z` exercise, explain specifically how `x` can end up as 2.

LECTURE 4

Shared Memory Programming with Pthreads & OpenMP

Overview

The practical half of the course begins here. **Part 1** covers **Pthreads**, the low-level POSIX threading API: creating and joining threads, mutexes, busy-waiting, and read-write locks. **Part 2** covers **OpenMP**, a high-level pragma-based API that hides most of that machinery. Slides extended from *An Introduction to Parallel Programming* by Peter Pacheco.

4.1 The Shared Memory System

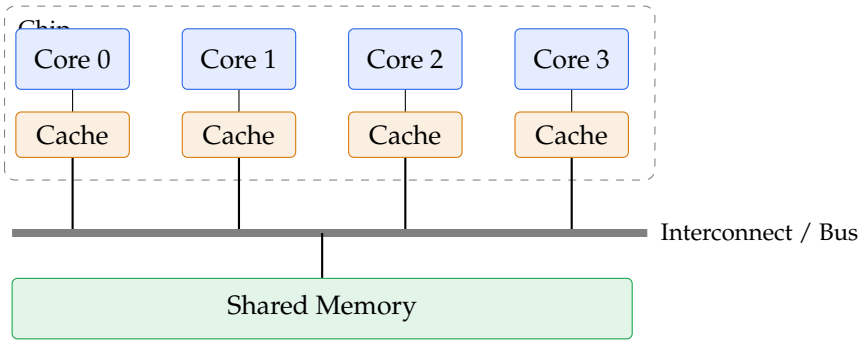


Figure 4.1: A shared-memory system: every core can reach every memory location.

Why this matters for programming

Every core can reach **every** memory location. Threads therefore communicate **implicitly** by reading and writing the same variables, which is convenient and dangerous in equal measure.

Part 1: POSIX Threads (Pthreads)

4.2 What Pthreads Is

POSIX Threads

- Also known as **Pthreads**; a **standard** for Unix-like operating systems.
- A **library** that can be linked with C programs.
- Specifies an **API** for multi-threaded programming.

Pthreads is **not** a language extension. It is ordinary C function calls, which is why the compiler

cannot help you catch concurrency mistakes.

4.2.1 Hello World

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>       /* Declares Pthreads functions, constants, types */

int thread_count;          /* global: visible to all threads */

void* Hello(void* rank);   /* thread function prototype */

int main(int argc, char* argv[]) {
    long       thread;                 /* use long in case of a 64-bit system */
    pthread_t* thread_handles;

    thread_count   = strtol(argv[1], NULL, 10);
    thread_handles = malloc(thread_count * sizeof(pthread_t));

    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread], NULL,
                       Hello, (void*) thread);

    printf("Hello from the main thread\n");

    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);

    free(thread_handles);
    return 0;
}

void* Hello(void* rank) {
    long my_rank = (long) rank;
    printf("Hello from thread %ld of %d\n", my_rank, thread_count);
    return NULL;
}
```

4.2.2 Compiling and Running

Compile

```
gcc -g -Wall -o pth_hello pth_hello.c -lpthread
```

`-lpthread` **links the Pthreads library**. Forgetting it is the most common first error.

Running

```
./pth_hello 1                    ./pth_hello 4
  Hello from the main thread       Hello from the main thread
  Hello from thread 0 of 1         Hello from thread 0 of 4
```

```
Hello from thread 3 of 4
Hello from thread 2 of 4
Hello from thread 1 of 4
```

Note the output order with 4 threads: 0, 3, 2, 1. Output order is **nondeterministic** and will differ between runs.

4.3 The Fork/Join Model

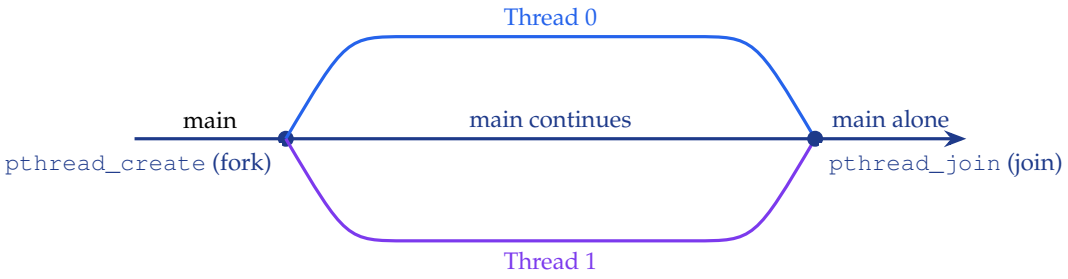


Figure 4.2: Fork/join: threads are created, run concurrently, and are joined back into the main thread.

The pattern

The main thread **forks** additional threads with `pthread_create`, they run concurrently, and the main thread **joins** them with `pthread_join`. After the join, only the main thread remains.

Global variables

Global variables can introduce **subtle and confusing bugs**. Use them **only when they are essential**. In the Hello World example, `thread_count` is global purely so every thread can read it without being passed a struct.

4.4 Starting and Stopping Threads

4.4.1 `pthread_create`

```
int pthread_create(
    pthread_t*              thread_p,                /* out */
    const pthread_attr_t*   attr_p,                  /* in  */
    void*                  (*start_routine)(void*),  /* in  */
    void*                   arg_p);                  /* in  */
```

Argument	Dir.	Purpose
`thread_p`	out	**One `pthread_t` object for each thread**. Filled in by the call.
`attr_p`	in	Thread attributes. Pass `NULL` for defaults.
`start_routine`	in	The function the new thread will run
`arg_p`	in	The single argument passed to that function

We **ignore the return value** from `pthread_create` in these examples (it is 0 on success, an error number otherwise).

The required function signature

```
void* thread_function(void* args_p);
```

- `void*` can be **cast to any pointer type** in C.
- So `args_p` can point to a **list containing one or more values** needed by `thread_function`.
- Similarly, the **return value** of `thread_function` can point to a list of one or more values.

This is C's workaround for the lack of generics. One pointer smuggles in arbitrarily much data, e.g.:

```
struct args { int a, b, c; };
struct args* p = malloc(sizeof *p);  p→a = 1; p→b = 2; p→c = 3;
pthread_create(&h, NULL, worker, p);   /* worker casts void* back to
    struct args* */
```

4.4.2 `pthread_join`

```
int pthread_join(
    pthread_t   thread,        /* in  */
    void**      ret_val_p);    /* out */
```

Semantics

- A single call to `pthread_join` will **wait for the thread** associated with the `pthread_t` object to complete.
- It **suspends execution of the calling thread** until the target thread terminates, unless it has already terminated.
- Call `pthread_join` **once for each thread**.

4.5 Example: Matrix-Vector Multiplication

4.5.1 Serial Version

$$y_i = \sum_{j=0}^{n-1} a_{ij} \cdot x_j \qquad \text{for } i = 0, 1, \ldots, m-1$$

```
/* Serial pseudo-code */
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    for (j = 0; j < n; j++)
        y[i] += A[i][j] * x[j];
}
```

4.5.2 Parallel Decomposition

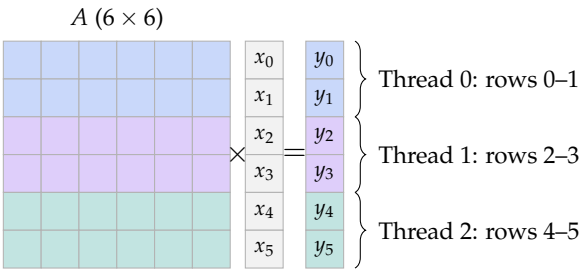


Figure 4.3: Block-row decomposition of $y = Ax$ for 3 threads. Every thread reads all of x but writes only its own y entries.

General case

With m rows and t threads, thread q handles rows

$$\left[q \cdot \frac{m}{t},\ (q+1) \cdot \frac{m}{t} - 1\right]$$

```
void* Pth_mat_vect(void* rank) {
    long   my_rank      = (long) rank;
    int    i, j;
    int    local_m      = m / thread_count;
    int    my_first_row = my_rank * local_m;
    int    my_last_row  = (my_rank + 1) * local_m - 1;

    for (i = my_first_row; i <= my_last_row; i++) {
        y[i] = 0.0;
        for (j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    }
    return NULL;
}
```

Why this needs no locking

Each thread writes to a **disjoint** set of `y[i]` entries and only **reads** `A` and `x`. No two threads ever write the same location, so there is no race at all. Contrast this with the next example, where they do.

4.6 Example: Estimating π (and How It Breaks)

$$\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots\right) = 4\sum_{k=0}^{n-1} \frac{(-1)^k}{2k+1}$$

```
void* Thread_sum(void* rank) {
    long   my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n        = n / thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i   = my_first_i + my_n;
```

```
    if (my_first_i % 2 == 0) factor =  1.0;
    else                     factor = -1.0;

    for (i = my_first_i; i < my_last_i; i++, factor = -factor)
        sum += factor / (2*i + 1);      /* sum is a SHARED variable */

    return NULL;
}
```

Observed behaviour on a dual-core processor

- As we increase n, the estimate with **1 thread gets better and better**.
- The **2-thread case produces different answers in different runs**.

Why? `sum` is a **shared variable** and `sum += ...` is not atomic. It is the lost-update problem from Lecture 3, now in real code.

4.6.1 Fix Attempt 1: Busy-Waiting

```
void* Thread_sum(void* rank) {
    /* ... compute my_sum locally ... */

    while (flag != my_rank);     /* BUSY-WAIT: spin until it is my turn */
    sum += my_sum;
    flag = (flag + 1) % thread_count;

    return NULL;
}
```

Busy-waiting works but wastes the CPU

The spinning thread holds a core doing **nothing useful**. It also forces the threads into a **fixed order** (0, 1, 2, ...), removing any scheduling flexibility.

4.7 Mutexes

What a mutex is

A **mut**ual **ex**clusion lock. It makes sure **only 1 thread is in the critical region** at a time. The Pthreads standard includes a special type for mutexes: **`pthread_mutex_t`**.

Operation	Function	When
Init	`pthread_mutex_init`	Create the mutex before threads use it
Lock	`pthread_mutex_lock`	To gain access to a critical section
Unlock	`pthread_mutex_unlock`	When a thread is finished executing the critical section
Termination	`pthread_mutex_destroy`	When a program finishes using a mutex

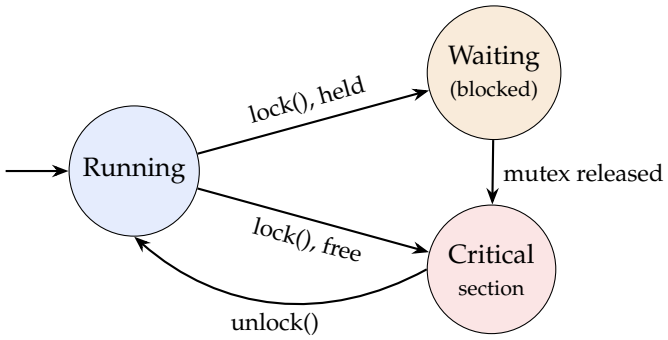


Figure 4.4: A thread's life around a mutex. Blocked threads sleep instead of spinning.

4.7.1 Global Sum Using a Mutex

```c
#include <pthread.h>

pthread_mutex_t mutex;
double sum = 0.0;

/* in main, before creating threads: */
pthread_mutex_init(&mutex, NULL);
/* ... create and join threads ... */
pthread_mutex_destroy(&mutex);

void* Thread_sum(void* rank) {
    long   my_rank = (long) rank;
    double factor, my_sum = 0.0;
    long long i;
    long long my_n       = n / thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i  = my_first_i + my_n;

    if (my_first_i % 2 == 0) factor =  1.0;
    else                     factor = -1.0;

    for (i = my_first_i; i < my_last_i; i++, factor = -factor)
        my_sum += factor / (2*i + 1);      /* private accumulator */

    pthread_mutex_lock(&mutex);
    sum += my_sum;                         /* critical section: one line */
    pthread_mutex_unlock(&mutex);

    return NULL;
}
```

The key design idea

Accumulate into a **private** `my_sum` for the whole loop, and enter the critical section **exactly once** at the end. The critical section shrinks from n executions to 1.

4.7.2 Busy-Waiting vs Mutex

Threads	Busy-Wait	Mutex
1	2.90	2.90
2	1.45	1.45
4	0.73	0.73
8	0.38	0.38
16	0.50	0.38
32	0.80	0.40
64	3.56	0.38

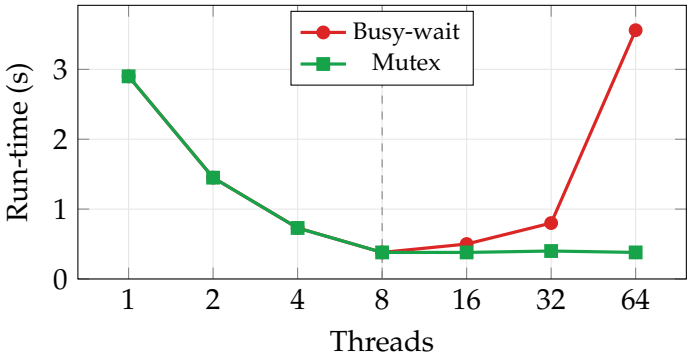


Figure 4.5: Run-times, $n = 10^8$ terms, 2×4-core processors.

Reading the results

The two are identical while threads $\leq$ cores. **Once threads exceed cores, busy-waiting collapses** because spinning threads burn timeslices that the thread holding the turn actually needs. The mutex version stays flat because blocked threads yield the CPU.

4.8 Read-Write Locks

4.8.1 Motivating Example: A Shared Sorted Linked List

Setup

The shared data structure is a **sorted linked list of ints**, and the operations of interest are **Member, Insert, and Delete**.

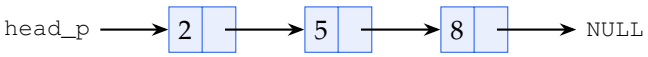


Figure 4.6: A sorted linked list of ints.

```
int Member(int value) {
    struct list_node_s* curr_p = head_p;

    while (curr_p != NULL && curr_p→data < value)
        curr_p = curr_p→next;

    if (curr_p == NULL || curr_p→data > value)
        return 0;
    else
        return 1;
}
```

```
int Insert(int value) {
    struct list_node_s* curr_p = head_p;
    struct list_node_s* pred_p = NULL;
    struct list_node_s* temp_p;

    while (curr_p != NULL && curr_p→data < value) {
```

```
        pred_p = curr_p;
        curr_p = curr_p→next;
    }

    if (curr_p == NULL || curr_p→data > value) {
        temp_p = malloc(sizeof(struct list_node_s));
        temp_p→data = value;
        temp_p→next = curr_p;
        if (pred_p == NULL)  head_p = temp_p;   /* new first node */
        else                 pred_p→next = temp_p;
        return 1;
    } else {
        return 0;                                /* value already in list */
    }
}
```

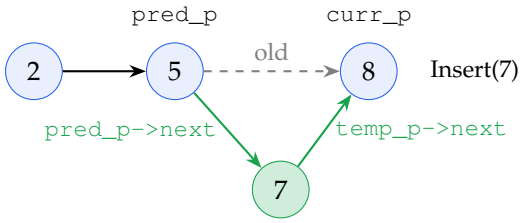


Figure 4.7: Insert splices a new node between `pred_p` and `curr_p`.

```
int Delete(int value) {
    struct list_node_s* curr_p = head_p;
    struct list_node_s* pred_p = NULL;

    while (curr_p != NULL && curr_p→data < value) {
        pred_p = curr_p;
        curr_p = curr_p→next;
    }

    if (curr_p != NULL && curr_p→data == value) {
        if (pred_p == NULL) {          /* deleting first node */
            head_p = curr_p→next;
            free(curr_p);
        } else {
            pred_p→next = curr_p→next;
            free(curr_p);
        }
        return 1;
    } else {
        return 0;                      /* value not in list */
    }
}
```

Making it multi-threaded

To share access to the list, we define `head_p` to be a **global variable**. This simplifies the function headers for Member, Insert, and Delete, because we will not need to pass in either `head_p` or a pointer to it. We only need to pass in the value of interest.

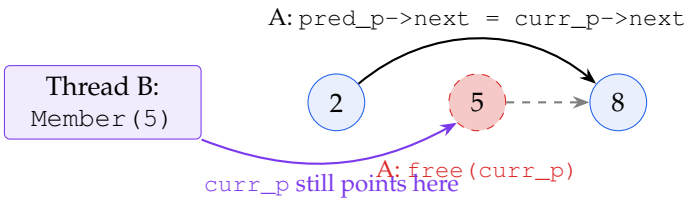


Figure 4.8: Thread A deletes node 5 while Thread B is reading it: a use-after-free.

Simultaneous access by 2 threads

If Thread A executes `Delete(5)` while Thread B executes `Member(5)`, Thread B may be holding a pointer to a node that Thread A has already `freed`. **This is a use-after-free, and it corrupts memory rather than merely returning a wrong answer.**

4.8.2 Solution 1: One Mutex for the Whole List

```
pthread_mutex_lock(&list_mutex);
Member(value);
pthread_mutex_unlock(&list_mutex);
```

Issues

- This is **serializing access to the list**.
- If the vast majority of our operations are calls to `Member`, we **fail to exploit the opportunity for parallelism**.
- If most of our operations are calls to `Insert` and `Delete`, **this may be the best solution**.

4.8.3 Solution 2: One Mutex per Node

Idea

Instead of locking the entire list, lock **individual nodes**. A **finer-grained** approach, using **hand-over-hand locking** as the traversal walks the list.

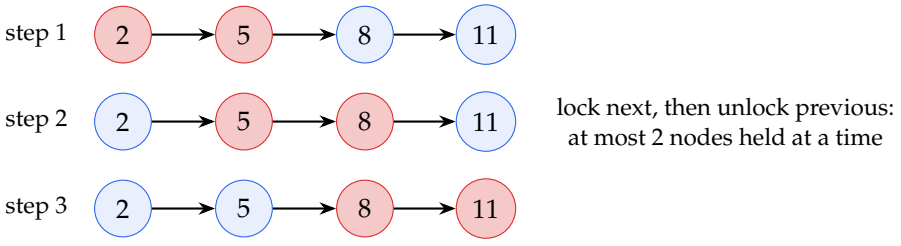


Figure 4.9: Hand-over-hand (lock-coupling) traversal with a mutex per node (red = locked).

Issues

- **Much more complex** than the original `Member` function.
- **Much slower**, because each time a node is accessed a mutex must be locked and unlocked.
- Adding a mutex field to each node **substantially increases the memory** needed for the list.

4.8.4 Solution 3: Read-Write Locks

The insight neither previous solution exploits

Neither multi-threaded linked list exploits the potential for **simultaneous access to any node by threads that are executing `Member`**. The 1st solution only allows **1 thread to access the entire list** at any instant; the 2nd only allows **1 thread to access any given node** at any instant. But readers do not conflict with each other at all.

How a read-write lock works

A read-write lock is somewhat like a mutex except that it provides **2 lock functions**: one locks it **for reading**, the other locks it **for writing**.

- **Multiple threads** can simultaneously obtain the lock by calling the **read-lock** function.
- Only **1 thread** can obtain the lock by calling the **write-lock** function.

Currently held for	Reader requests	Writer requests
Nothing	Granted	Granted
Reading	Granted	Blocked
Writing	Blocked	Blocked

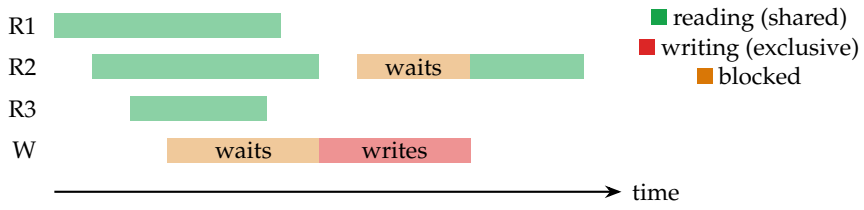


Figure 4.10: Readers overlap freely; a writer waits for all readers to leave and then excludes everyone.

```
pthread_rwlock_rdlock(&rwlock);       /* many readers allowed */
Member(value);
pthread_rwlock_unlock(&rwlock);

pthread_rwlock_wrlock(&rwlock);       /* exclusive */
Insert(value);
pthread_rwlock_unlock(&rwlock);

pthread_rwlock_wrlock(&rwlock);       /* exclusive */
Delete(value);
pthread_rwlock_unlock(&rwlock);
```

4.8.5 Performance Comparison

	Read-heavy (99.9% Member)			**Write-heavy** (80% Member)		
Threads	RW lock	One mutex	Mutex/node	RW lock	One mutex	Mutex/node
1	0.213	0.211	1.680	2.48	2.50	12.00
2	0.123	0.450	5.700	4.97	5.13	29.60
4	0.098	0.385	3.450	4.69	5.04	17.10
8	0.115	0.457	2.700	4.71	5.11	12.90

100,000 ops/thread. Read-heavy: 99.9% Member, 0.05% Insert, 0.05% Delete. Write-heavy: 80% Member, 10% Insert, 10% Delete. Times in seconds.

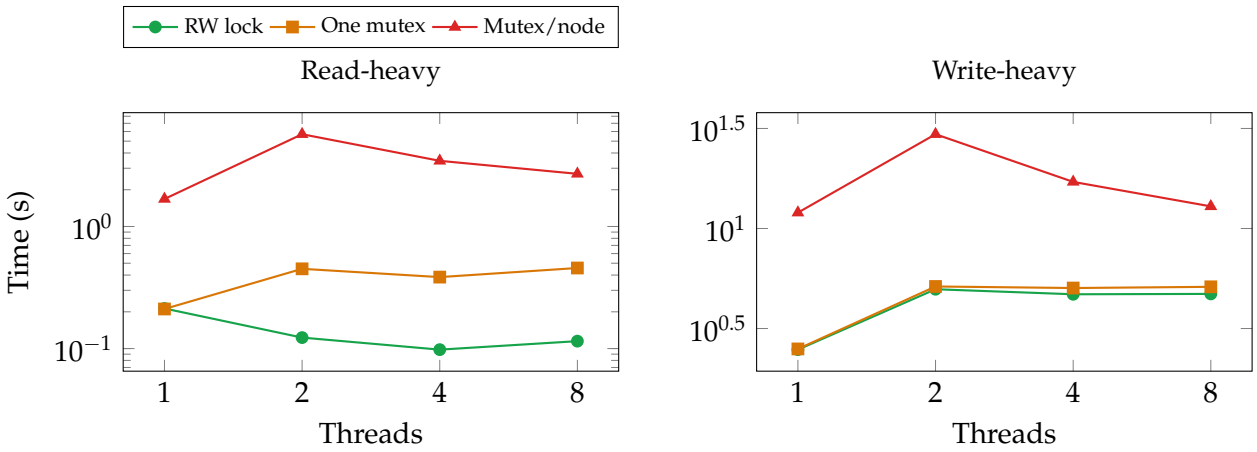


Figure 4.11: Linked-list run-times (log scale). The RW lock only wins when reads dominate.

Interpretation

Read-heavy: the read-write lock **scales**; the single mutex **gets worse** with more threads; per-node mutexes are catastrophic because of lock overhead.
Write-heavy: **the read-write lock loses its advantage entirely** because writes are exclusive anyway. The right lock depends on the **workload mix**, not on the data structure alone.

Part 2: OpenMP

4.9 What OpenMP Is

Definition

A **high-level API for shared-memory parallel programming**. MP stands for **multiprocessing**. It uses **pragmas**: special preprocessor instructions beginning with `#pragma`, typically added to support behaviours that are not part of the basic C specification.

	Pthreads	OpenMP
Level	Low-level library	High-level pragmas
Thread management	Explicit create/join	Implicit
Code changes	Restructure the program	Annotate existing loops
Control	Total	Limited to what pragmas expose
Non-OpenMP compiler	N/A	Pragmas ignored; code still compiles and runs serially

A real advantage

An OpenMP program is still a valid serial program.

4.9.1 Hello World in OpenMP

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void);

int main(int argc, char* argv[]) {
    int thread_count = strtol(argv[1], NULL, 10);

#   pragma omp parallel num_threads(thread_count)
    Hello();

    return 0;
}

void Hello(void) {
    int my_rank      = omp_get_thread_num();
    int thread_count = omp_get_num_threads();

    printf("Hello from thread %d of %d\n", my_rank, thread_count);
}
```

Compiling and running

```
gcc -g -Wall -fopenmp -o omp_hello omp_hello.c
./omp_hello 4
```

`-fopenmp` is what activates the pragmas.

Three possible outcomes with 4 threads

```
Hello from thread 0 of 4     Hello from thread 1 of 4     Hello from thread 3 of 4
Hello from thread 1 of 4     Hello from thread 2 of 4     Hello from thread 1 of 4
Hello from thread 2 of 4     Hello from thread 0 of 4     Hello from thread 2 of 4
Hello from thread 3 of 4     Hello from thread 3 of 4     Hello from thread 0 of 4
```

All three are correct. Nondeterminism is expected.

4.10 OpenMP Pragmas and Clauses

4.10.1 The Parallel Directive

`#pragma omp parallel`

- The **most basic parallel directive**.
- The original thread is called the **master**; additional threads are called **slaves** (workers).
- The original thread and the new threads together are called a **team**.

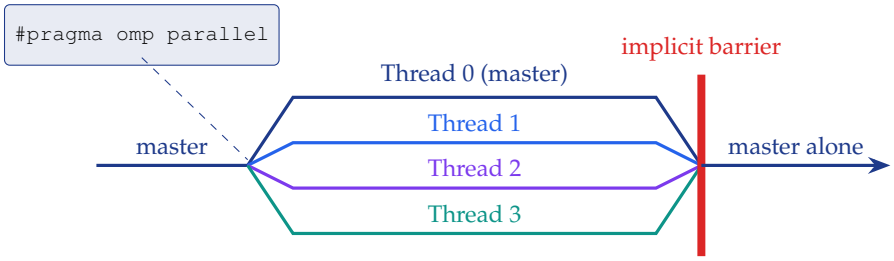


Figure 4.12: A parallel region: the team is created, every thread runs the block, and an implicit barrier ends the region.

4.10.2 Clauses

Definition

A **clause** is text that modifies a directive. The **`num_threads`** clause can be added to a parallel directive. It allows the programmer to specify the number of threads that should execute the following block.

```
#pragma omp parallel num_threads(thread_count)
```

Be aware

- There may be **system-defined limitations** on the number of threads that a program can start.
- The OpenMP standard **does not guarantee** that this will actually start `thread_count` threads.
- Most current systems can start hundreds or even thousands of threads, so unless we are trying to start a lot of threads, we will almost always get the desired number.

4.10.3 Mutual Exclusion in OpenMP

```
#pragma omp critical
    global_result += my_result;
```

`#pragma omp critical`

Only 1 thread can execute the following structured block at a time. Compare this to the `pthread_mutex_init` / `lock` / `unlock` / `destroy` boilerplate it replaces.

4.11 Example: The Trapezoidal Rule

The rule

To approximate $\int_a^b f(x)\,dx$ with n trapezoids of width $h = \dfrac{b-a}{n}$:

$$\int_a^b f(x)\,dx \approx h\left[\frac{f(a)}{2} + \frac{f(b)}{2} + \sum_{i=1}^{n-1} f(a+ih)\right]$$

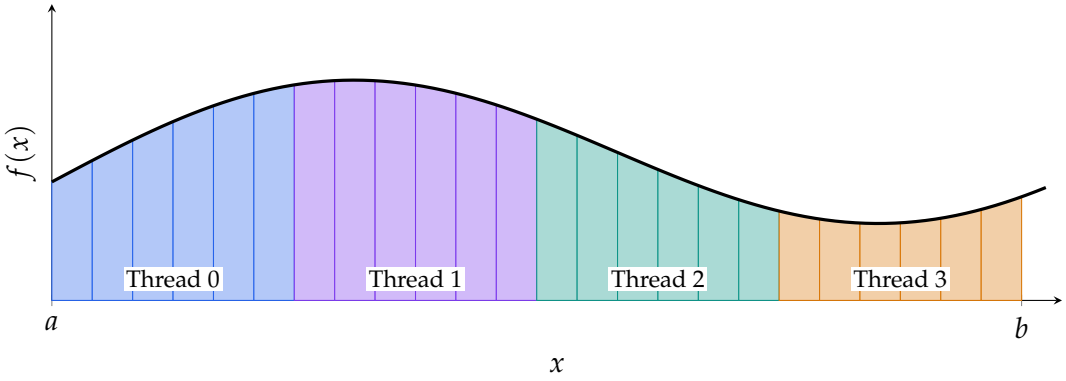


Figure 4.13: $n = 24$ trapezoids split into 4 contiguous blocks of 6, one block per thread.

4.11.1 Serial Algorithm

```
h = (b - a) / n;
approx = (f(a) + f(b)) / 2.0;
for (i = 1; i <= n - 1; i++) {
    x_i = a + i * h;
    approx += f(x_i);
}
approx = h * approx;
```

Each thread integrates a contiguous **block** of the interval (thread q gets $[\text{local_a}, \text{local_b}]$ with $\text{local_a} = a + q \cdot \text{local_n} \cdot h$), then contributes its partial result to the global sum.

4.11.2 OpenMP Version

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
    double global_result = 0.0;
    double a, b;
    int    n, thread_count;

    thread_count = strtol(argv[1], NULL, 10);
    printf("Enter a, b, and n\n");
    scanf("%lf %lf %d", &a, &b, &n);

#   pragma omp parallel num_threads(thread_count)
```

```
    Trap(a, b, n, &global_result);

    printf("With n = %d trapezoids, our estimate\n", n);
    printf("of the integral from %f to %f = %.14e\n", a, b, global_result);
    return 0;
}

void Trap(double a, double b, int n, double* global_result_p) {
    double h, x, my_result;
    double local_a, local_b;
    int    i, local_n;
    int    my_rank      = omp_get_thread_num();
    int    thread_count = omp_get_num_threads();

    h       = (b - a) / n;
    local_n = n / thread_count;
    local_a = a + my_rank * local_n * h;
    local_b = local_a + local_n * h;

    my_result = (f(local_a) + f(local_b)) / 2.0;
    for (i = 1; i <= local_n - 1; i++) {
        x = local_a + i * h;
        my_result += f(x);
    }
    my_result = my_result * h;

#   pragma omp critical
    *global_result_p += my_result;          /* the only shared write */
}
```

The same pattern as the Pthreads mutex example

Compute a **private partial result**, then enter the critical section **once**. This pattern (local accumulation plus a single guarded combine) is the single most useful idiom in shared-memory programming.

4.12 Summary

Lecture Summary

- **Pthreads** is a POSIX **library** for C. Compile with `-lpthread`.
- **Fork/join**: `pthread_create` starts a thread running `void* f(void*)`, `pthread_join` waits for it. Call join **once per thread**.
- Data decomposition with **disjoint writes** (matrix-vector) needs no synchronization at all.
- A **shared accumulator** (the π example) does. `sum += x` is not atomic.
- **Busy-waiting** is correct but wastes CPU and collapses when threads exceed cores. **Mutexes** block instead of spinning and stay flat.
- The winning idiom: **accumulate privately, combine once inside the critical section**.
- **Lock granularity is a trade-off**: one global mutex serializes; per-node mutexes cost more in overhead and memory than they save.

- **Read-write locks** allow unlimited concurrent readers but exclusive writers. They win big on **read-heavy** workloads and are no better than a mutex on write-heavy ones.
- **OpenMP** replaces all of that with pragmas: `#pragma omp parallel num_threads(t)` and `#pragma omp critical`.

Self-check Questions

1. Why must a Pthreads thread function have signature `void* f(void*)`, and how do you pass it three integers?
2. In the matrix-vector example, why is no mutex required?
3. Explain why the 2-thread π program gives a different answer on each run.
4. At what point does busy-waiting become dramatically worse than a mutex, and why?
5. Why is it better to lock once at the end of a loop than once per iteration?
6. Give one workload where a single global mutex beats a read-write lock.
7. Why do per-node mutexes make the linked list slower rather than faster?
8. Write the OpenMP equivalent of `pthread_mutex_lock` / `unlock` around a shared increment.

LECTURE 5

The Mutual Exclusion Problem

Overview

A step-by-step derivation of a correct software-only mutual exclusion algorithm for two processes, using **only Load and Store** as atomic instructions. Four failed attempts, each failing for a different and instructive reason, lead to **Dekker's algorithm** and then to the simpler **Peterson's algorithm**. Slides based mostly on Professor N.S. Scott's (Queen's University, Belfast) slides.

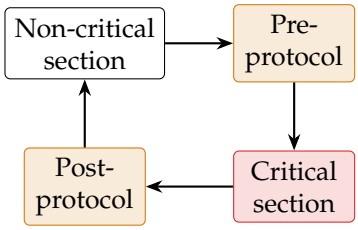
5.1 Statement of the Problem

Definition

Eliminating undesirable interleavings is called the mutual exclusion problem. We need to:

- Identify **critical sections** that only one thread at a time can enter.
- Devise a **pre-protocol** and a **post-protocol** to keep two or more threads from being in their critical sections at the same time.

```
while (true) {
    nonCriticalSection;
    preProtocol;       /* entry:
   the hard part */
    criticalSection;
    postProtocol;      /* exit */
}
```



5.2 Formal Requirements for N Processes

The six requirements

1. Mutual exclusion. N processes execute, in an infinite loop, a sequence of instructions divided into a **critical section** and a **non-critical section**. Instructions from the critical sections of two or more processes **must not be interleaved**.
2. Structure of the solution. The solution is described by inserting into the loop additional instructions to be executed by a process wishing to **enter** and **leave** its critical section: the **pre-protocol** and **post-protocol**. These protocols may require additional variables.
3. Halting rules. A process **may halt in its non-critical section**. It may **not** halt during execution of its protocols or critical section. If one process halts in its non-critical section, it **must not interfere** with the operation of other processes.
4. No deadlock. If some processes are trying to enter their critical sections then **one of them must eventually succeed**. The program is deadlocked if no process ever succeeds in making the transition from pre-protocol to critical section.
5. No starvation. If a process indicates its intention to enter its critical section by commencing execution of the pre-protocol, then **eventually it must succeed**.

6. No starvation in the absence of contention. In the absence of contention, a single process wishing to enter its critical section **will succeed**. A good solution will have **minimal overhead** in this case.

Requirement 3 is the one students forget

A process may sit forever in its non-critical section. Your algorithm must still work for everyone else. Attempt 1 fails precisely here.

5.2.1 Ground Rules for Two Processes

Atomic instructions available

We solve the mutual exclusion problem for two processes using **Load and Store to common memory as the only atomic instructions**.

```
c1 = 1;       /* atomic store  */
c2 = c1;      /* atomic load then atomic store */
```

No test-and-set, no compare-and-swap, no hardware help of any kind.

5.3 First Attempt: Strict Alternation

```
int turn = 1;

process P1                          process P2
while (true) {                      while (true) {
  nonCriticalSection1;                nonCriticalSection2;
  while (turn == 2) {}                while (turn == 1) {}
  criticalSection1;                   criticalSection2;
  turn = 2;                           turn = 1;
}                                   }
end P1;                             end P2;
```

A single shared variable `turn` indicates whose turn it is to enter the critical section.

Mutual exclusion	No deadlock	No starvation	No starvation w/o contention
✓	✓	✓	×

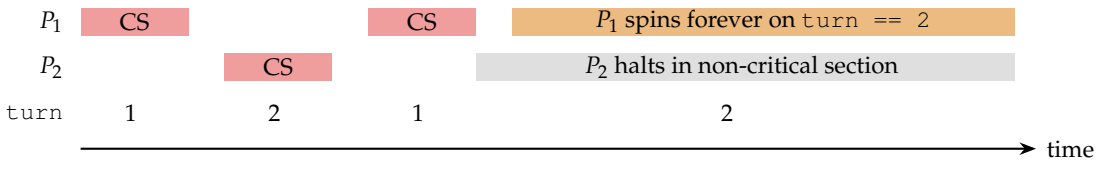


Figure 5.1: Strict alternation forces P1, P2, P1, P2...; if P2 stops wanting the CS, P1 is blocked forever.

Proof: mutual exclusion is satisfied

Suppose that at some point both processes are in their critical sections. Without loss of generality, assume P_1 entered at time t_1 and P_2 entered at time t_2, where $t_1 < t_2$. P_1 remained in its critical section during $[t_1, t_2]$. At t_1, `turn == 1`, and at t_2, `turn == 2`. But during $[t_1, t_2]$, P_1 remained in its critical section and **did not execute its post-protocol**, which is the

only means of assigning 2 to `turn`. So at t_2, `turn` must still be 1, **contradicting** the previous statement. ■

Proof: the solution cannot deadlock

For the program to deadlock, each process must execute the test on `turn` infinitely often, failing each time. Therefore in P_1 we need `turn == 2` and in P_2 we need `turn == 1`, which is **impossible** since `turn` holds one value. ■

Proof: there is no starvation

For starvation to exist, one process must enter its critical section infinitely often while the other executes its pre-protocol forever without progressing. But if P_1 executes its critical section even once, it will set `turn = 2` in its post-protocol, allowing P_2 to enter. ■

Proof: there IS starvation in the absence of contention

Suppose P_2 **halts in its non-critical section**. Then `turn` will never be changed from 2 to 1. P_1 may enter its critical section **at most one more time**. Once P_1 sets `turn` to 2, it will **never again** be able to progress from its pre-protocol to its critical section.
A process that is not even interested in the critical section can permanently block a process that is. This violates requirement 6.

5.4 Second Attempt: Test Then Set Own Flag

```
int c1 = 1;
int c2 = 1;

process P1                          process P2
while (true) {                      while (true) {
  nonCriticalSection1;                nonCriticalSection2;
  while (c2 != 1) {}                  while (c1 != 1) {}
  c1 = 0;                             c2 = 0;
  criticalSection1;                   criticalSection2;
  c1 = 1;                             c2 = 1;
}                                   }
end P1;                             end P2;
```

Each process P_i now has its own variable c_i. Shared variable $c_i = 0$ signals that P_i **is about to enter** its critical section.

Mutual exclusion
×

Proof: mutual exclusion is NOT satisfied

Consider the following interleaving beginning with the initial state:

Step	Action
1	P_1 checks `c2` and finds `c2 == 1`
2	P_2 checks `c1` and finds `c1 == 1`
3	P_1 sets `c1` to 0
4	P_2 sets `c2` to 0
5	P_1 **enters its critical section**
6	P_2 **enters its critical section**

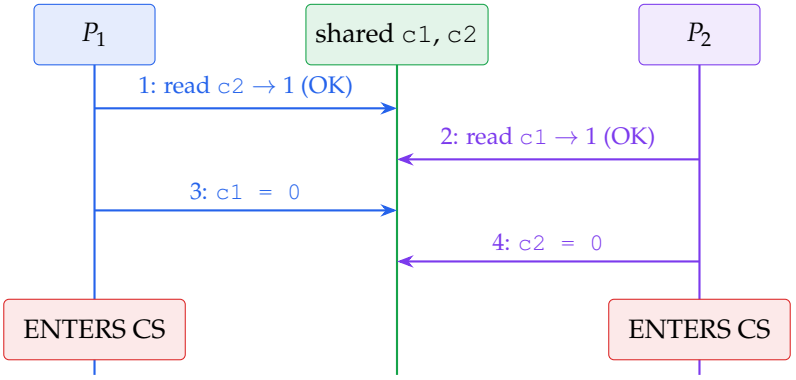


Figure 5.2: Attempt 2: both processes pass the test before either sets its flag.

Why it fails

The **test** and the **set** are two separate atomic actions with a gap between them. Both processes pass the test **before** either one sets its flag. This is the classic **check-then-act** race.

5.5 Third Attempt: Set Own Flag Then Test

```
int c1 = 1;
int c2 = 1;

process P1                          process P2
while (true) {                      while (true) {
  nonCriticalSection1;                nonCriticalSection2;
  c1 = 0;                             c2 = 0;
  while (c2 != 1) {}                  while (c1 != 1) {}
  criticalSection1;                   criticalSection2;
  c1 = 1;                             c2 = 1;
}                                   }
end P1;                             end P2;
```

The change

In Attempt 2 the assignment $c_i = 0$ was effectively located **in** the critical section. Here we move it to the **beginning of the pre-protocol**. Now $c_i = 0$ signals that P_i **wishes to enter** its critical section. This is exactly the **Flag Principle** from Lecture 1: raise your flag first, then look.

Mutual exclusion	No deadlock
✓	×

Proof: mutual exclusion is satisfied

Suppose both processes are in their critical sections. WLOG P_1 entered at t_1 and P_2 at t_2, with $t_1 < t_2$. P_1 remained in its critical section throughout $[t_1, t_2]$. At t_1, `c1 == 0` and `c2 == 1`. At t_2, `c2 == 0` and `c1 == 1`. But during $[t_1, t_2]$, P_1 remained in its critical section and did not execute its post-protocol, which is the **only** means of assigning 1 to `c1`. So at t_2, `c1` must still be 0, **contradicting** the previous statement. ■

Proof: the program CAN deadlock

Step	Action
1	P_1 sets `c1` to 0
2	P_2 sets `c2` to 0
3	P_1 tests `c2` and **remains in the loop**
4	P_2 tests `c1` and **remains in the loop**

Both processes are locked forever in their pre-protocols. Neither will ever lower its flag, because lowering only happens after the critical section.

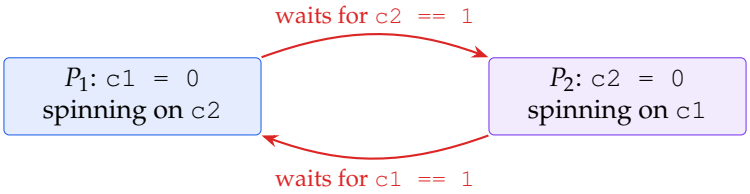


Figure 5.3: Attempt 3 deadlock: each waits for a flag the other will only lower after its CS.

5.6 Fourth Attempt: Back Off

```
int c1 = 1;
int c2 = 1;

process P1                          process P2
while (true) {                      while (true) {
  nonCriticalSection1;                nonCriticalSection2;
  c1 = 0;                             c2 = 0;
  while (c2 != 1) {                   while (c1 != 1) {
    c1 = 1; c1 = 0;                     c2 = 1; c2 = 0;
  }                                   }
  criticalSection1;                   criticalSection2;
  c1 = 1;                             c2 = 1;
}                                   }
end P1;                             end P2;
```

The processes **back off** entering their critical sections if they detect both are trying to enter at the same time.

Mutual exclusion	No livelock	No starvation
✓	×	×

Deadlock vs Livelock

Livelock is a form of deadlock.

- In a **deadlocked** computation, there is **no possible execution sequence** which succeeds.
- In a **livelocked** computation, there **are** successful computations, but there are one or more execution sequences in which **no process enters its critical section**.

A deadlocked process is frozen. A livelocked process is frantically busy and getting nowhere.

Mutual exclusion is satisfied

The argument is the **same as that for the third attempt**.

Proof: a process can be starved

Step	Action
1	P_1 sets `c1` to 0
2	P_2 sets `c2` to 0
3	P_2 checks `c1` and **resets `c2` to 1**
4	P_1 completes a **full cycle**: checks `c2`, enters CS, resets `c1`, enters non-CS, sets `c1` to 0
5	P_2 sets `c2` to 0

Repeating steps 3 to 5 forever: P_1 **enters its critical section infinitely often,** P_2 **remains indefinitely in its pre-protocol.**

Proof: a program can livelock

Step	Action	Step	Action
1	P_1 sets `c1` to 0	5	P_1 resets `c1` to 1
2	P_2 sets `c2` to 0	6	P_2 resets `c2` to 1
3	P_1 checks `c2`, stays in loop	7	P_1 resets `c1` to 0
4	P_2 checks `c1`, stays in loop	8	P_2 resets `c2` to 0

As with deadlock, both processes are locked in their pre-protocols. **However, the slightest deviation from the above sequence will allow one process to enter its critical section.** This is the "two people stepping aside for each other in a corridor, forever" scenario.

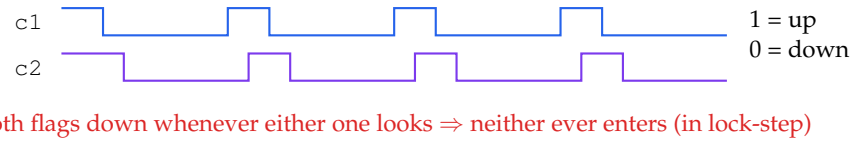


Figure 5.4: Livelock: the two flags toggle in perfect lock-step, so each process always sees the other's flag down.

5.7 A Correct Solution: Dekker's Algorithm

The idea

- It **avoids the strict alternation** of a naive turn-taking algorithm, and was one of the **first mutual exclusion algorithms to be invented**.
- If both processes attempt to access the critical section at the same time, the algorithm will **choose the process whose turn it is**. If the other process is already in the critical section, it will **wait** for that process to finish.

```
int c1 = 1;
int c2 = 1;
int turn = 1;

process P1                          process P2
while (true) {                      while (true) {
  nonCriticalSection1;                nonCriticalSection2;
  c1 = 0;                             c2 = 0;
  while (c2 != 1)                     while (c1 != 1)
    if (turn == 2) {                    if (turn == 1) {
      c1 = 1;                             c2 = 1;
      while (turn != 1) {}                while (turn != 2) {}
      c1 = 0;                             c2 = 0;
    }                                   }
  criticalSection1;                   criticalSection2;
  c1 = 1; turn = 2;                   c2 = 1; turn = 1;
}                                   }
end P1;                             end P2;
```

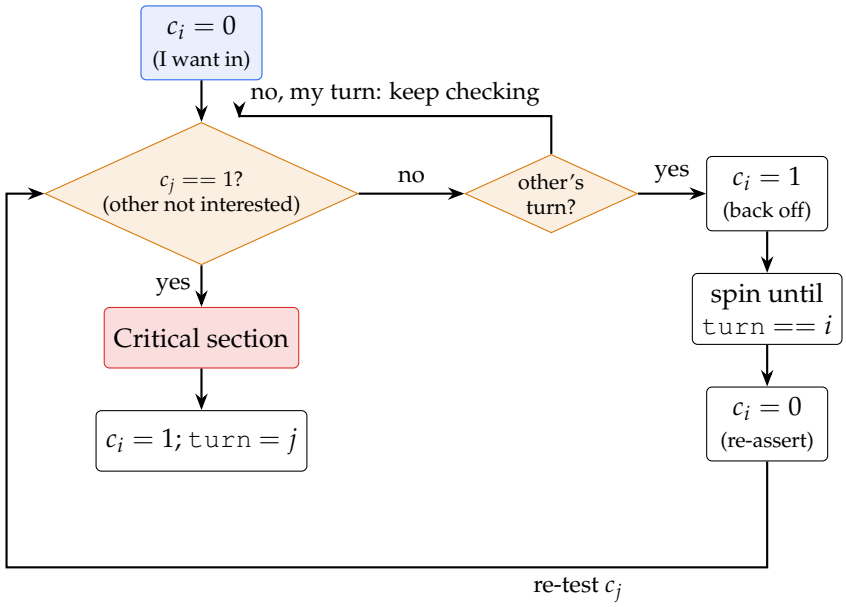


Figure 5.5: Dekker's pre-protocol for process i (the other process is j).

The key sentence

The threads now take turns at backing off. Attempt 4 failed because *both* processes backed off symmetrically. Dekker adds `turn` so that **only one** of them backs off, and which one alternates. The flags give mutual exclusion, and `turn` breaks the tie fairly.

Mutual exclusion	No deadlock	No starvation	No starvation w/o contention
✓	✓	✓	✓

Why the absence-of-contention case works

If P_2 halts in its non-critical section, `c2` stays 1 forever, so P_1's `while (c2 != 1)` loop is never entered and P_1 sails straight into its critical section every time. Compare this with Attempt 1, which failed exactly here.

5.8 Peterson's Algorithm

The simplification

- **Both the `turn` variable and the status flags are used**, as in Dekker's algorithm.
- After setting our flag we **immediately give away the turn**.
- We then wait for the turn **if and only if** the other flag is set.
- By waiting on the **conjunction of two conditions**, we avoid the need to clear and reset the flags.

```
int c1 = 1;
int c2 = 1;
int turn = 1;

process P1                              process P2
```

```
while (true) {
  nonCriticalSection1;
  c1 = 0;
  turn = 2;
  while (turn == 2 && c2 == 0) {};
  criticalSection1;
  c1 = 1;
}
end P1;
```

```
while (true) {
  nonCriticalSection2;
  c2 = 0;
  turn = 1;
  while (turn == 1 && c1 == 0) {};
  criticalSection2;
  c2 = 1;
}
end P2;
```

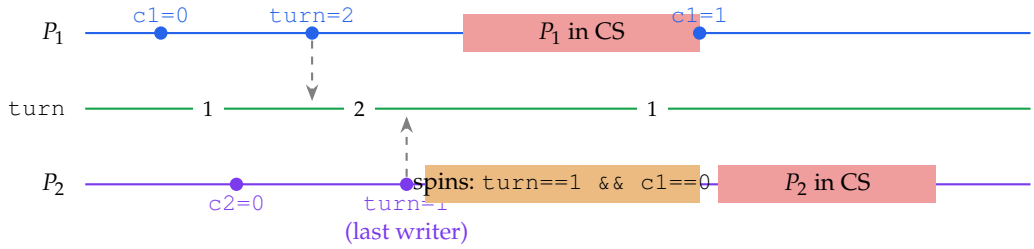


Figure 5.6: Peterson: whoever writes `turn` last waits; the other proceeds. When the winner lowers its flag, the loser enters.

Why "give away the turn" works

Both processes write `turn` immediately after raising their flag. **The last writer loses.** Whichever process wrote `turn` second sees its own value and spins. The other proceeds. Since exactly one write is last, exactly one process proceeds.
If the other process is not interested, its flag is 1 (`c_j == 0` is false), so the conjunction fails and we enter immediately regardless of `turn`.

	Dekker	Peterson
Correctness	✓ all four properties	✓ all four properties
Pre-protocol structure	Nested loop with back-off and re-assertion	**One flat `while` loop**
Flag manipulation inside the wait	Yes (clear and reset)	**None**
Readability	Awkward	Elegant

Dekker vs Peterson

Peterson's algorithm is the one to memorise. It appears again in Lecture 6 as the base case of the Filter algorithm.

5.9 Summary Table of All Attempts

Attempt	Mechanism	Mut. excl.	No deadlock	No livelock	No starv.	No starv. w/o cont.
1. Strict alternation	Single `turn`	✓	✓	✓	✓	×
2. Test then set	Two flags, test first	×				
3. Set then test	Two flags, set first	✓	×			
4. Back off	Two flags + retreat	✓		×	×	
Dekker	Two flags + `turn`	✓	✓	✓	✓	✓
Peterson	Two flags + give away `turn`	✓	✓	✓	✓	✓

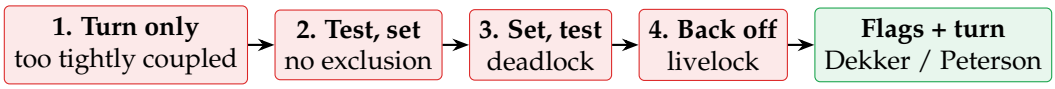


Figure 5.7: The narrative arc of the four attempts.

The narrative arc

1. **Turn alone** gives fairness but couples the processes too tightly.
2. **Flags alone, tested first** gives no exclusion (check-then-act race).
3. **Flags alone, set first** gives exclusion but deadlocks (nobody yields).
4. **Flags with symmetric back-off** avoids deadlock but livelocks (everybody yields).
5. **Flags plus turn** gives **asymmetric** yielding, which is exactly what was missing.

5.10 Exercise

Exercise

Consider the following proposed solution to the mutual exclusion problem.

```
int p = 1, q = 1;

process P {                          process Q {
  while (true) {                       while (true) {
a1:   non-critical-1;                a2:   non-critical-2;
b1:   while (q != 1) {}              b2:   while (p != 1) {}
c1:   p = 0;                         c2:   q = 0;
d1:   critical-1;                    d2:   critical-2;
e1:   p = 1;                         e2:   q = 1;
  }                                    }
}                                    }
```

For each of (i) and (ii) below, **justify your answer**: **(i)** Does the algorithm operate correctly in the absence of contention? **(ii)** Can the algorithm deadlock?

Solution

First, recognise the structure: this is **exactly the Second Attempt**, with `p` and `q` in place of `c1` and `c2`. The test on the *other* process's variable comes **before** setting your own.

(i) Yes, it operates correctly in the absence of contention. Suppose Q halts in its non-critical section. Then `q` remains 1 forever, so P's loop at `b1` never blocks. P sets `p = 0`, enters its critical section, and resets `p = 1`. P can repeat this indefinitely with no overhead beyond one test. A halted process does not interfere.

(ii) No, the algorithm cannot deadlock. For deadlock, both processes must spin forever: P stuck at `b1` requires `q == 0` permanently, and Q stuck at `b2` requires `p == 0` permanently. But `p` is set to 0 only at `c1`, which P reaches only **after** passing `b1`. If `p == 0`, then P is somewhere between `c1` and `e1`, i.e. inside or about to enter its critical section. By assumption a process may not halt in its critical section, so P will reach `e1` and set `p = 1`, releasing Q. The symmetric argument applies to Q. Hence no permanent circular wait. ■

But the algorithm is still wrong

It **violates mutual exclusion**, by the identical interleaving that broke the Second Attempt: P reads `q == 1`, Q reads `p == 1`, P sets `p = 0`, Q sets `q = 0`, and **both enter their critical sections.** The lesson: an algorithm can be deadlock-free, starvation-free, and efficient, and still be **completely useless** because it fails the one property that actually matters.

5.11 Summary

Lecture Summary

- Mutual exclusion = **eliminating undesirable interleavings** via a pre-protocol and post-protocol around each critical section.
- Six requirements: mutual exclusion, correct structure, **halting only in the non-critical section**, no deadlock, no starvation, and efficiency without contention.
- Only **atomic Load and Store** are assumed. No hardware primitives.
- **Attempt 1** (turn only): fails when one process halts outside its critical section.
- **Attempt 2** (test then set): fails mutual exclusion via a check-then-act race.
- **Attempt 3** (set then test): correct exclusion, but **deadlocks**.
- **Attempt 4** (symmetric back-off): correct exclusion, but **livelocks and starves**.
- **Dekker**: flags for exclusion plus `turn` for **asymmetric** back-off. All four properties hold.
- **Peterson**: same guarantees, far simpler. Raise flag, **give away the turn**, wait on the conjunction.
- **Livelock** differs from deadlock: successful executions exist, but some infinite executions make no progress.

Self-check Questions

1. Why does Attempt 1 satisfy no-starvation yet fail the absence-of-contention requirement? Are those not contradictory?
2. Write out the exact interleaving that breaks mutual exclusion in Attempt 2.
3. Attempt 3 satisfies mutual exclusion. Reproduce the proof by contradiction.
4. Distinguish deadlock from livelock, and say which of Attempts 3 and 4 exhibits each.
5. In Dekker's algorithm, what specifically does `turn` add that Attempt 4 lacked?
6. In Peterson's algorithm, explain why writing `turn = j` **immediately after** raising your own flag is essential.
7. Why does Peterson's algorithm never need to clear and reset its flag inside the wait loop?
8. Give a property that the exercise algorithm satisfies and one that it violates.

LECTURE 6

Basic Constructs

Overview

The `Lock` interface as a programming construct, the four necessary conditions for deadlock, and a ladder of lock implementations: `LockOne` and `LockTwo` (each broken in a different way), **Peterson's algorithm** (correct for 2 threads), the **Filter algorithm** (correct for n threads but unfair), and the **Bakery algorithm** (correct, fair, and first-come-first-served).

Outline

Mutual exclusion, deadlocks, and starvation → Locks → Bakery Algorithm

6.1 Why Concurrent Programming is Hard

Two separate difficulties

Threads and processes often need to **share data**.
1. Data Consistency — enforced through **critical sections / atomic operations**; e.g. preventing **race conditions**.
2. Cooperation (synchronization) — control the way programs make progress; e.g. the **producer–consumer problem**.
Both are complicated by the **lack of timing guarantees**: cache misses, page faults, context switches. Threads are scheduled by the OS or thread library, not by you.

6.1.1 How Things Can Go Wrong

```
int balance = 200;      // shared

Thread1 {
  run() {
    if (balance >= 100) {
      balance = balance - 100;
      System.out.println("Withdraw 100, balance " + balance);
    } else {
      System.out.println("Insufficient Balance " + balance);
    }
  }
}

Thread2 {
  run() {
    if (balance >= 200) {
      balance = balance - 200;
      System.out.println("Withdraw 200, balance " + balance);
    } else {
      System.out.println("Insufficient Balance " + balance);
    }
  }
```

```
}
```

The bad interleaving

Step	Thread1	Thread2	`balance`
1	checks `balance >= 100` → true		200
2		checks `balance >= 200` → true	200
3	`balance = 200 - 100`		100
4		`balance = 100 - 200`	**−100**

The account is overdrawn even though both threads individually checked that sufficient funds existed. This is **check-then-act** again, exactly as in Attempt 2 of Lecture 5, but now with money.

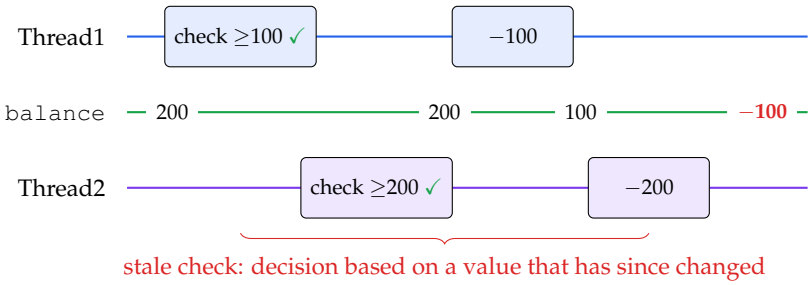


Figure 6.1: Check-then-act on a bank balance.

6.2 Mutual Exclusion vs Synchronization

	Mutual Exclusion	Synchronization
Goal	Ensure at most **1 thread** can be in the critical section at any given time	Make sure different threads are **in sync** with each other
Achieved via	**Atomic operations**	Ordering constraints
Tools	**Locks, Monitors, Mutexes**	**Semaphores** and some patterns
Examples	Shared counter, bank balance	Producer/consumer, readers/writers

Requirements of a good parallel algorithm

1. Safety (consistency) → prevent race conditions. **2. Liveness** → avoid deadlock and starvation. **3. Efficiency.**

6.3 Deadlocks

Definition

A set of threads/processes is **deadlocked** if **each thread/process in the set is waiting for an event that only another process in the set can cause**. Deadlocks occur when processes are granted **exclusive access to devices**. We refer to these devices generally as **resources**.

6.3.1 The Four Conditions for Deadlock (Coffman conditions)

All four must hold simultaneously

1. Mutual exclusion condition. Each resource is assigned to 1 process, or is available.
2. Hold and wait condition. A process holding resources can request additional ones.
3. No preemption condition. Previously granted resources cannot be forcibly taken away.
4. Circular wait condition. There must be a **circular chain of 2 or more processes**, each waiting for a resource held by the next member of the chain.

Why "all four" matters

Break **any one** condition and deadlock becomes impossible. Most practical strategies attack **circular wait** by imposing a global lock ordering, because the other three are usually unavoidable.

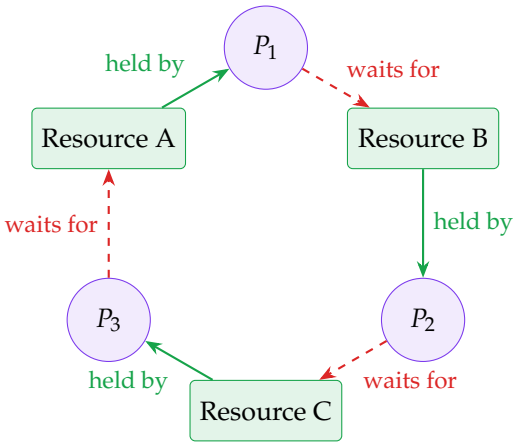


Figure 6.2: A cycle in the resource-allocation graph is the circular-wait condition made visible.

6.4 Livelock and Starvation

Livelock

Each thread/process is **repeating the same state again and again, but does not progress any further**. Example: two people trying to give way to each other in a doorway, stepping aside in perfect symmetry forever.

Starvation

Example: preemptive clients in a public office. Often **caused by priority**. Handled by adding **round robin** nature to algorithms, or by **elevating priorities** over time (ageing). **If you have an algorithm that has some kind of priority, you have to watch out.**

	Deadlock	Livelock	Starvation
CPU activity	Blocked, idle	**Busy**, spinning	Busy or blocked
Who is stuck	All in the set	All in the set	**One** unlucky thread
System progress	None	None	Continues normally
Escape possible?	Never	Yes, any deviation from the sequence	Yes, if scheduling changes
Typical cause	Circular wait	Symmetric back-off	Priority or unfair ordering

6.5 The Lock Interface

```
public interface Lock {
    public void lock();      // acquire lock
    public void unlock();    // release lock
}
```

Naming

`acquire()` and `release()` are also used as names for these two operations.

6.5.1 Using Locks

```
Lock x;
...
x.lock();
// critical section
x.unlock();
...
```

6.5.2 Example: A Correct Counter

```
public class Counter {
    private long value;
    private Lock lock;

    public long getAndIncrement() {
        lock.lock();                    // acquire lock
        try {
            int temp = value;
            value = value + 1;
        } finally {
            lock.unlock();              // release lock (no matter what)
        }
        return temp;
```

```
    }
}
```

Why `try` / `finally` is not optional

If the critical section throws an exception, the `finally` block still runs and the lock is still released. **Without it, one exception deadlocks the entire program forever.**

Locks in Java

Look at `java.util.concurrent.locks.*`, specifically the `Lock` interface and `ReentrantLock`.

6.6 Exercise: Lock Ordering

Will a combination of 2 locks be deadlock free?

```
Lock L1, L2;

// Case A                          // Case B
Thread 1          Thread 2        Thread 1          Thread 2
  L1.lock()         L1.lock()       L1.lock()         L2.lock()
  L2.lock()         L2.lock()       L2.lock()         L1.lock()
```

Solution

Case A is deadlock free. Both threads acquire locks in the **same global order** L1 then L2. Whichever thread obtains L1 first will obtain L2 unobstructed, because the other thread cannot hold L2 without first holding L1. **The circular wait condition is broken.**
Case B can deadlock:

Step	Thread 1	Thread 2
1	acquires L1	
2		acquires L2
3	blocks waiting for L2	
4		blocks waiting for L1

A perfect cycle. Neither will ever proceed.

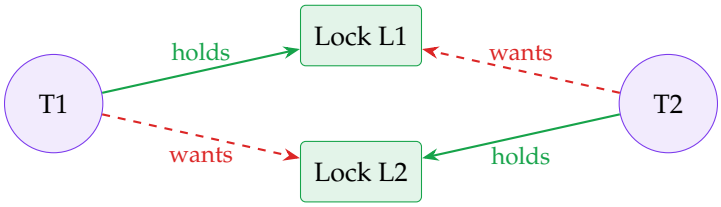


Figure 6.3: Case B: inconsistent lock order creates a cycle.

The rule

Define a **total order over all locks** in the program and always acquire them in that order. This is the single most effective deadlock-prevention discipline in practice.

6.7 Building a Lock: 2-Thread Solutions First

Strategy

2-thread solutions first, because they illustrate the most basic ideas. **Then n-thread solutions.**

```
class ... implements Lock {
    // thread-local index, 0 or 1
    public void lock() {
        int i = ThreadID.get();
        int j = 1 - i;
        ...
    }
}
```

Convention for the rest of this lecture

i is the current thread, j is the other thread.

6.7.1 LockOne

```
class LockOne implements Lock {
    private volatile boolean[] flag = new boolean[2];

    public void lock() {
        flag[i] = true;            // Set my flag
        while (flag[j]) {}         // Wait for other flag to go false
    }

    public void unlock() {
        flag[i] = false;
    }
}
```

LockOne satisfies mutual exclusion

This can be proven. The argument is the same "raise flag, then look" contradiction proof used for Attempt 3 in Lecture 5.

LockOne fails deadlock-freedom

Concurrent execution can deadlock:

```
flag[i] = true;              flag[j] = true;
while (flag[j]) {}           while (flag[i]) {}
```

Both threads set their flags, then both spin forever. **This is Lecture 5's Attempt 3, restated as code.**

6.7.2 LockTwo

```
public class LockTwo implements Lock {
    private volatile int victim;

    public void lock() {
        victim = i;                    // Let the other go first
        while (victim == i) {};        // Wait for permission
    }

    public void unlock() {}            // Nothing to do
}
```

LockTwo satisfies mutual exclusion

If thread *i* is in the critical section, then `victim == j`. It cannot be both 0 and 1 at once, so both threads cannot be in the critical section together.

LockTwo deadlocks in the absence of contention

Concurrent execution does **not** deadlock. **Sequential** execution **deadlocks**: **a thread alone cannot get in.**

```
public void LockTwo() {
    victim = i;
    while (victim == i) {};      // nobody will ever overwrite victim
}
```

LockTwo depends entirely on the other thread showing up to change `victim`.

The complementary failure

	LockOne	LockTwo
Works when	Threads run **sequentially**	Threads run **concurrently**
Fails when	Threads run **concurrently**	One thread runs **alone**

They fail in **exactly opposite** situations. That is the clue that combining them will work.

6.8 Peterson's Algorithm

LockOne + LockTwo = a correct lock

```
public void lock() {
    flag[i] = true;                          // Announce I'm interested
    victim  = i;                             // Defer to other
    while (flag[j] && victim == i) {};       // Wait while other
   interested
}                                            //   AND I'm the victim

public void unlock() {
    flag[i] = false;                         // No longer interested
}
```

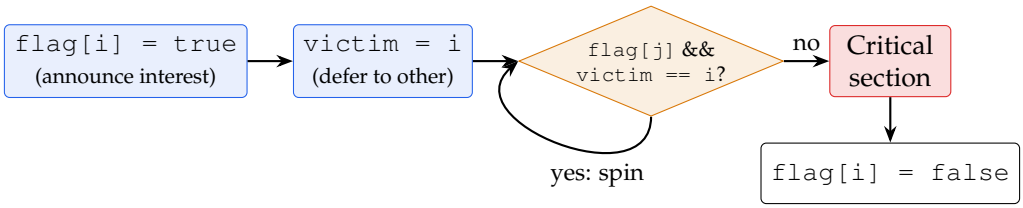


Figure 6.4: Peterson's lock for thread i.

Proof of mutual exclusion

If thread 0 is in the CS: `flag[0] == true` and `victim == 1`. If thread 1 is in the CS: `flag[1] == true` and `victim == 0`. **`victim` cannot be both 0 and 1.** Both cannot be true. ■

Proof of deadlock freedom

A thread is blocked **only** at the `while` loop, and **only if it is the victim**. `victim` holds a single value, so **one or the other must not be the victim**. That thread proceeds. ■

Proof of starvation freedom

Thread i is blocked only if j **repeatedly re-enters**, so that `flag[j] == true` and `victim == i`. But when j re-enters, **it sets `victim` to** j. So i's condition `victim == i` becomes false and i **gets in**. ■

6.9 The Filter Algorithm for n Threads

The idea

There are $n-1$ **"waiting rooms" called levels**. At each level: **at least 1 thread enters** level ℓ, and **at least 1 is blocked** if many try to enter level ℓ. **Only 1 thread makes it through** all $n-1$ levels into the critical section.

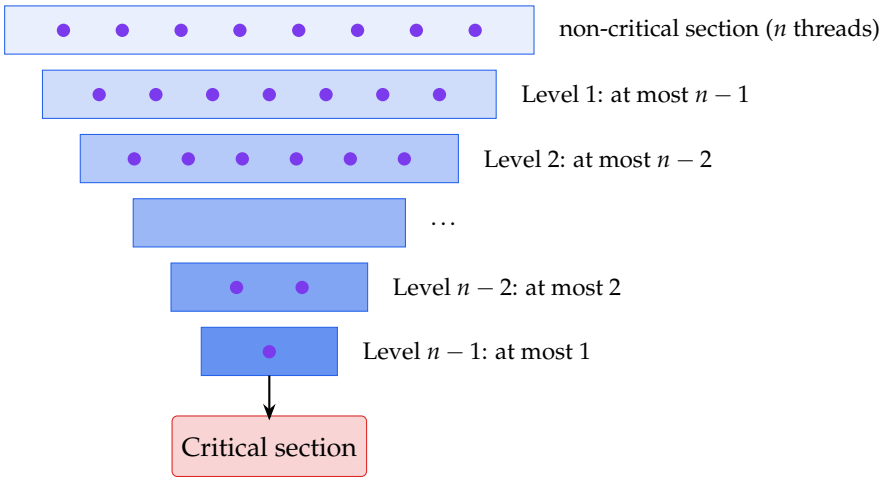


Figure 6.5: The Filter lock as a funnel: each level filters out at least one contender.

Filter is essentially **Peterson's algorithm applied $n-1$ times in series**, each round eliminating at least one contender.

6.9.1 The Code

```
class Filter implements Lock {
    volatile int[] level;    // level[i]  for thread i
    volatile int[] victim;   // victim[L] for level L

    public Filter(int n) {
        level  = new int[n];
        victim = new int[n];
        for (int i = 1; i < n; i++) {
            level[i] = 0;
        }
    }

    public void lock() {
        for (int L = 1; L < n; L++) {        // one level at a time
            level[i]  = L;        // announce intention to enter level L
            victim[L] = i;        // give priority to anyone but me
            // wait while someone else is at same or higher level
            // AND I'm the designated victim
            while ((exists k != i) (level[k] >= L)
                   && victim[L] == i) {};
        }
    }

    public void unlock() {
        level[i] = 0;
    }
}
```

Reading the two arrays

- **`level[i]`** = how far thread i has climbed. `level[2] = 4` means thread 2 is at level 4.
- **`victim[L]`** = which thread most recently arrived at level L, and is therefore the one who must wait.

A thread **enters level L when it completes the loop body** for that L.

Filter lock claim

Start at level $L = 0$: **At most $n - L$ threads enter level L.** Therefore at level $L = n - 1$ **at most one thread** is present, which gives **mutual exclusion**.

No starvation

The Filter lock satisfies the same properties **just like Peterson's algorithm at any level**, so **no one starves**.

But what about fairness?

Threads can be overtaken by others. A thread may sit at level 3 while other threads repeatedly arrive, climb past it, use the critical section, and come back around. It will get in eventually, but there is **no bound** on how long that takes.

6.10 Bounded Waiting

Why starvation-freedom is not enough

Starvation-freedom guarantees that **every thread that calls `lock()` eventually enters the critical section**, but gives **no guarantee about how long it will take**. We want a **stronger fairness guarantee**: a thread must not be **"overtaken" too much**.

6.11 The Bakery Algorithm

The idea

Provides **First-Come-First-Served**. How? **Take a "number"** (like a ticket dispenser at a bakery) and **wait until lower numbers have been served**.

Lexicographic order

$$(a, i) < (b, j) \iff a < b, \text{ or } a = b \text{ and } i < j$$

The thread ID is the tie-breaker.

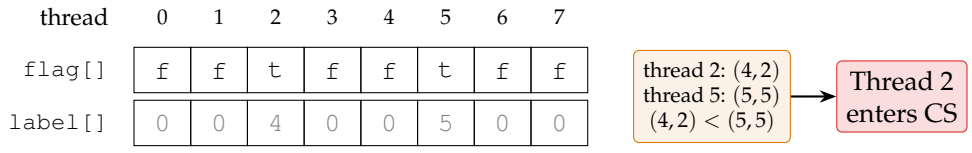


Figure 6.6: Bakery: among interested threads, the smallest (label, id) pair goes first.

6.11.1 The Code

```
class Bakery implements Lock {
    volatile boolean[] flag;
```

```
    volatile Label[]   label;

    public Bakery(int n) {
        flag  = new boolean[n];
        label = new Label[n];
        for (int i = 0; i < n; i++) {
            flag[i] = false;
            label[i] = 0;
        }
    }

    public void lock() {
        flag[i]  = true;                                  // 1. I'm interested
        label[i] = max(label[0], ..., label[n-1]) + 1; // 2. Take increasing
     label
        //  ^^^^^^^^^^^^^^^^ the doorway ^^^^^^^^^^^^^^^^
        // wait for anyone interested with a lower (label, id)
        while ((exists k) flag[k] &&
               (label[i], i) > (label[k], k)) {};
    }

    public void unlock() {
        flag[i] = false;                                    // No longer
    interested
    }
}
```

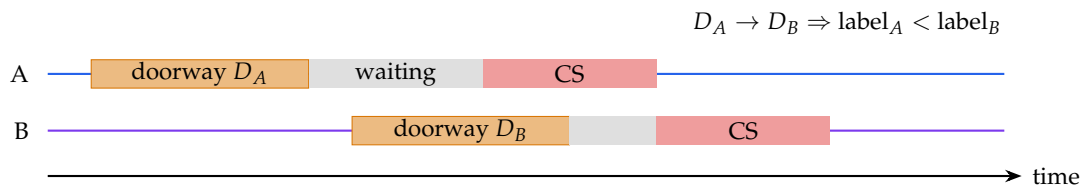


Figure 6.7: First-come-first-served: if A's doorway finishes before B's starts, A enters first.

The doorway

The first two statements (`flag[i] = true` and taking a label) form the **doorway**. It is important that the doorway completes in a **bounded number of steps**, since that is what gives first-come-first-served its meaning. **Labels are always increasing.**

6.11.2 Properties

No deadlocks

There is always one thread with the earliest label. That thread's while-condition is false for every k, so it enters. **Ties are impossible.** Two threads may compute the **same** `label` value, but the pair $(\text{label}, \text{id})$ still differs because **thread IDs are distinct**. The lexicographic order is therefore a **total** order on contenders.

First-Come-First-Served

If $D_A \to D_B$ (A's doorway completes before B's doorway begins), then **A's label is smaller**, so A enters first. This is exactly the **bounded waiting** guarantee that Filter lacked.

Mutual exclusion

Labels are strictly increasing, so no two threads in the critical section can both hold the minimum (label, id) pair.

Will the solution fail if `max()` is not atomic?

No. Two threads executing `max()` concurrently may read overlapping snapshots and end up with the **same** label value. That is fine, because the comparison is on the **pair** (label, id), and IDs are unique. The tie is broken deterministically and exactly one of them proceeds. This is what makes Bakery remarkable: it needs **no atomic operation beyond safe reads and writes**, yet works for n threads with first-come-first-served fairness.

6.12 Comparison of All Lock Implementations

Algorithm	Threads	Mutual excl.	Deadlock free	Starvation free	Bounded wait / FCFS
LockOne	2	✓	×	×	×
LockTwo	2	✓	× (alone)	×	×
Peterson	2	✓	✓	✓	✓
Filter	n	✓	✓	✓	×
Bakery	n	✓	✓	✓	✓

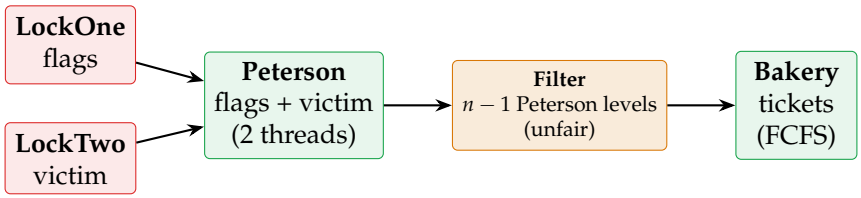


Figure 6.8: The progression of lock algorithms.

The progression

LockOne and **LockTwo** each solve half the problem and fail in complementary situations. **Peterson** combines them and is complete for 2 threads. **Filter** generalises Peterson to n threads by chaining $n - 1$ Peterson-like rounds, but loses fairness. **Bakery** recovers fairness by replacing "who was here last" with an explicit **ticket number**.

6.13 Summary

Lecture Summary

- Concurrency is hard because of **data consistency** and **cooperation**, both worsened by **no timing guarantees**.
- **Mutual exclusion** uses locks, monitors, mutexes. **Synchronization** uses semaphores and patterns. They are different problems.
- A good parallel algorithm needs **safety, liveness, and efficiency**.
- **Four conditions for deadlock:** mutual exclusion, hold and wait, no preemption, circular

wait. Break any one to prevent it.

- **Consistent global lock ordering** is the practical cure for circular wait.
- **Livelock** is busy and stuck. **Starvation** hits one thread while the system runs fine.
- Always release locks in a **`finally`** block.
- **LockOne** (flags) deadlocks under concurrency. **LockTwo** (victim) deadlocks when running alone.
- **Peterson** = flags + victim. Mutual exclusion, deadlock freedom and starvation freedom all follow from `victim` holding one value.
- **Filter**: $n-1$ levels, at most $n-L$ threads at level L. Correct but threads can be **overtaken**.
- **Bakery**: take a ticket, compare $(\text{label}, \text{id})$ lexicographically. **First-come-first-served**, and it works even if `max()` is not atomic.

Self-check Questions

1. Construct the interleaving that overdraws the bank account, and state which condition it violates.
2. List the four conditions for deadlock and give a practical technique for breaking each.
3. Why does `LockOne` fail under concurrency while `LockTwo` fails in isolation?
4. Prove Peterson's mutual exclusion in two lines using the value of `victim`.
5. In Filter, explain why at most $n-L$ threads can be at level L.
6. Give a concrete scenario in which a Filter-locked thread is overtaken three times.
7. Why are ties impossible in the Bakery algorithm?
8. Explain why a non-atomic `max()` does not break the Bakery algorithm.
9. Rank Peterson, Filter, and Bakery by fairness and explain the trade-off each makes.

LECTURE 7

Introduction to Accelerators

Overview

Accelerators are specialised processors attached to a host CPU to speed up numerically intensive, **data-parallel** work. This lecture covers the **GPU** as a many-core throughput engine, the **CUDA** programming model (host/device split, grids, blocks, threads, device memory management, qualifiers, atomics, error handling), and the **Intel Xeon Phi** many-core alternative with its **Knights Landing** architecture and MCDRAM memory modes.

Outline

GPUs → CUDA programming → Xeon Phi

7.1 What is a GPU?

Definition

A **Graphics Processing Unit (GPU)** is typically a computer card installed into a **PCI Express slot**. Market leaders: **NVIDIA**, **AMD (ATI)**. Intel's entry is the **MIC (Many Integrated Core)** line. NVIDIA GPUs are the ones available at UoM (e.g. GeForce GTX 480, Tesla 2070).

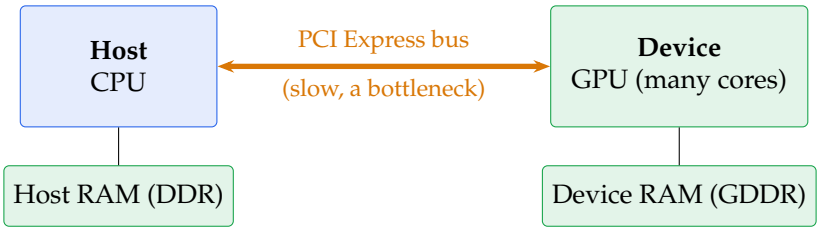


Figure 7.1: An accelerator system: a host CPU and a GPU card, each with its own memory, joined by PCIe.

7.1.1 Why GPUs Exist

The evolution

1. Originally designed to accelerate the **large number of computations performed in graphics rendering**.
2. This **offloaded numerically intensive computation from the CPU**.
3. GPUs grew with the demand for **high-performance graphics**.
4. Eventually GPUs became **much more powerful than CPUs for many computations**.
5. They hold a **cost–power–performance advantage**.

Today's GPUs

- High-performance, **many-core** processors usable to accelerate a **wide range of applications**.
- GPUs have led the race for **floating-point performance since the start of the 21st century**.
- **GPGPU** = using GPUs as **general-purpose parallel processors**, not just for graphics.

7.2 Example Specifications

	GTX 480	Tesla 2070	Tesla K80
Peak **double** precision FP	650 Gigaflops	515 Gigaflops	**2.91 Teraflops**
Peak **single** precision FP	1.3 Teraflops	1.03 Teraflops	**8.74 Teraflops**
CUDA cores	480	448	**4992**
Frequency of CUDA cores	1.40 GHz	1.15 GHz	560/875 MHz
Memory size (GDDR5)	1536 MB	6 GB	**24 GB**
Memory bandwidth	177.4 GB/sec	150 GB/sec	**480 GB/sec**
ECC memory	No	Yes	Yes

Read the table sideways

Notice that the **clock speed goes down** while performance goes up. GPU performance comes from **core count and memory bandwidth**, not from clock frequency. That is the entire design philosophy in one row.

7.3 CPU vs GPU Architecture

The single most important slide

The GPU devotes more transistors to computation. A CPU spends its transistor budget on **control logic and large caches** so that a *single* thread runs as fast as possible. A GPU spends it on **ALUs**, so that *thousands* of threads run at once, each slowly.

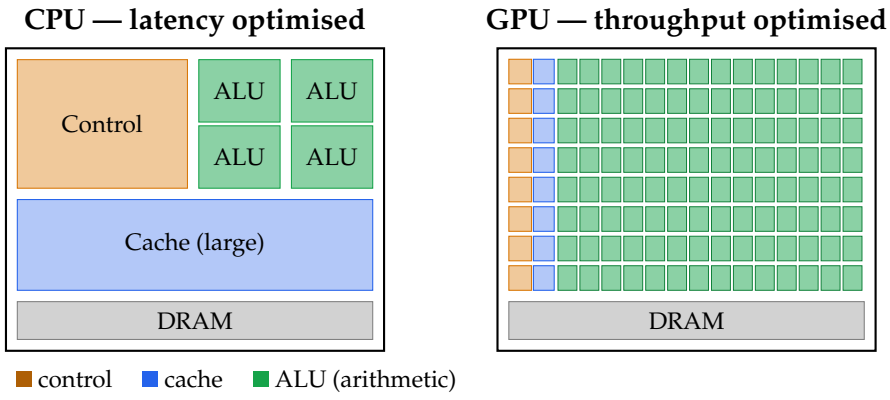


Figure 7.2: Where the transistors go: a few big, cache-backed cores vs. many rows of small ALUs with tiny control and cache.

7.3.1 FLOPS and GFLOPS

Definition

FLOPS = floating-point operations per second.

$$\text{Performance} = \text{cores} \times \text{FLOPS per core (per cycle)} \times \text{clock speed}$$

Worked example from the slides

	CPU	GPU
Number of cores	4	448
FLOPS per core	4	1
Clock speed (GHz)	2.5	1.15
Performance (GFLOPS)	**40**	**515**

CPU: $4 \times 4 \times 2.5 = 40$ GFLOPS. GPU: $448 \times 1 \times 1.15 = 515$ GFLOPS.
The GPU wins with fewer FLOPS per core and a lower clock — purely on core count.

The performance gap

Plots of peak GFLOPS over time show the **CPU–GPU performance gap widening** year on year, which is why GPGPU became attractive.

7.4 A Simple Performance Test

Matrix multiplication $C = B \times A$

CPU: Intel Core i7, `cblas_dgemm(...)` from the BLAS library. **GPU:** NVIDIA GTX 480.

Dimensions (A, B, C)	CPU time (s)	GPU time (s)	**Speedup**
[400,800], [400,400], [400,800]	0.17	0.00109	155
[800,1600], [800,800], [800,1600]	2.10	0.00846	**258**
[1200,2400], [1200,1200], [2400,2400]	6.65	0.02860	232
[1600,2400], [1600,1600], [2400,2400]	15.18	0.06739	225
[2000,4000], [4000,4000], [4000,4000]	29.44	0.13178	223
[2400,4800], [2400,4800], [4800,4800]	50.21	0.22703	221

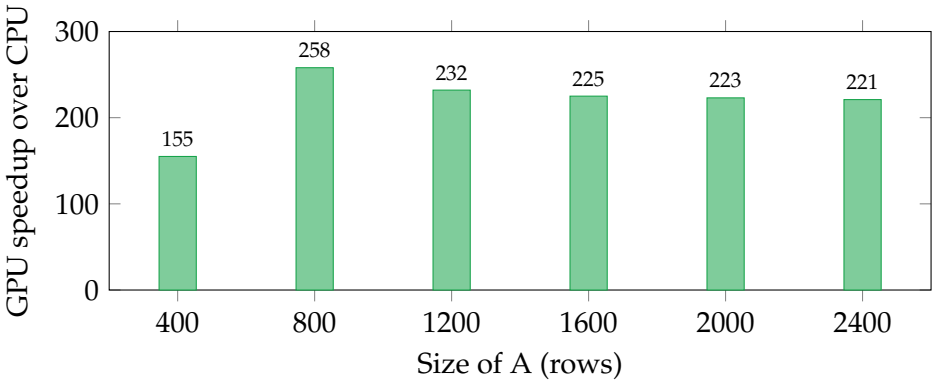


Figure 7.3: GTX 480 vs. Core i7 BLAS: speedup rises then plateaus around 220–260×.

What the numbers say

Speedup **rises then plateaus around 220–260×**. Matrix multiply is the ideal GPU workload: massively data-parallel, high arithmetic intensity, regular memory access.

Applications of GPGPU (from the slides)

Computational Structural Mechanics · Bio-Informatics and Life Sciences · Computational Electromagnetics & Electrodynamics · Computational Finance · Computational Fluid Dynamics · Data Mining, Analytics & Databases · Imaging & Computer Vision · Medical Imaging · Molecular Dynamics · Numerical Analytics · Weather, Atmospheric, Ocean Modeling & Space Sciences

7.5 Programming GPUs

Option	Notes
CUDA	For **NVIDIA** GPU products. **C**ompute **U**nified **D**evice **A**rchitecture. Based on **C**, compiled with **`nvcc`**. Lots of tools for analysis, debug, profile.
OpenCL	**Open Computing Language**. Based on C. Supports **GPU *and* CPU** programming, i.e. portable across vendors.
Others	Support for **Java, Python, Matlab**, etc. Lots of active research, e.g. **automatic code generation for GPUs**.

Caution!

- A GPU is designed as a **numeric computing engine**. It **will not perform well on some tasks** the way CPUs do.
- **Most applications will use both CPUs and GPUs.**
- For some computations the **cost to transfer data between CPU and GPU can be high**; this transfer often dominates the runtime and can wipe out the speedup entirely.
- **SIMD-type data parallelism is key** to benefiting from GPUs.

7.6 CUDA Architecture

Two main parts

1. Host (CPU part) — Single Program, **Single** Data. **2. Device (GPU part)** — Single Program, **Multiple** Data.
CUDA is NVIDIA's solution to access the GPU, and can be seen as an **extension to C/C++** with its own software stack.

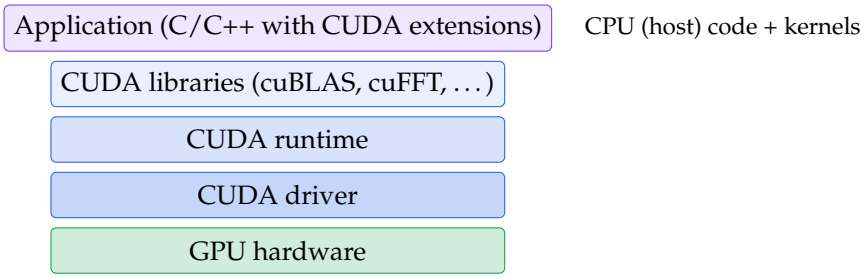


Figure 7.4: The CUDA software stack.

7.6.1 Grid, Block, Thread

The three-level hierarchy

- **Grid** — a group of **threads all running the same kernel**.
- **Block** — grids are composed of blocks. Each block is a **logical unit containing a number of coordinating threads and some amount of shared memory**.
- **Thread** — the individual worker, identified within its block.

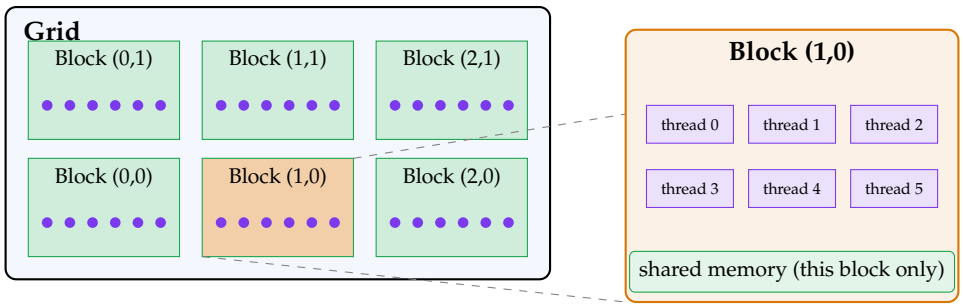


Figure 7.5: A kernel launch is a grid of blocks; each block is a group of threads with its own shared memory.

Global thread index (1-D)

$$\text{tid} = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}$$

With `blockDim.x = 6`, block 0 covers threads 0–5, block 1 covers 6–11, block 2 covers 12–17.

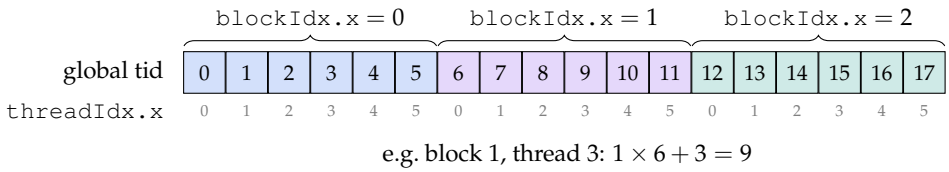


Figure 7.6: Mapping (block, thread) to a global index with `blockDim.x = 6`.

7.6.2 Hello World

```
#include <cuda.h>
#include <stdio.h>

__global__ void kernel (void) { }       // runs on the DEVICE

int main (void)
{
    kernel <<< 1, 1 >>> ();             // 1 block, 1 thread
    printf("Hello World!\n");
    return 0;
}
```

Reading the syntax

- **`__global__`** says the function is to be compiled to run on a **"device" (GPU)**, not the **"host" (CPU)**.
- **Angle brackets «< and >»** pass parameters to the **runtime**: the grid size (number of blocks) and block size (number of threads).
- A function executed on the GPU (device) is called a **kernel**.

7.7 CUDA Device Memory Model

Separate address spaces

Host and device have **separate memory spaces** (e.g. hardware cards with their own DRAM). A host pointer is meaningless on the device and vice versa.

The four-step ritual for every kernel

1. **Allocate** memory on the device — `cudaMalloc()`
2. **Transfer** data: host memory → device memory — `cudaMemcpy(..., cudaMemcpyHostToDevice)`
3. **Execute** the kernel on the device
4. **Transfer** results back: device → host, then **free** device memory — `cudaMemcpy(..., cudaMemcpyDeviceToHost)`, `cudaFree()`

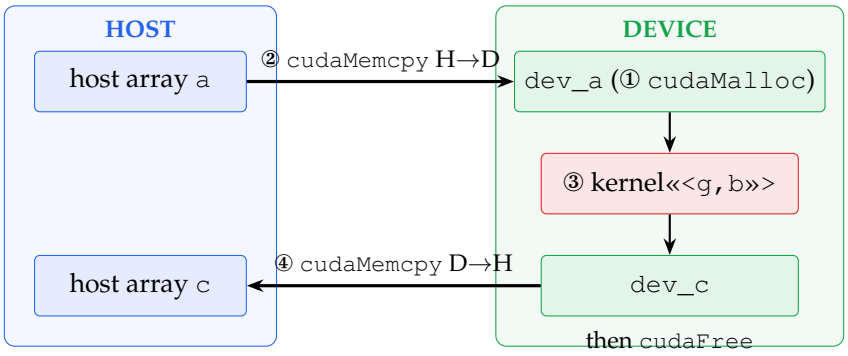


Figure 7.7: The host–device round trip for every kernel.

7.7.1 Minimal Memory Example

```
int main (void) {
    int c, *dev_c;

    cudaMalloc((void **) &dev_c, sizeof(int));
    add <<< 1, 1 >>> (2, 7, dev_c);
    cudaMemcpy(&c, dev_c, sizeof(int), cudaMemcpyDeviceToHost);
    printf("2 + 7 = %d\n", c);
    cudaFree(dev_c);

    return 0;
}
```

7.7.2 Vector Addition — One Block per Element

```
#include "stdio.h"
#define N 10

__global__ void add(int *a, int *b, int *c)
{
    int tID = blockIdx.x;           // one block per element
    if (tID < N) {
        c[tID] = a[tID] + b[tID];
    }
}

int main()
{
    int a[N], b[N], c[N];
    int *dev_a, *dev_b, *dev_c;

    cudaMalloc((void **) &dev_a, N*sizeof(int));
    cudaMalloc((void **) &dev_b, N*sizeof(int));
    cudaMalloc((void **) &dev_c, N*sizeof(int));

    for (int i = 0; i < N; i++) a[i] = i, b[i] = 1;

    cudaMemcpy(dev_a, a, N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(dev_b, b, N*sizeof(int), cudaMemcpyHostToDevice);

    add<<<N,1>>>(dev_a, dev_b, dev_c);        // N blocks, 1 thread each

    cudaMemcpy(c, dev_c, N*sizeof(int), cudaMemcpyDeviceToHost);
```

```
    for (int i = 0; i < N; i++)
        printf("%d + %d = %d\n", a[i], b[i], c[i]);
    return 0;
}
```

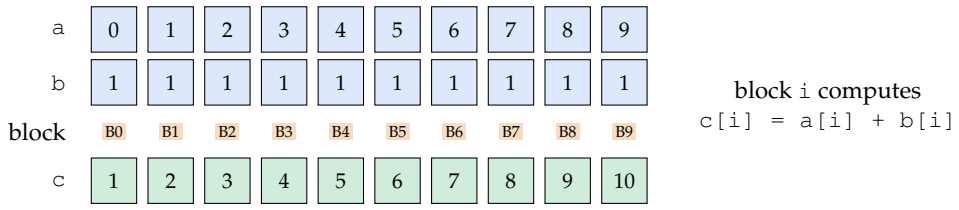


Figure 7.8: `add<<<N, 1>>>`: one block (with a single thread) per vector element.

The `if (tID < N)` guard

Grid dimensions are usually rounded up to whole blocks, so **more threads are launched than there is data**. Every kernel needs a bounds check, or the extra threads write out of range.

7.8 Case Study: Matrix Multiplication in CUDA

7.8.1 The Original CPU Kernel

```
// a = b * c
void mmult(float *a, float *b, float *c, int N)
{
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            for (int k = 0; k < N; k++)
                a[j+i*N] += b[k+i*N] * c[j+k*N];
}
```

Matrix representation in memory

Matrices are stored in **row-major order**, so element (i, j) of an $n \times n$ matrix lives at index `j + n*i`.

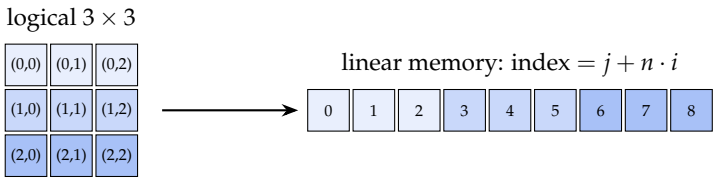


Figure 7.9: Row-major storage: rows are laid out one after another.

7.8.2 Mapping Loops onto the Grid

The transformation

The **two outer loops disappear**: `i` and `j` are no longer loop counters but **thread coordinates**. Only the innermost **reduction loop over k survives** inside the kernel.

```
dim3 grid(1024/32, 1024);   // 32 x 1024 grid of thread blocks
```

```
dim3 threads(32);           // 32 x 1 x 1 threads per block
```

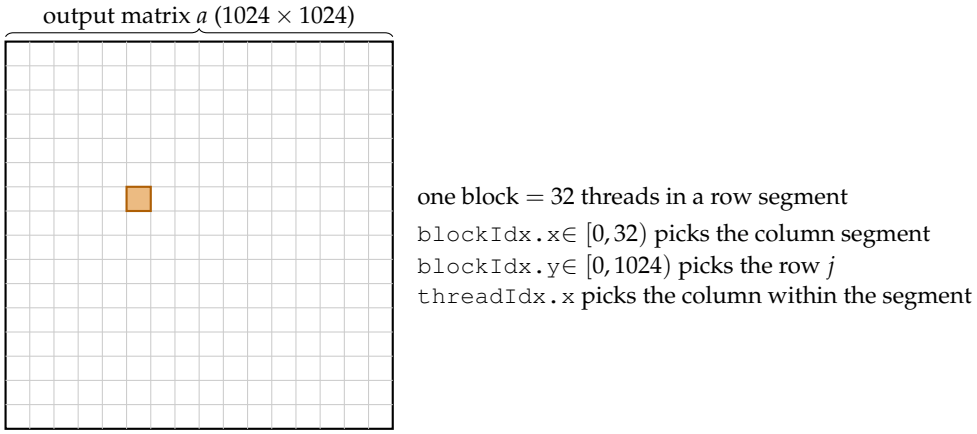


Figure 7.10: Each thread computes one element $a_{j,i}$; a block covers 32 consecutive columns of one row (schematic).

7.8.3 The GPU Kernel

```
__global__
void mmult(float *a, float *b, float *c, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;    // column
    int j = blockIdx.y;                               // row
    float sum = 0.0;

    for (int k = 0; k < N; k++)
        sum += b[k+N*j] * c[i+N*k];

    a[i+N*j] = sum;
}

// launch
dim3 dimGrid(32, 1024);
dim3 dimBlock(32);
mmult<<<dimGrid, dimBlock>>>(devPtrA, devPtrB, devPtrC, N);
```

	CPU version	GPU version
Loop over `i`	explicit `for`	`blockIdx.x * blockDim.x + threadIdx.x`
Loop over `j`	explicit `for`	`blockIdx.y`
Loop over `k`	explicit `for`	**still an explicit `for`**
Accumulation	`a[...]` += in memory	local `sum`, written once

7.8.4 The Full Host Program

```
int main(int argc, char* argv[])
{
    int N = 1024;
```

```
    float *a = (float *)malloc(N*N*sizeof(float));
    float *b = (float *)malloc(N*N*sizeof(float));
    float *c = (float *)malloc(N*N*sizeof(float));
    minit(a, b, c, N);

    // allocate device memory
    float *devPtrA, *devPtrB, *devPtrC;
    cudaMalloc((void**)&devPtrA, N*N*sizeof(float));
    cudaMalloc((void**)&devPtrB, N*N*sizeof(float));
    cudaMalloc((void**)&devPtrC, N*N*sizeof(float));

    // copy input arrays to the device memory
    cudaMemcpy(devPtrB, b, N*N*sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(devPtrC, c, N*N*sizeof(float), cudaMemcpyHostToDevice);

    // define grid and thread block sizes
    dim3 dimGrid(32, 1024);
    dim3 dimBlock(32);

    // launch GPU kernel
    mmult<<<dimGrid, dimBlock>>>(devPtrA, devPtrB, devPtrC, N);

    // check for errors
    cudaError_t err = cudaGetLastError();
    if (cudaSuccess != err) {
        fprintf(stderr, "CUDA error: %s.\n", cudaGetErrorString(err));
        exit(EXIT_FAILURE);
    }

    // wait until GPU kernel is done
    cudaThreadSynchronize();

    // copy results to host
    cudaMemcpy(a, devPtrA, N*N*sizeof(float), cudaMemcpyDeviceToHost);

    cudaFree(devPtrA); cudaFree(devPtrB); cudaFree(devPtrC);
    free(a); free(b); free(c);
}
```

Kernel launches are asynchronous

`mmult«<...»>` **returns immediately**. `cudaThreadSynchronize()` (now `cudaDeviceSynchronize()`) is what makes the host wait until the GPU is done. Timing a kernel without synchronising measures nothing but launch overhead.

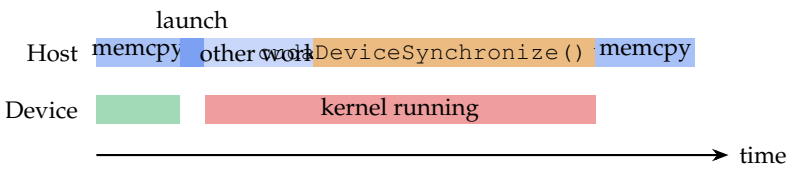


Figure 7.11: The launch returns at once; the host blocks only when it synchronises.

7.9 CUDA Language Reference

7.9.1 Function Type Qualifiers

Qualifier	Executed on the:	Only callable from the:
`__device__ float DeviceFunc()`	device	device
`__global__ void KernelFunc()`	device	**host**
`__host__ float HostFunc()`	host	host

Restrictions

- `__device__` and `__global__` functions **do not support recursion** and **cannot declare static variables** inside their body (true of early compute capabilities, as in the slides).
- `__host__` and `__device__` **can be used together**; the function is then compiled for both.
- `__global__` and `__host__` **cannot be used together**.
- A `__global__` function must have **`void` return type**, its **execution configuration must be specified**, and the **call is asynchronous**.

7.9.2 Variable Type Qualifiers

Declaration	Memory	Scope	Lifetime
`__device__ int GlobalVar;`	global	grid	application
`__device__ __shared__ int SharedVar;`	**shared**	**block**	**block**
`__device__ __constant__ int ConstantVar;`	constant	grid	application

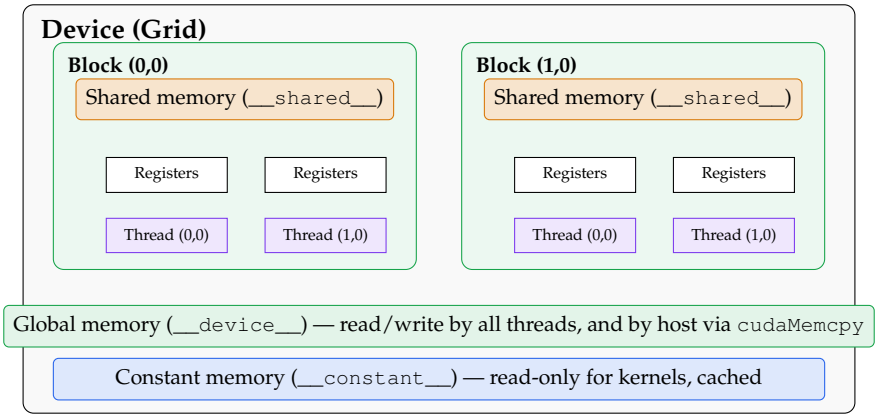


Figure 7.12: CUDA memory spaces: per-thread registers, per-block shared memory, grid-wide global and constant memory.

7.9.3 Execution Configuration

```
__global__ void kernel(float* param);

kernel<<<Dg, Db, Ns, S>>>(param);
```

The four configuration parameters

- **`Dg`** (type `dim3`) — dimension and size of the **grid**, such that `Dg.x*Dg.y*Dg.z` equals the **number of blocks** launched.
- **`Db`** (type `dim3`) — dimension and size of each **block of threads**, such that `Db.x*Db.y*Db.z` equals the **number of threads per block**.
- **`Ns`** (optional, `size_t`) — bytes of **shared memory dynamically allocated per block** for this call, **in addition to** the statically allocated memory.
- **`S`** (optional, `cudaStream_t`) — the **stream** associated with this kernel call.

7.9.4 Built-in Variables

Variable	Type	Description
`gridDim`	`dim3`	dimensions of the **grid**
`blockIdx`	`uint3`	**block index** within the grid
`blockDim`	`dim3`	dimensions of the **block**
`threadIdx`	`uint3`	**thread index** within the block
`warpSize`	`int`	**warp size** in threads (32)

Two prohibitions

It is **not allowed to take the address** of any built-in variable, and **not allowed to assign values** to any of them.

7.9.5 Built-in Vector Types

Derived from basic integer and float types

`char1..4`, `uchar1..4`, `short1..4`, `ushort1..4`, `int1..4`, **`uint1`**, **`uint2`**, **`uint3 (= dim3)`**, **`uint4`**, `long1..4`, `ulong1..4`, `longlong1..2`, `float1..4`, `double1..2`. They are all **structures**:

```
typedef struct { float x, y, z, w; } float4;
```

Note that **`dim3` is based on `uint3`**; unspecified components default to 1.

7.9.6 Intrinsic Functions

Supported on the device only

- **Start with** `__`, as in `__sinf(x)`.
- **End with a rounding-mode suffix**: `_rn` (round-to-nearest-even), `_rz` (round-towards-zero), `_ru` (round-up), `_rd` (round-down), as in `__fadd_rn(x,y)`.
- Families: **mathematical** (`__log10f(x)`), **type conversion** (`__int2float_rn(x)`), **type casting** (`__int_as_float(x)`), and **bit manipulation** (`__ffs(x)`).

7.10 Synchronization, Fencing and Atomics

Function	Description
`void __threadfence()`	Wait until all **global and shared** memory accesses made by the calling thread become visible to **all threads in the device** (global) and **all threads in the block** (shared).
`void __threadfence_block()`	Wait until all global and shared memory accesses made by the calling thread become visible to **all threads in the thread block**.
`void __syncthreads()`	Waits until **all threads in the thread block have reached this point** *and* all their global/shared memory accesses are visible to all threads in the block.

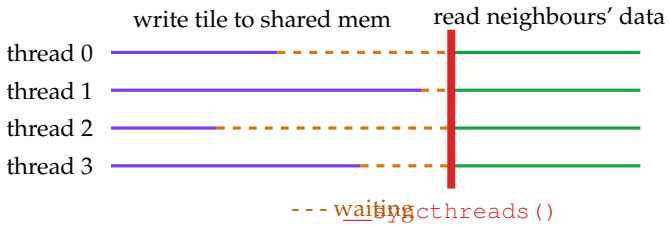


Figure 7.13: A barrier orders execution: no thread passes until every thread in the block arrives.

Fence vs barrier

A **fence** orders *memory*: it says nothing about where other threads are. **`__syncthreads()`** is a **barrier**: it orders *execution* as well. Only `__syncthreads()` makes "everyone has finished writing the tile, now read it" safe.

7.10.1 Atomic Functions

Function	Effect
`atomicAdd()`	`new = old + val`
`atomicSub()`	`new = old - val`
`atomicExch()`	`new = val`
`atomicMin()`	`new = min(old, val)`
`atomicMax()`	`new = max(old, val)`
`atomicInc()`	`new = ((old >= val) ? 0 : (old+1))`
`atomicDec()`	`new = (((old==0) | (old > val)) ? val : (old-1))`
`atomicCAS()`	`new = (old == compare ? val : old)`
`atomicAnd()` / `Or()` / `Xor()`	`old & val`, `old | val`, `old ^ val`

What "atomic" means here

An atomic function performs a **read-modify-write** atomic operation on **one 32-bit or one 64-bit word** residing in **global or shared memory**. The operation is **guaranteed to be performed without interference from other threads** — exactly the guarantee we spent Lectures 5 and 6 building in software. All atomics return the **old** value.

```
// e.g. histogram: many threads may hit the same bin
__global__ void hist(const unsigned char* in, int n, unsigned int* bins) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) atomicAdd(&bins[in[i]], 1);   // bins[k]++ without a lost
    update
}
```

7.11 Device and Memory Management APIs

Function	Description
`cudaGetDeviceCount()`	Returns the number of **compute-capable devices**
`cudaGetDeviceProperties()`	Returns information on the compute device
`cudaSetDevice()`	Sets the device to be used for GPU execution
`cudaGetDevice()`	Returns the device currently being used
`cudaChooseDevice()`	Selects the device that **best matches given criteria**

```
void cudaDeviceInit() {
    int devCount, device;
    cudaGetDeviceCount(&devCount);
    if (devCount == 0) {
        printf("No CUDA capable devices detected.\n");
        exit(EXIT_FAILURE);
    }
    for (device = 0; device < devCount; device++) {
        cudaDeviceProp props;
        cudaGetDeviceProperties(&props, device);
        // If a device of compute capability >= 1.3 is found, use it
        if (props.major > 1) break;
    }
    if (device == devCount) {
        printf("No device above 1.2 compute capability detected.\n");
        exit(EXIT_FAILURE);
    }
    else cudaSetDevice(device);
}
```

Function	Description
`cudaMalloc()`	Allocates memory on the GPU
`cudaMallocPitch()`	Allocates memory for **2D arrays**; may **pad** the allocation to meet alignment requirements
`cudaFree()`	Frees memory allocated on the GPU
`cudaMallocArray()`, `cudaFreeArray()`	Allocates / frees an **array** on the GPU
`cudaMallocHost()`, `cudaFreeHost()`	Allocates / frees **page-locked** memory on the host
`cudaMemset()`	Initialises or sets GPU memory to a value
`cudaMemcpy()`	Copies data between host and device
`cudaMemcpyToArray()`, `cudaMemcpyFromArray()`, `cudaMemcpyArrayToArray()`, `cudaMemcpyToSymbol()`, `cudaMemcpyFromSymbol()`, `cudaGetSymbolAddress()`, `cudaGetSymbolSize()`	Array- and symbol-oriented copy variants (copy into CUDA arrays, or into `__constant__`/`__device__` symbols)

Why page-locked host memory matters

`cudaMallocHost()` memory cannot be swapped out, so the DMA engine can transfer it **without an intermediate staging copy** — noticeably faster transfers, at the cost of pinning physical RAM.

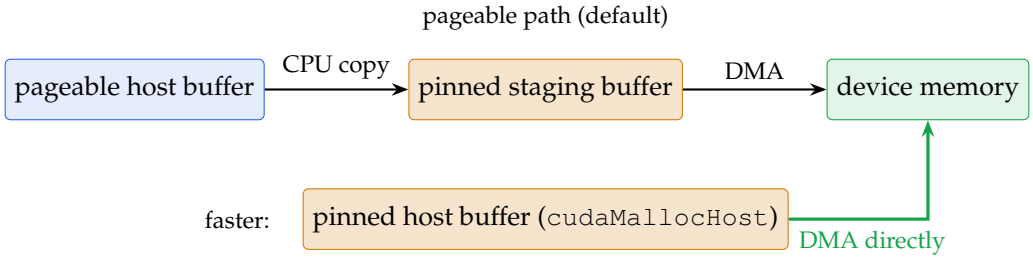


Figure 7.14: Pageable vs. pinned transfers.

7.11.1 Error Handling

Errors are sticky and per-thread

All CUDA runtime API functions return an **error code**. The runtime maintains an **error variable for each host thread**, overwritten by the error code every time an error occurs.

Function	Description
`cudaGetLastError()`	Returns the error variable and **resets it to `cudaSuccess`**
`cudaGetErrorString()`	Returns the **message string** from an error code

```
cudaError_t err = cudaGetLastError();
```

```
if (cudaSuccess != err) {
    fprintf(stderr, "CUDA error: %s.\n", cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
```

Because kernel launches are asynchronous

A kernel launch itself cannot return a runtime fault. The error shows up at the **next** API call, which is why you check `cudaGetLastError()` right after the launch **and** synchronise before trusting results.

7.12 Intel Xeon Phi

The other kind of accelerator

The Xeon Phi family is **many CPUs** rather than many simple shader cores — x86 cores that can run ordinary operating systems and ordinary code, with very wide vectors.

7.12.1 Knights Landing (KNL)

Definition

Knights Landing (KNL) is the code name for the **second-generation Intel Xeon Phi product family**, targeting **high performance computing (HPC)** and other parallel computing segments.

Overview of KNL

- Provides **massive thread-level parallelism** and **data parallelism**
- **High memory bandwidth**
- **Can boot on an off-the-shelf OS** (unlike a GPU, which is always a co-processor)
- Available in **three product varieties**
- Runs **AVX512** instructions — **512-bit vector extensions** of the AVX and AVX2 instruction sets

7.12.2 Architecture

Tiles

38 tiles — 36 active, 2 for recovery (yield harvesting). Each tile has **2 cores**, with **2 VPUs (vector processing units) per core**, and **4 threads per core**.

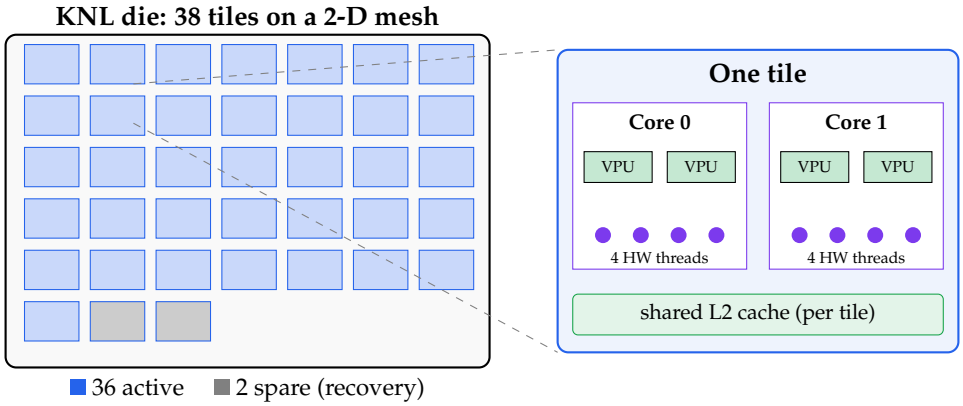


Figure 7.15: Knights Landing: tiles of 2 cores, each core with 2 AVX-512 VPUs and 4 hardware threads.

7.12.3 Memory

	MCDRAM (Multi-Channel DRAM)	**DDR4**
Total space	**16 GB**	**384 GB**
Aggregate bandwidth	more than **450 GB/s**	more than **90 GB/s**
Role	small and very fast	large and slower

Three combinations of these two memory types are available.

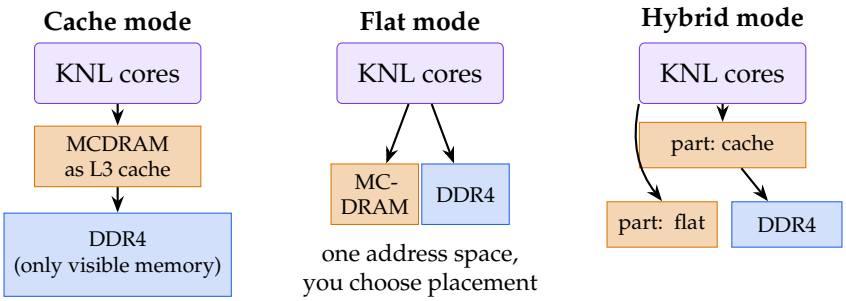


Figure 7.16: MCDRAM modes, selected at boot time in the BIOS.

Memory modes — selected at boot time using BIOS

1. **Cache mode** — MCDRAM acts as a **transparent cache** in front of DDR4. No code changes, no control.
2. **Flat mode** — MCDRAM is **addressable memory** alongside DDR4. Full control, but you must place data yourself.
3. **Hybrid mode — part cache, part flat**.

The trade-off in one line

Cache mode is easy and needs no source changes; **flat mode** is fast if you know which arrays are hot; **hybrid** hedges. All three are a **boot-time BIOS decision**, not a runtime one.

7.13 GPU vs Xeon Phi at a Glance

	GPU (CUDA)	Xeon Phi (KNL)
Core type	very many simple cores	many **x86** cores
Programming model	**CUDA / OpenCL** — new language extensions	ordinary C/C++/Fortran + **AVX512** vectorisation
Runs an OS?	No — co-processor behind PCIe	**Yes**, can boot an off-the-shelf OS
Memory	GDDR5, separate address space, explicit transfers	**MCDRAM + DDR4**, mode chosen in BIOS
Parallelism exposed as	grid / block / thread	threads (4 per core) + wide vectors
Main hazard	**host–device transfer cost**	getting code to actually **vectorise**

7.14 Summary

Lecture Summary

- A **GPU** is a PCIe card that **devotes more transistors to computation** than a CPU does; performance comes from **core count and bandwidth**, not clock speed.
- **GPGPU** is the use of GPUs for general-purpose parallel computation; they have led **floating-point performance since the start of the 21st century**.
- Performance = cores × FLOPS/core × clock; the slide example gives **40 GFLOPS (CPU) vs 515 GFLOPS (GPU)**.
- Matrix multiply on a GTX 480 gives roughly **220–260× speedup** over CBLAS on a Core i7.
- **CUDA** is NVIDIA-only and C-based (`nvcc`); **OpenCL** is open and supports **both GPU and CPU**.
- **SIMD-type data parallelism is key**; **host–device transfer cost can be high**, and most real applications use **both** CPU and GPU.
- CUDA splits work into **grid** → **blocks** → **threads**; a device function is a **kernel**, marked **`__global__`** and launched with **`«<Dg, Db, Ns, S»>`**.
- The global index of a thread is **`blockIdx.x * blockDim.x + threadIdx.x`**.
- Host and device have **separate memory spaces**: `cudaMalloc` → `cudaMemcpy` H2D → kernel → `cudaMemcpy` D2H → `cudaFree`.
- `__global__` functions return **`void`**, cannot recurse, and are **asynchronous**; check with **`cudaGetLastError()`** and wait with **`cudaThreadSynchronize()`**.
- **`__syncthreads()`** is a **block-wide barrier**; **atomics** give interference-free read-modify-write on a single 32/64-bit word.
- **Xeon Phi / Knights Landing**: **AVX512**, 38 tiles (36 active), **2 VPUs per core, 4 threads per core**, and **MCDRAM (16 GB, >450 GB/s) + DDR4 (384 GB, >90 GB/s)** in **cache, flat or hybrid** mode chosen at boot.

Self-check Questions

1. Explain why a GPU with a *lower* clock and *fewer* FLOPS per core beats a CPU by an order of magnitude.
2. Compute the GFLOPS of a device with 2048 cores, 2 FLOPS/core, at 900 MHz.
3. State the four steps required to run any CUDA kernel on data held on the host.
4. Why does every CUDA kernel need a bounds check such as `if (tID < N)`?
5. Rewrite the triple-nested CPU matrix multiply as a CUDA kernel and say which loops disappear and why.
6. Give the memory, scope and lifetime of `__device__`, `__shared__` and `__constant__` variables.
7. Which qualifier pairs may be combined, and which may not?
8. Explain the difference between `__threadfence_block()` and `__syncthreads()`.
9. Why must `cudaGetLastError()` be called immediately after a kernel launch rather than at the end of `main`?
10. Compare MCDRAM cache mode and flat mode, and give a workload that suits each.

LECTURE 8

Semaphores

Overview

A **semaphore** (Dijkstra, 1968) is a non-negative integer variable with exactly two atomic operations, **down** and **up**. It is a programming primitive at a **higher level than machine instructions**, and it solves both **mutual exclusion** and **condition synchronization**. This lecture proves the semaphore invariants, compares three semaphore definitions (**blocked-set, blocked-queue, busy-wait**), and applies semaphores to the classic problems: **producer–consumer**, **dining philosophers**, **readers and writers**, **sleeping barber** and the **baboon crossing**.

Outline

Definition and invariants → Mutual exclusion proofs → Semaphore definitions and starvation → Producer–consumer → Classic synchronization problems

8.1 The Railway Metaphor

Where the name comes from

A **railway semaphore** is a **signal flag that indicates whether or not the track ahead is clear or occupied by another train**. As the train proceeds, semaphores are **set and cleared**. They are a mechanism which ensures **mutually exclusive occupancy of a critical section of track**.

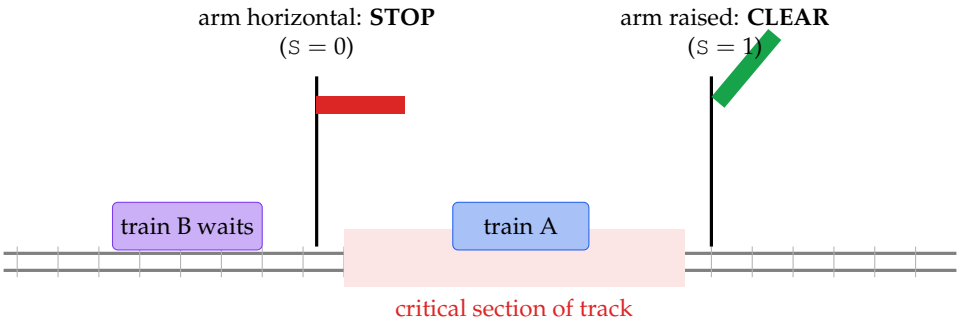


Figure 8.1: A railway semaphore guarding a single section of track: a train may enter only when the arm signals clear.

Where we are in the course

Previously we considered algorithms which run on a **bare machine** — Peterson, Filter, Bakery — using **only ordinary sequential programming constructs** (load and store). Here we introduce a **programming primitive on a higher level than machine instructions**. In concurrent programs semaphores provide a basic **signalling mechanism** to implement **mutual exclusion** *and* **condition synchronization**.

8.2 Definition of a Semaphore

Definition

A **semaphore** is an **integer-valued variable which can take only non-negative values**. Exactly **two** operations are defined on a semaphore S:

```
down(S)
    if S > 0 then
        S = S - 1
    else
        suspend the action of the process making the call.
        The process is said to be suspended on S.

up(S)
    if there are processes that have been suspended on S
        wake one of them.
    else
        S = S + 1
```

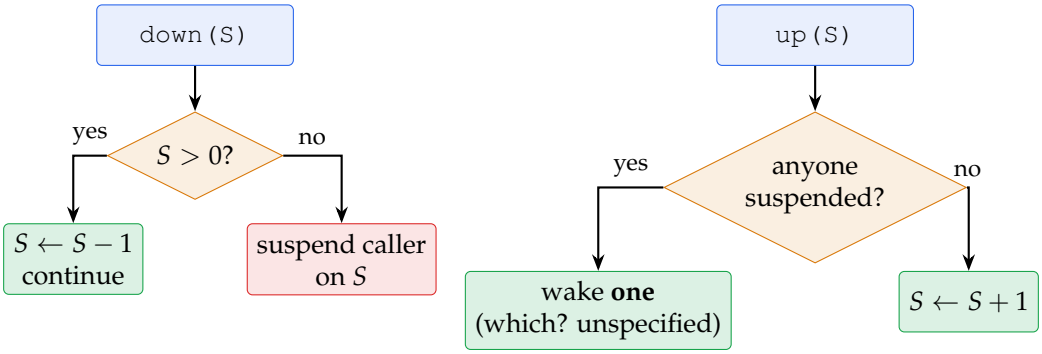


Figure 8.2: The two atomic semaphore operations.

Three defining properties

1. **`down(S)` and `up(S)` are atomic instructions.** In particular, **no instructions can be interleaved between the test that $S > 0$ and the decrement of S** or the suspension of the calling process.
2. A semaphore must be given a **non-negative initial value**.
3. The `up(S)` operation must **awaken one** of the suspended processes. **The definition does not specify which** process must be awakened.

	General semaphore	Binary semaphore
Values	any **non-negative** value	only **0 and 1**
`up(S)`	`S = S + 1`	`if ... else S = 1`
Typical use	counting resources	mutual exclusion (a mutex)

Why property 1 is the whole point

Everything we built by hand in Lectures 5 and 6 — flags, victims, tickets — existed to make **test-then-act** indivisible. A semaphore simply *is* that indivisibility, handed to you as a primitive.

Names you will meet

Dijkstra's original names were **P** (*proberen*, "try") for `down` and **V** (*verhogen*, "increase") for `up`. Other texts use `wait/signal`; POSIX uses `sem_wait/sem_post`; Java uses `acquire/release`.

8.2.1 The Semaphore Invariants

Show that a semaphore S satisfies the following two invariants

$$S \geq 0 \tag{1}$$
$$S = S_0 + \#Up(S) - \#Down(S) \tag{2}$$

where S_0 is the **initial value** of the semaphore, $\#Up(S)$ is the **number of up operations executed** on S, and $\#Down(S)$ is the number of **completed** `down` operations executed on S.

Reasoning

(1) holds by construction: `down` decrements **only** when $S > 0$, so S never goes below 0. When $S = 0$ the caller is suspended instead of decrementing.
(2) Every completed `down` subtracts exactly 1 and every `up` that finds no suspended process adds exactly 1. An `up` that **wakes** a process instead of incrementing is paired with the `down` of the woken process, which then **completes** — so the two cancel and the equation is preserved. The word **completed** in the definition of $\#Down(S)$ is what makes this exact.

8.3 Mutual Exclusion using a Semaphore

```
Semaphore S = new Semaphore(1);

Process P1                          Process P2
while (true) {                      while (true) {
  nonCriticalSection1;                nonCriticalSection2;
  down(S);                            down(S);
  criticalSection1;                   criticalSection2;
  up(S);                              up(S);
}                                   }
end P1;                             end P2;
```

The algorithm generalises to N processes, $N > 1$

In addition it satisfies **all the desirable properties described earlier** (with the caveats on starvation in §8.4).

8.3.1 Proof of Mutual Exclusion

The key invariant

Let $\#CS$ be the number of processes in their critical sections. We prove that

$$\#CS + S = 1 \tag{3}$$

is invariant. Then from invariant (1), $S \geq 0$, so $\#CS \leq 1$ — **which proves mutual exclusion**.

Proof that (3) is invariant

1. $\#CS = \#Down(S) - \#Up(S)$ — a process is in the CS exactly when it has completed a `down` and not yet done its `up`.
2. $S = 1 + \#Up(S) - \#Down(S)$ — this is invariant (2) with $S_0 = 1$.
3. Adding: $S = 1 - \#CS$, i.e. $\#CS + S = 1$. ■

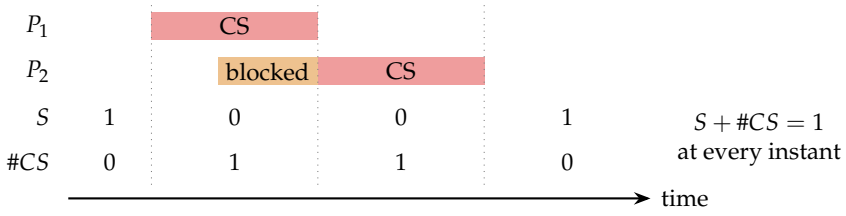


Figure 8.3: The invariant $\#CS + S = 1$ holds throughout an execution.

8.3.2 Proof of Deadlock Freedom

The solution cannot deadlock

For the program to deadlock, **both processes must be suspended on `down(S)`**. Then $S = 0$ and $\#CS = 0$. This contradicts invariant (3), $\#CS + S = 1$, which is **impossible**. ■

8.3.3 Proof of No Individual Starvation (for $N = 2$)

Argument

If P_1 is suspended, then $S = 0$. By invariant (3), P_2 **is in the critical section**. When P_2 exits the critical section it executes `up(S)`, which **wakes some process suspended on** S. Since $N = 2$, P_1 **is the only process suspended on** S, so P_1 is woken. ■

Why the proof needs $N = 2$

The step "P_1 is the only process suspended" is exactly what fails for $N > 2$: `up` may wake **any** suspended process, and it can keep choosing someone else forever. See §8.4.

8.3.4 No Starvation in the Absence of Contention

Argument

In the absence of contention $S = 1$, so **no single process will be delayed**. ■

8.4 Semaphore Definitions

There is no single semaphore

There are **many definitions of semaphore in the literature**. It is important to be able to **distinguish between them**, because the **correctness of a program will depend on the definition being used**.

8.4.1 Blocked-Set Semaphore

```
down(S)
    if S > 0 then S = S - 1
    else suspend the action of the process making the call.
up(S)
    if there are processes that have been suspended on S
        wake one of them.
    else S = S + 1
```

Behaviour

A signalling process **awakens one (unspecified) of the suspended processes**. Suspended processes form an unordered **set**.

8.4.2 Blocked-Queue Semaphore

```
down(S)
    if S > 0 then S = S - 1
    else suspend the action of the process making the call
         and append to a FIFO queue.
up(S)
    if there are processes that have been suspended on S
        wake the process at the head of the queue.
    else S = S + 1.
```

Behaviour

The suspended processes are kept in a **FIFO queue** and **awakened in the same order they were suspended**.

8.4.3 Busy-Wait Semaphore

```
down(S)
    loop
        <if (S > 0) {
             S = S - 1; break;
        }>
```

```
    end loop;
up(S)
    S = S + 1.
```

Behaviour

The value of S is tested in a **busy-wait loop**. The **entire `if`-statement is executed as an atomic operation** (shown by < >), but there **may be interleaving between cycles of the loop**.

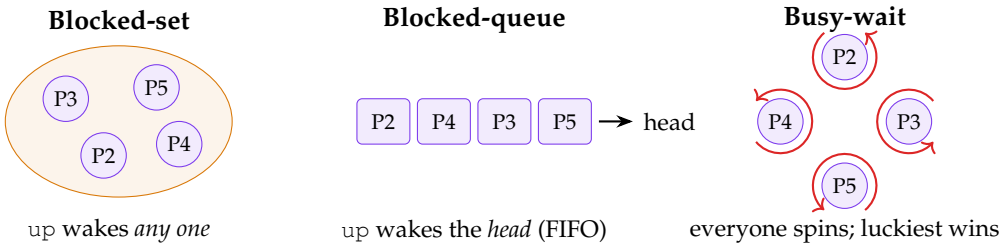


Figure 8.4: How waiting processes are held under the three semaphore definitions.

8.4.4 Which Definitions Starve?

Definition	Starvation possible?	Why
Busy-wait	**Yes**	A looping process can be repeatedly overtaken
Blocked-queue	**No**	FIFO bounds the wait at $N-1$
Blocked-set	**No for $N=2$; Yes for $N>2$**	`up` may keep ignoring one process

Busy-wait semaphore: starvation is possible

1. P_1 executes `down(S)` and enters its CS.
2. P_2 finds $S=0$ and **loops**.
3. P_1 completes an **entire cycle** consisting of post-protocol, non-CS, pre-protocol and **re-enters its CS**.
4. P_2 finds $S=0$ and **loops**.

Repeat forever: P_2 never sees $S>0$ at the moment it looks.

Blocked-queue semaphore: starvation is impossible

Suppose P_1 is blocked on S. Then there are **at most $N-1$ processes ahead of P_1** on the queue for S. Thus P_1 **will enter its CS after at most $N-1$ more executions of `up(S)`**. This is a genuine **bounded waiting** guarantee.

Blocked-set semaphore: starvation is possible for $N>2$

For $N=2$ we showed starvation is impossible. For $N>2$ it is possible to construct an execution sequence where there will **always be at least two processes blocked on** S. Since `up(S)` is only required to **wake one arbitrary process**, it can **conspire to ignore one process**, causing starvation.

Sketch for $N = 3$: P_1 in CS, P_2 and P_3 blocked. P_1's `up` wakes P_2; P_1 loops round and blocks again. P_2's `up` wakes P_1; P_2 loops and blocks. Alternate between P_1 and P_2 forever — P_3 is always in the set but never chosen.

8.5 Exercise: Bill and Ben in the Cafe

The problem

Bill and Ben work in a cafe that specialises in a **soup and sandwich** combination. Bill makes 2 sandwiches while, **at the same time**, Ben makes 2 soups. When the sandwiches and soup are ready Bill and Ben each, **at the same time**, serve a soup and sandwich combination to a customer. These actions are repeated throughout the day. **Bill cannot serve until Ben has made the soup, and Ben cannot serve until Bill has made the sandwiches.**

```
process Bill {                          process Ben {
  while (true) {                          while (true) {
    // Bill makes sandwiches //             // Ben makes soup //
    SYNCH 1;                                SYNCH 2;
    // Bill delivers //                     // Ben delivers //
  }                                       }
}                                       }
```

Provide variable declarations (including initialization) and code for **SYNCH 1** and **SYNCH 2** using semaphores so that the algorithm behaves correctly.

Solution 1

```
Semaphore first  = 0;
Semaphore second = 0;

process Bill {                          process Ben {
  while (true) {                          while (true) {
    // Bill makes sandwiches //             // Ben makes soup //
    down(first);                            up(first);
    up(second);                             down(second);
    // Bill delivers //                     // Ben delivers //
  }                                       }
}                                       }
```

Bill waits for Ben's soup on `first`; Ben then waits for Bill's acknowledgement on `second`. Correct, but **strictly sequential** at the rendezvous.

Solution 2 — OR . . .

```
process Bill {                          process Ben {
    up(second);                             down(second);
    down(first);                            up(first);
}                                       }
```

(Same declarations: `first = 0, second = 0`.)

Solution 3 — the symmetric rendezvous

```
process Bill {                    process Ben {
    up(second);                       up(first);
    down(first);                      down(second);
}                                 }
```

Signal first, then wait. This is the standard **rendezvous** pattern: each announces its own completion before waiting on the other.

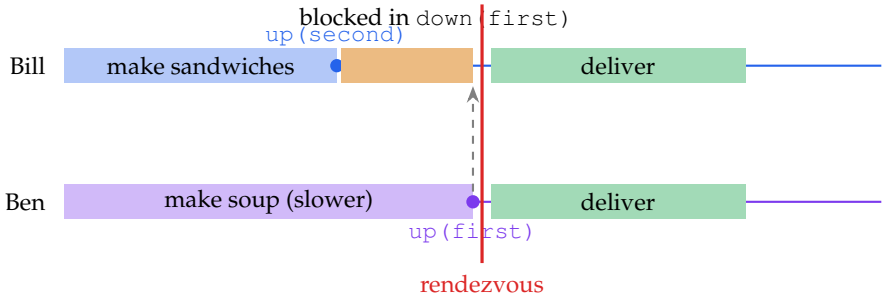


Figure 8.5: Rendezvous: whoever finishes first signals and then waits; both deliver together.

The classic mistake

If you write **down before up in *both* processes** — Bill does `down(first); up(second)` while Ben does `down(second); up(first)` — **both block immediately** and neither signal is ever sent. **Deadlock.** Solution 1 escapes it only because Ben's `up(first)` comes first.

8.6 The Producer–Consumer Problem

An abstraction of communication

The **mutual exclusion problem** is an abstraction of **synchronization** problems common in computer systems. Here we consider an abstraction of **communication** problems called the **producer–consumer problem**.

- **Producer:** by executing an internal operation, **produce**, this process creates a data element which must be sent to the consumers.
- **Consumer:** upon receipt of a data element, this process performs some computation in an internal operation **consume**.

For example this models an **application producing a listing that must be consumed by a printer**.

Synchronous vs asynchronous communication

Synchronous — when one process is **ready to send** and the other **ready to receive**, the data is sent. Elementary, but rigid. **Asynchronous** — if more flexibility is required, a **buffer** is used. The sending process **appends data elements to the tail** of the queue; the receiving process **removes an element from the head**.

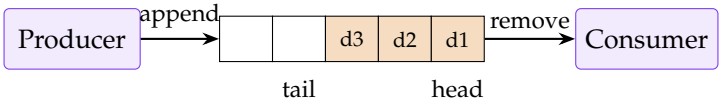


Figure 8.6: Asynchronous communication through a buffer.

8.6.1 Infinite Buffer — First Attempt

```
integer [] buffer = new integer [infinite];
integer head = 0, tail = 0;

process producer                process consumer
integer i;                      integer i;
while (true) {                  while (true) {
  i = produce();                  <wait until (tail > head);>
  buffer[tail] = i;               i = buffer[head];
  tail = tail + 1;                head = head + 1;
}                                 consume(i);
end producer;                   }
                                end consumer;
```

Mutual exclusion is not an issue. Why?

Because the producer touches **only `tail`** and the consumer touches **only `head`**, and with an **infinite buffer** they never address the same slot. There is **no shared variable written by both**, so there is nothing to protect — only a **condition** to wait for.

8.6.2 Infinite Buffer — With a Semaphore

The buffer invariants

$$\#E = 0 + tail - head, \qquad \#E \geq 0$$

where $\#E$ is the number of elements in the buffer. This has **exactly the shape of the semaphore invariants (1) and (2)** with $S_0 = 0$, which suggests solving the problem with a semaphore, `elements`, that **counts the number of data elements in the buffer**.

```
integer [] buffer = new integer [infinite];
integer head = 0, tail = 0;
Semaphore elements = new Semaphore(0);

process producer                process consumer
integer i;                      integer i;
while (true) {                  while (true) {
  i = produce();                  down(elements);
  buffer[tail] = i;               i = buffer[head];
  tail = tail + 1;                head = head + 1;
  up(elements);                   consume(i);
}                               }
end producer;                   end consumer;
```

How it works

The consumer executes **`down(elements)`**, which **suspends if the number of elements in the buffer is 0**. At the end of each loop iteration the producer executes **`up(elements)`**. Here the semaphore is a **resource counter**, not a lock — its value *is* the number of items available.

8.6.3 Bounded (Circular) Buffer

The practical case

We extend the previous solution to a **bounded buffer**, specifically a **circular buffer**. Semaphores serve as **resource counters**: one counts the number of **empty buffer slots** and the other counts the number of **occupied slots**.

Three synchronization problems

1. **Suspend the producer when the buffer is full.**
2. **Suspend the consumer when the buffer is empty.**
3. **Make sure that only one process at a time manipulates a buffer slot.**

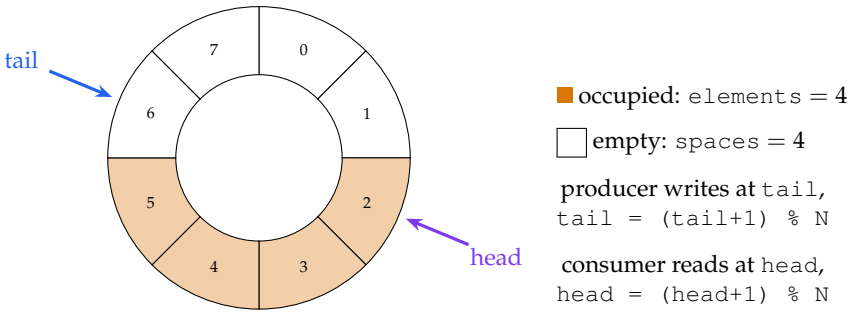


Figure 8.7: A circular buffer of $N = 8$ slots. The two semaphores count occupied and empty slots.

```
integer [] buffer = new integer [N];
integer head = 0, tail = 0;
Semaphore elements = new Semaphore(0);   // occupied slots
Semaphore spaces   = new Semaphore(N);   // empty slots

process producer                    process consumer
integer i;                          integer i;
while (true) {                      while (true) {
  down(spaces);                       down(elements);
  i = produce();                      i = buffer[head];
  buffer[tail] = i;                   head = (head + 1) % N;
  tail = (tail + 1) % N;              consume(i);
  up(elements);                       up(spaces);
}                                   }
end producer;                       end consumer;
```

How it works

The producer now executes **`down(spaces)`**, which **suspends if the buffer is full**. At the end of each loop iteration the consumer executes **`up(spaces)`**.

	`elements`	`spaces`
Initial value	**0**	**N**
`down` by	consumer	producer
`up` by	producer	consumer
Suspends when	buffer **empty**	buffer **full**

Invariant: `elements + spaces + (in-flight operations) = N`.

Order matters

`down(spaces)` must come **before** writing the slot, and `up(elements)` **after**. Reversing them announces an item that is not there yet. With **one** producer and **one** consumer no mutex is needed; with **several** producers you must also protect `tail` (and `head` for several consumers) with a binary semaphore.

8.7 The Dining Philosophers

The problem

Five philosophers sit around a circular table. Each philosopher spends his life **alternately thinking and eating**. In the centre of the table is a large bowl of spaghetti. A philosopher needs **two forks** to eat a helping of spaghetti. **One fork is placed between each pair of philosophers**, and they agree that each will only use the fork to his **immediate right and left**.

The synchronization problem

Coordinate the use of the forks so that: **only one philosopher at a time uses a fork**; there is **no deadlock** (each philosopher holding a fork and refusing to relinquish it); **no philosopher starves** (as a result of neighbours collaborating). Because forks are a **shared resource**, we focus on **acquiring and releasing** them. **Each fork is like a critical section**: it can be held by at most one philosopher at a time.

```
Semaphore[] fork = {1,1,1,1,1};

process philosopher[i=0..3]
while (true) {
  down(fork[i]);
  down(fork[i+1]);
  eat;
  up(fork[i]);
  up(fork[i+1]);
  think;
}
end philosopher;
```

```
process philosopher[4]
while (true) {
  down(fork[0]);
  down(fork[4]);
  eat;
  up(fork[0]);
  up(fork[4]);
  think;
}
end philosopher;
```

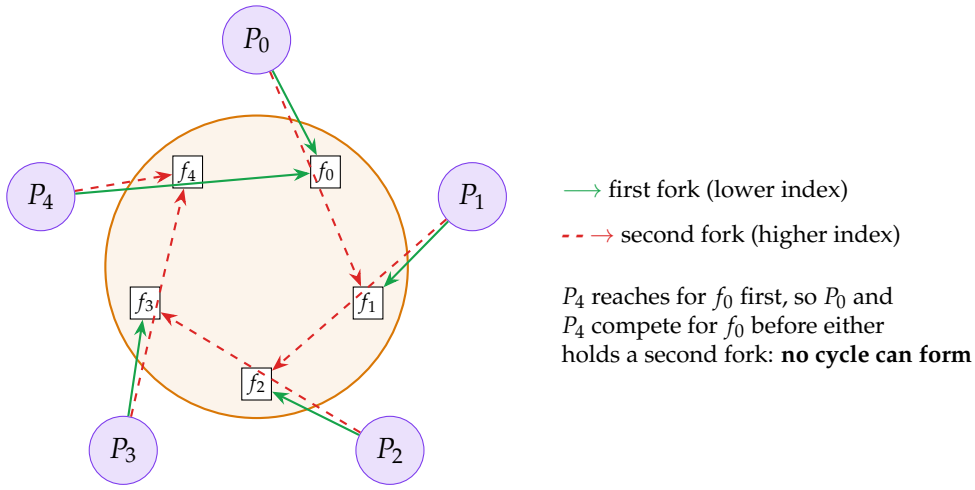


Figure 8.8: Asymmetric fork ordering: everybody acquires forks in increasing index order.

Why philosopher 4 is written separately

Philosophers 0–3 take the **lower-numbered fork first**, then higher. Philosopher 4 would naturally take `fork[4]` then `fork[0]` — closing the cycle. Writing him as **`fork[0]` then `fork[4]`** makes **every** philosopher acquire forks in **increasing index order**, which is exactly the **global lock ordering** rule from Lecture 6: it **breaks the circular wait condition**, so this solution cannot deadlock.

The naive version deadlocks

If **all five** philosophers execute `down(fork[i]); down(fork[(i+1)%5])`, they can each pick up their left fork **simultaneously**, and then all five block forever on the right fork. **All four deadlock conditions hold at once: mutual exclusion on forks, hold-and-wait, no preemption, and a perfect circular wait.**

8.8 The Readers and Writers Problem

The problem

Readers may access a database simultaneously as long as no writer writes, but **only one writer at a time may write**, provided there are **no active readers**.

8.8.1 An Over-Constrained Solution

```
Semaphore rw = 1;

process Reader[i=1..M]              process Writer[i=1..N]
while (true) {                      while (true) {
  down(rw);                           down(rw);
  read_the_database;                  write_the_database;
  up(rw);                             up(rw);
}                                   }
end Reader;                         end Writer;
```

Why it is over-constrained

This treats reading exactly like writing, so **only one reader at a time** may read. Correct, but it **throws away the entire point of the problem** — concurrent reading.

8.8.2 The Proper Solution

```
int nr = 0;                          // number of active readers
Semaphore rw = 1;                    // guards the database
Semaphore mutexr = 1;                // guards nr

process Reader[i=1..M]               process Writer[i=1..N]
while (true) {                       while (true) {
  down(mutexr);                        down(rw);
    nr = nr + 1;                       write_the_database;
    if (nr == 1) down(rw);             up(rw);
  up(mutexr);                        }
  read_the_database;                 end Writer;
  down(mutexr);
    nr = nr - 1;
    if (nr == 0) up(rw);
  up(mutexr);
}
end Reader;
```

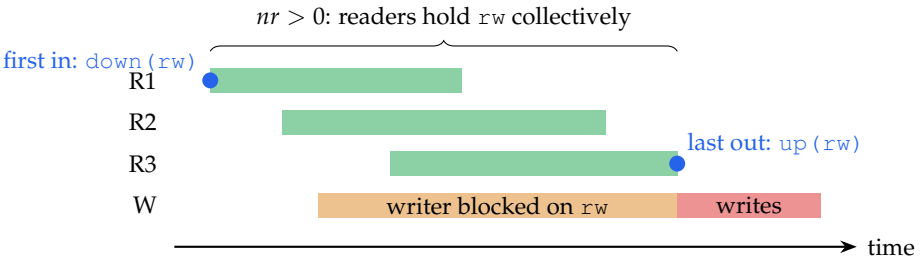


Figure 8.9: The lightswitch pattern: the first reader in turns the lock on, the last reader out turns it off.

The idea: the first reader locks out writers, the last one lets them back in

- `nr` counts **active readers**, and is itself protected by the binary semaphore **`mutexr`**.
- **`if (nr == 1) down(rw)`** — only the **first** reader acquires the database lock on behalf of all readers.
- **`if (nr == 0) up(rw)`** — only the **last** reader releases it.
- Readers 2, 3, 4 . . . walk straight in, so **reading is genuinely concurrent**.

This is a reader-preference solution

A **continuous stream of readers** keeps `nr > 0` forever, so `up(rw)` is never executed and **writers can starve**. Fixing that requires an extra "no new readers while a writer waits" turnstile.

8.9 The Sleeping Barber

The problem

Customers who need a haircut enter the **waiting room**.

- If the room is **full**, the customer **comes back later**.
- If the barber is **busy but there is a vacant chair**, the customer **takes a seat**.
- If the waiting room is **empty and the barber is daydreaming**, the customer **sits in the barber chair and wakes the barber**.
- When the barber finishes cutting a customer's hair, the barber **fetches another customer** from the waiting room. If the waiting room is empty, the barber **daydreams**.

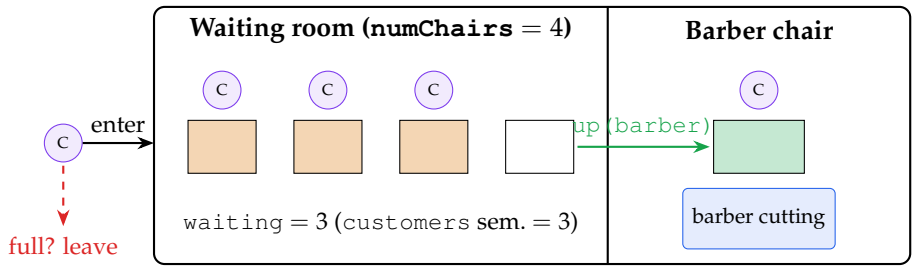


Figure 8.10: The barber shop: chairs buffer *processes* rather than data.

```
Semaphore mutex = 1;
Semaphore customers = 0, cutting = 0, barber = 0;
int waiting = 0, numChairs = ..;

process Customer[i=1..M]
while (true) {
  down(mutex);
  if (waiting < numChairs) {
     waiting++;
     up(customers);
     up(mutex);
     down(barber);
  }
  else up(mutex);
}
end Customer;
```

```
process SleepingBarber
while (true) {
  down(customers);
  down(mutex);
    waiting--;
    up(barber);
  up(mutex);
}
end SleepingBarber;
```

Reading the code

- **mutex** protects the shared counter `waiting`.
- **customers** counts **waiting customers** — the barber's `down(customers)` is where he **daydreams**.
- **barber** is the customer's signal that **it is his turn**.
- The customer who finds `waiting == numChairs` executes **up(mutex) and leaves** — this is the "comes back later" branch, and it is why the `else` must still release the mutex.

What this example illustrates

The use of semaphores for **both mutual exclusion and condition synchronization**. The customers and barber use semaphores to **control each other's flow** through the barber shop. The customer and barber **rendezvous** to interact: **each waits at a certain point for the other to arrive**.

Note the asymmetry with the buffer problem

In producer–consumer, the buffer holds *data*. Here the "buffer" holds *processes* — the customers themselves wait in the chairs. Same counting semaphore, different thing being counted.

8.10 Exercise: The Baboon Crossing Problem

The problem

There is a deep canyon somewhere in **Kruger National Park, South Africa**, and a **single rope** that spans the canyon. Baboons can cross by swinging **hand-over-hand** on the rope, but **if two baboons going in opposite directions meet in the middle, they will fight and drop to their deaths**. Furthermore, the rope is **only strong enough to hold 5 baboons** — more than that and it breaks.
Assuming that we can teach the baboons to use semaphores, design a synchronization scheme with the following properties:

1. Once a baboon has begun to cross, it is **guaranteed to get to the other side without running into a baboon going the other way**.
2. There are **never more than 5 baboons on the rope**.
3. A continuing stream of baboons crossing in one direction should **not bar baboons going the other way indefinitely** (**no starvation**).

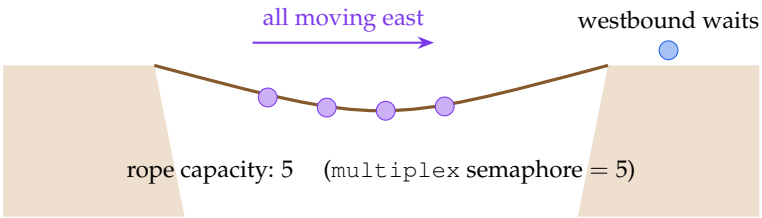


Figure 8.11: Baboon crossing: one direction at a time, at most five on the rope.

How to attack it

Requirement 2 is a **multiplex**: a counting semaphore initialised to **5**. Requirement 1 is **readers–writers in disguise**: baboons going one way are "readers" with respect to each other and "writers" with respect to the other direction — so the **first-in/last-out counter pattern** of §8 applies, with a counter per direction. Requirement 3 is exactly the **writer-starvation** flaw of §8, so it needs a **turnstile** that new baboons must pass and that a waiting opposite-direction baboon can close.

One possible solution sketch (not from the slides)

```
Semaphore rope      = 5;   // multiplex: at most 5 on the rope
Semaphore turnstile = 1;   // fairness: a waiting baboon blocks
   newcomers
Semaphore empty     = 1;   // rope belongs to one direction at a time
Semaphore mutexE = 1, mutexW = 1;
int countE = 0, countW = 0;

process EastBaboon {               // WestBaboon is symmetric (W for E)
  down(turnstile);
    down(mutexE);
      countE++;
      if (countE == 1) down(empty);     // first eastbound claims the
    rope
    up(mutexE);
  up(turnstile);

  down(rope);  cross_east();  up(rope);

  down(mutexE);
    countE--;
    if (countE == 0) up(empty);         // last eastbound releases the
    rope
  up(mutexE);
}
```

A westbound baboon that arrives while eastbound ones are crossing blocks on `empty` *while holding* `turnstile`, so no new eastbound baboon can start; once the current ones finish, the westbound side gets the rope. Fairness ultimately depends on the semaphores being fair (e.g. blocked-queue).

8.11 Comparison of the Classic Problems

Problem	Semaphores used	Core lesson
Mutual exclusion	one binary, init **1**	The semaphore *is* the critical section
Bill and Ben	two, init **0**	**Rendezvous**: signal before you wait
Producer–consumer (infinite)	`elements`, init **0**	Semaphore as a **resource counter**
Producer–consumer (bounded)	`elements`= 0, `spaces`=**N**	Two mirrored counters bound both ends
Dining philosophers	one per fork, init **1**	**Global ordering** breaks circular wait
Readers and writers	`rw`= 1, `mutexr`= 1, counter `nr`	First-in locks, last-out unlocks; risks **writer starvation**
Sleeping barber	`mutex`, `customers`, `barber`	Mutual exclusion **and** condition synchronization together
Baboon crossing	multiplex(5) + direction counters	Combine multiplex, readers–writers and anti-starvation

8.12 Summary

Lecture Summary

- A **semaphore** is a **non-negative integer** with two **atomic** operations, `down` and `up`; introduced by **Dijkstra in 1968** as a primitive **above machine instructions**.
- `down`: decrement if $S > 0$, else **suspend**. `up`: **wake one suspended process** if any, else increment. The definition **does not say which** process is woken.
- Invariants: $S \geq 0$ and $S = S_0 + \#Up - \#Down$ (counting only **completed** downs).
- With $S_0 = 1$, $\#CS + S = 1$ is invariant $\Rightarrow$ **mutual exclusion**; deadlock would force $\#CS = S = 0$, contradicting it.
- **General** semaphores count; **binary** semaphores take only 0 and 1.
- **Busy-wait** $\Rightarrow$ starvation possible. **Blocked-queue** (FIFO) $\Rightarrow$ starvation impossible, wait $\leq N - 1$ ups. **Blocked-set** $\Rightarrow$ starvation possible for $N > 2$.
- **Rendezvous** (Bill and Ben): `up` then `down` in both processes; `down` first in both **deadlocks**.
- **Producer–consumer**: infinite buffer needs only `elements` (init 0); the **circular buffer** needs `elements` (init 0) **and** `spaces` (init N).
- **Dining philosophers**: one semaphore per fork; philosopher 4 takes forks in the **opposite order** so that all acquire in **increasing index order** — no circular wait.
- **Readers and writers**: `nr` guarded by `mutexr`; the **first reader** does `down(rw)`, the **last** does `up(rw)`. The naive one-semaphore version is **over-constrained**; the proper one can **starve writers**.
- **Sleeping barber** shows semaphores doing **mutual exclusion and condition synchronization at once**, with a **rendezvous** between barber and customer.

Self-check Questions

1. State both semaphore invariants and explain why $\#Down$ counts only *completed* operations.
2. Prove $\#CS + S = 1$ and derive mutual exclusion from it.
3. Show that the semaphore mutual-exclusion solution cannot deadlock.
4. Why does the "no individual starvation" proof require $N = 2$?
5. Construct the execution sequence that starves a process under a **busy-wait** semaphore.
6. Explain why a **blocked-set** semaphore can starve a process when $N > 2$ but not when $N = 2$.
7. Write a Bill-and-Ben solution that **deadlocks**, and say exactly why.
8. In the infinite-buffer producer–consumer, why is mutual exclusion not an issue?
9. Give the initial values of `elements` and `spaces` for a bounded buffer of size N, and say what each `down` protects against.
10. Show that the dining philosophers code as given cannot deadlock, naming the condition it breaks.
11. Why is the one-semaphore readers–writers solution "over-constrained", and how can the proper solution starve a writer?

12. Sketch a semaphore solution to the baboon crossing satisfying all three requirements.

LECTURE 9

Basic Constructs II — Mutex, Monitors & Condition Variables

Overview

Why **busy waiting** is wasteful, what a **mutex** is, and why semaphores — although sufficient — are a **low-level, error-prone** mechanism. The answer is the **monitor**: a language construct that gives mutual exclusion *for free* and adds **condition variables** for event synchronization. The lecture then works through the three **signalling disciplines** (signal and exit, signal and continue, signal and wait), how **Java** implements monitors with `synchronized`, `wait()`, `notify()` and `notifyAll()`, and closes with two patterns: the **multiplex** and the **barrier**.

Outline

Semaphores → Mutex → Monitors → Condition Variables, Message Passing, Multiplex, Barriers

9.1 Issues with Busy Waiting

Two problems

1. Wastes CPU time. A spinning thread burns a core doing nothing useful.
2. Unexpected effects. Consider a process h **with high priority** and a process l **with low priority**:

- l is inside the critical region.
- h becomes runnable and **busy waits** for the region.
- Because h has higher priority, the scheduler **never runs** l.
- l **will never get a chance to leave the critical region**, so h spins forever.

This is the priority inversion trap: the high-priority process is blocked by, and simultaneously blocking, the low-priority one.

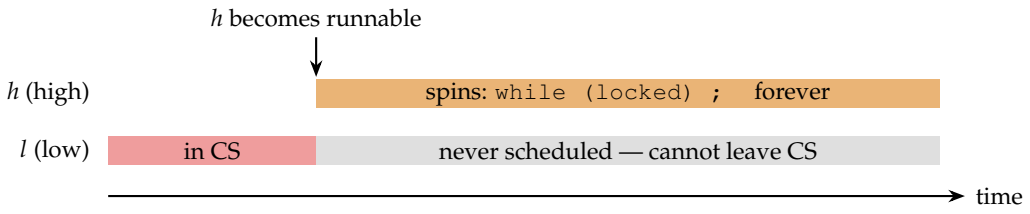


Figure 9.1: Busy waiting + strict priorities: the spinner monopolises the CPU the lock holder needs.

Solution

Sleep and wakeup — block the waiting process instead of spinning, and have the releasing process wake it. That is precisely what a **blocking semaphore** or a **monitor** does.

9.2 Semaphores, Recapped

What a semaphore is, in one box

An **internal counter** capable of providing **mutual exclusion and synchronization**. The counter can go **up and down**: if `down()` when the counter is **zero**, the thread **blocks**; on `up()` the counter increases, and **if there are waiting threads, one of them is released / woken**.

9.3 Mutex

Definition

A **mutex** is what you use **when a semaphore's ability to count is not needed** — i.e. purely **to enforce mutual exclusion**. Implemented as `mutex_lock` and `mutex_unlock`.

	Mutex	**Semaphore**
Values	locked / unlocked (0 or 1)	any non-negative count
Purpose	**mutual exclusion** only	mutual exclusion **and** condition synchronization
Ownership	conventionally, the locker unlocks	**any** process may `up`
Analogy	a key	a counter of tickets

9.4 Why Semaphores Are Not Enough

Semaphores are a low-level mechanism

Semaphores are **fundamentally a synchronization primitive**. They can be used in a systematic way to solve mutual exclusion and synchronization problems. **However, they have a number of disadvantages.** In a sense their use is **similar to that of `goto` statements and pointers**: in all cases programming with them is **prone to errors**. For example, it is easy:

- to **omit an `up` or `down` accidentally**;
- to execute **one too many `up` or `down`** operations;
- to **employ the wrong semaphores**;
- to **fail to protect all critical sections**.

Two further disadvantages

Semaphores are global to all processes. To see how a semaphore — or any other shared variable — is being used, **the entire program must be examined**. There is no locality, no encapsulation.
Mutual exclusion and synchronization are programmed using the same set of primitives.

They are **distinct concepts** and **ideally should be programmed in different ways**.

The monitor concept

The **monitor** provides a **structured concurrent programming primitive** that helps avoid these difficulties. Monitors are related to **objects and object-oriented programming**, and to **abstract data types and encapsulation**.

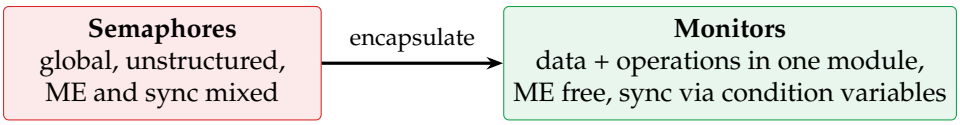


Figure 9.2: From semaphores to monitors.

9.5 Monitor Definition

Definition

A **monitor** is a **programming language module or object that encapsulates items of data together with service operations which operate upon the data**.

The three rules

1. **Only one process at a time is allowed to be active in a monitor** — that is, executing one of the service operations.
2. Processes wishing to execute monitor code concurrently **may not do so**: they must **queue up on the service queue** for that monitor.
3. In addition, a monitor provides **condition variables** that may be used to **synchronize the activities** of processes using the monitor.

9.5.1 Condition Variables

A condition variable *C* is a synchronization primitive with three operations

- `Wait(C)` — the process that called the monitor operation containing this statement is **suspended on a FIFO queue associated with** *C*. **The mutual exclusion on the monitor is released.**
- `Signal(C)` — if the queue associated with *C* is **non-empty**, the process at the **head of the queue is awakened** and **joins the service queue** for that monitor.
- `Non-empty(C)` — a **Boolean function** which returns **true if the queue for** *C* **is non-empty**, and false otherwise.

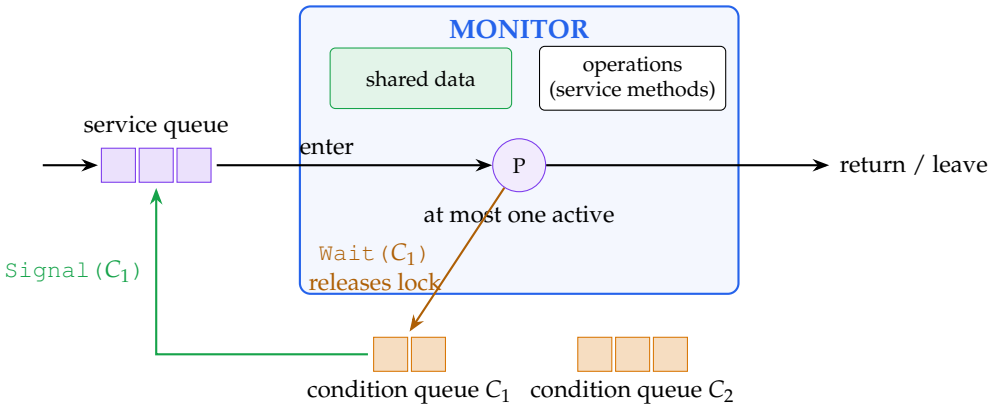


Figure 9.3: Anatomy of a monitor: a service queue to get in, one active process inside, and a FIFO queue per condition variable.

The crucial clause

`Wait(C)` releases the monitor lock. Without that, a process waiting inside a monitor would lock everyone else out, including the process that would have signalled it — instant deadlock.

A condition variable is *not* a semaphore

A `Signal` on an **empty** queue is **lost** — it is not remembered the way `up(S)` increments a counter. This is why waits are always guarded by a test of the actual condition, never used bare.

9.5.2 The Two Essential Capabilities

Monitors provide both requirements for shared-memory concurrent programming

Mutual exclusion — this comes **'for free'**, as by definition **only one process may be active in a monitor at any one time**. If a process P calls a monitor operation while another process is executing monitor code (having previously called an operation in the **same** monitor), then P must **join the service queue** for that monitor and wait for the exclusion to be released.
Event synchronization — obtained using the **condition queue** concept.

9.6 Producer–Consumer with a Monitor

```
monitor PCM
  int N = ..;
  int[] buffer = new int [N];
  int tail = 0, head = 0, count = 0;
  condition notFull, notEmpty;

  public void append (integer i) {
    if (count == N) wait(notFull);
    buffer[tail] = i;
    tail = (tail + 1) % N;
    count++;
    signal(notEmpty);
  }

  public integer take() {
    int i;
```

```
    if (count == 0) wait(notEmpty);
    i = buffer[head];
    head = (head + 1) % N;
    count--;
    signal(notFull);
    return i;
  }
end PCM;
```

```
process producer
int i;
while (true) {
  produce(i);
  PCM.append(i);
}
end producer;
```

```
process consumer
int i;
while (true) {
  i = PCM.take();
  consume(i);
}
end consumer;
```

Compare with the semaphore version

The semaphore solution needed **two counting semaphores** whose values *encoded* the state. The monitor version keeps an **ordinary variable `count`**, and uses **two named condition variables** merely to say *"wait here until something changes"*. The state is explicit and readable — that is the encapsulation payoff.

Notice the `if`

This code uses **`if (count == N) wait(notFull)`**. Whether that is safe depends entirely on the **signalling discipline** — see §9.7. Under **signal and continue** it is **wrong**, and must become a `while`.

9.7 Monitor Signalling Disciplines

The scenario that forces the question

- Process **A** has entered a monitor **MON** and executed **`wait(Q)`**. This causes it to **leave** the monitor.
- Subsequently process **B** enters the monitor and begins executing monitor code.
- Process **C** then calls a method of MON and is **queued on the service queue** for MON.
- Then process **B** executes **`signal(Q)`**.

Now we have **one process executing monitor code (B)** and **two processes wishing to enter** the monitor: **A** on the condition queue Q, and **C** on the service queue. **What happens next?**

Why this matters

This identifies the need for a **signalling discipline**, which defines the **order in which suspended, signalling and third-party processes gain entry to a monitor**. Three distinct disciplines may be identified: **signal and exit**, **signal and wait**, and **signal and continue**.

9.7.1 Signal and Exit

Definition

A process executing (inside a monitor) a **signal** on a condition queue is **required to leave the monitor immediately** after generating the signal, by executing a **return statement** in the service method it invoked. Thus it is **not allowed to change any variables before the signalled process wakes up and resumes**. A process from the wait set for that condition is **awakened and resumes executing inside the monitor**. Thus the **signalled process finds the condition that led to the signal still true** when it resumes. It **gets priority** to execute inside the monitor over **all** processes waiting on the service queue.

9.7.2 Signal and Continue

Definition

A process executing a signal is **not required to leave the monitor**: it **continues executing until completion of the method** it is executing. Also, the **signalled process does not have priority** to enter the monitor before processes waiting on the service queue. Thus it **cannot be guaranteed that the condition leading to the signal is still true** when the signalled process resumes execution:

- the **signalling process may change variables** (and thus invalidate the condition) before exiting the monitor; **or**
- **other processes may barge ahead** of the signalled thread and invalidate the condition.

The consequence — a rule you must memorise

Since the truth of the condition cannot be guaranteed upon resumption of the signalled thread, we must execute the wait in a **`while`** statement rather than an **`if`** statement.

use `while (condition) wait();` rather than `if (condition) wait();` (wrong)

9.7.3 Signal and Wait

Definition

A process executing a signal **hands over access to the monitor immediately to the signalled process** (which has priority over service-queue processes). When the **signalled process completes its monitor method and exits**, the **signaller resumes its execution within the monitor** (again with priority over service-queue processes).

9.7.4 The Three Disciplines Side by Side

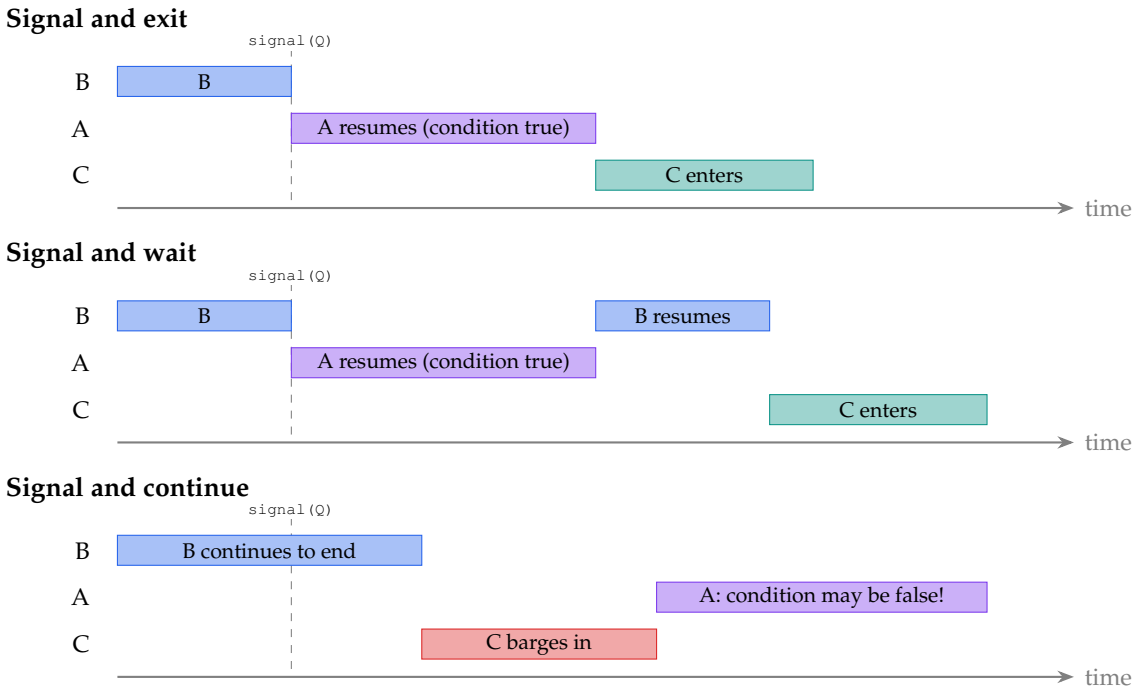


Figure 9.4: Who runs after B executes `signal(Q)` under each discipline (A waits on Q, C waits on the service queue).

	Signal and exit	**Signal and wait**	**Signal and continue**
Signaller after `signal`	must **return immediately**	**suspends**, resumes later	**keeps running**
Signalled process	resumes **at once**, priority over service queue	resumes **at once**, priority over service queue	joins the queue, **no priority**
Condition still true on resumption?	**Yes, guaranteed**	**Yes, guaranteed**	**No guarantee**
Wait must be written as	`if` is safe	`if` is safe	**`while` is mandatory**
Used by	—	some classical monitors	**Java**

9.8 Monitors in Java

How Java expresses a monitor

A monitor may be represented in Java by an **object (class)** in which **data is declared `private`** so it can only be accessed by the object's methods, and the **methods are qualified by the modifier `synchronized`**.

```
class class_name {
    private data;
    ...
    public synchronized type method_name(...) {
        ...
    }
    ...
```

```
}
```

9.8.1 The Object Lock

Each object in Java has associated with it a lock

A thread invoking a `synchronized` method in an object must **obtain the object's lock** before executing the method's code. The thread goes into the **Seeking Lock** state if the lock is currently held by some other thread. When a thread that owns the lock **passes out of the synchronized code it automatically gives up the lock**.

9.8.2 Condition Synchronization in Java

Java has only one, unnamed condition queue

Java **does not provide named condition queues** as described earlier. Instead it provides only a **single (unnamed) thread wait queue per object**. The operations are defined in class **`Object`**, from which all other classes are derived:

- **`public final void notify()`** — wakes up a **single** thread waiting on the object's wait queue (**not necessarily the one which has been waiting longest**).
- **`public final void notifyAll()`** — wakes up **all** threads waiting on the object's wait queue.
- **`public final void wait() throws InterruptedException`** — **enters the wait queue and releases the object's lock**.

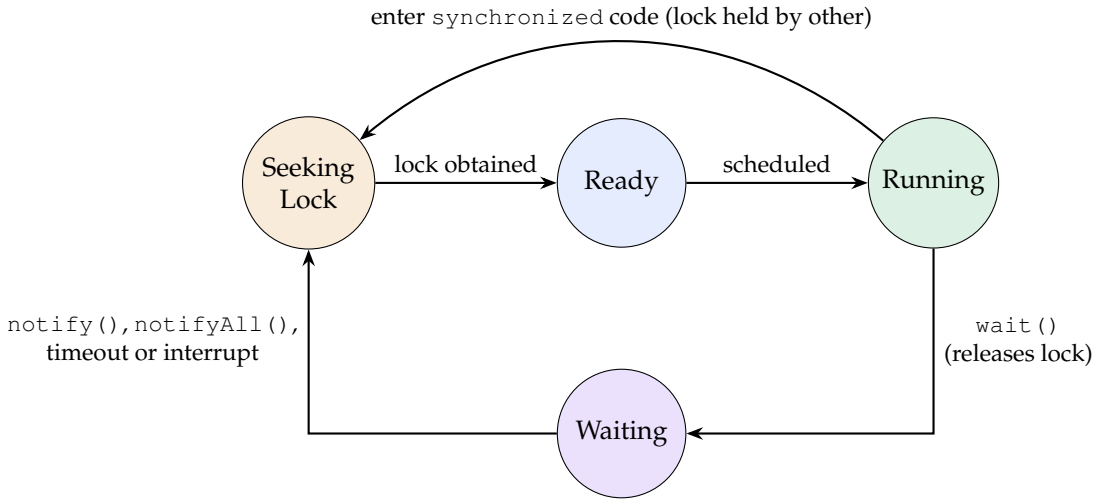


Figure 9.5: Java thread states around an object's monitor.

Consequence of a single unnamed queue

Because there is only one queue per object, a `notify()` may wake a thread waiting for **the wrong condition** (a producer instead of a consumer). This is why **`notifyAll()` is the safe default** in Java: wake everybody, and let each re-test its own `while` condition.

9.9 Producer–Consumer in Java — and the Barging Problem

Java uses the signal and continue discipline

Because of this, **care must be taken** to ensure correct behaviour. In particular, **Java allows a thread on the service queue (Ready state) to barge ahead and enter the monitor before the signalled thread continues** inside the monitor.

The concrete failure

Suppose there are **two consumer processes C1 and C2** and **one producer P1**.

1. **C1 is suspended, C2 has just called `take`**, and **P1 is about to notify**.
2. **P1 awakens C1** and the buffer is **not empty** — so far so good.
3. But if **C2 is allowed to enter the monitor**, it could **make the buffer empty again before C1 has a chance to act**.
4. Once C2 leaves the monitor, **C1 is allowed to continue and will incorrectly take from an empty buffer**.

The condition that was true at the moment of notification is false by the time the notified thread runs.

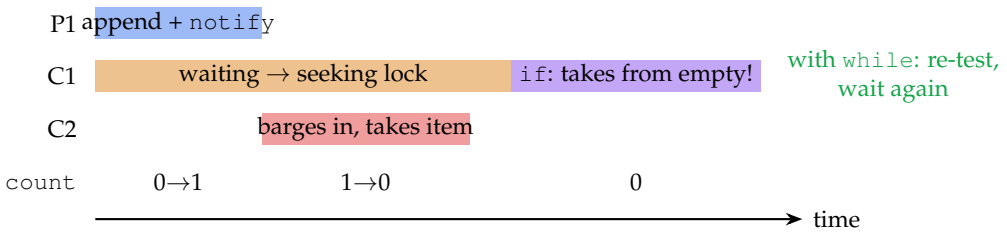


Figure 9.6: The barging problem under signal-and-continue.

The fix

Such erroneous behaviour may be avoided by ensuring that **`wait()` is called in a `while` statement, and not in an `if` statement**. Then, if the condition has been **negated between the execution of `notify()`/`notifyAll()` and the resumption of the signalled thread** (by a third party barging), the signalled process will simply **fail to exit the `while` loop and will execute `wait()` once more**.

```
class PCM {
    private int N = ...;
    private int[] buffer = new int [N];
    private int tail = 0, head = 0;
    private int count = 0;

    public synchronized void append (int i)
    {
        while (count == N)                    // WHILE, not IF
            try { wait(); }
            catch (InterruptedException e) {}
        buffer[tail] = i;
        tail = (tail + 1) % N;
        count++;
        notifyAll();
```

```
    }

    public synchronized int take()
    {
        int i;
        while (count == 0)                    // WHILE, not IF
            try { wait(); }
            catch (InterruptedException e) {}
        i = buffer[head];
        head = (head + 1) % N;
        count--;
        notifyAll();
        return i;
    }
}
```

	Classical monitor (§6)	Java (§9)
Condition queues	**named**: `notFull`, `notEmpty`	**one unnamed** queue per object
Wait	`wait(notFull)`	`wait()`
Signal	`signal(notEmpty)`	`notify()` / **`notifyAll()`**
Discipline	signal and exit / signal and wait	**signal and continue**
Guard	`if` acceptable	**`while` mandatory**

9.10 Multiplex

Definition

A **multiplex** is a critical section that lets **only** N **threads enter at a given time**. **Can be implemented using a semaphore initialised to** N**.**

```
Semaphore multiplex = new Semaphore(N);

multiplex.down();       // at most N threads get past here
    critical_section();
multiplex.up();
```

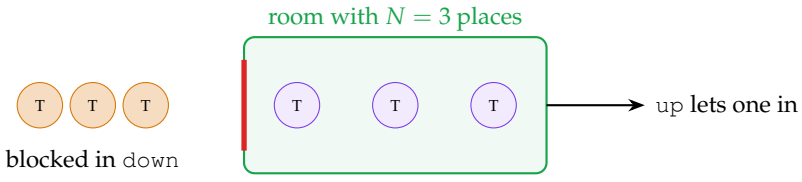


Figure 9.7: A multiplex: a counting semaphore initialised to N.

Uses

A car park with N spaces, a connection pool of N sockets, the **5-baboon rope** from Lecture 8. Note that $N = 1$ degenerates to an ordinary mutex — the multiplex is the general case.

9.11 Barriers

Use of a barrier

- Processes **approach a barrier**.
- **All processes but 1 are blocked** at the barrier.
- The **last process arrives**, and **all are let through**.

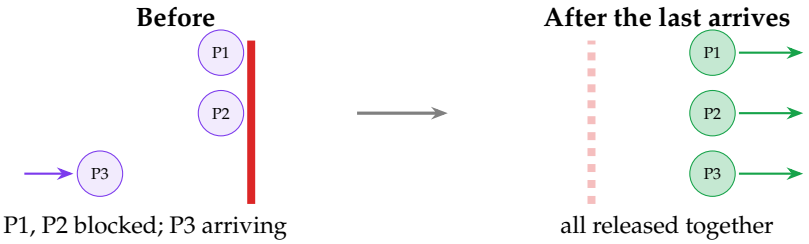


Figure 9.8: A barrier releases every process only when the last one arrives.

Where barriers come from

This is exactly `__syncthreads()` from Lecture 7 and the **implicit barrier at the end of an OpenMP parallel region** from Lecture 4. A barrier turns a set of asynchronous threads into a sequence of synchronous **phases** — essential whenever phase $k+1$ reads data that all threads wrote in phase k.

A simple (single-use) barrier with semaphores — for reference

```
int count = 0;  Semaphore mutex = 1;  Semaphore barrier = 0;

down(mutex);
  count = count + 1;
  if (count == n) up(barrier);    // last one opens the turnstile
up(mutex);
down(barrier);                     // turnstile: pass one at a time ...
up(barrier);                       // ... and let the next one through
```

Reusable barriers need a second turnstile so fast threads cannot lap the slow ones; POSIX provides `pthread_barrier_wait()` ready-made.

9.12 The Construct Ladder

Construct	Provides	Cost / risk
Busy-wait loops (Peterson, Filter, Bakery)	mutual exclusion with load-/store only	**wastes CPU**, priority inversion
Mutex	mutual exclusion	no counting, no condition synchronization
Semaphore	mutual exclusion + synchronization	**global, low-level, easy to misuse**
Monitor	ME **for free** + condition variables, **encapsulated**	discipline-dependent semantics
Java `synchronized` + `wait`/`notifyAll`	a monitor per object	**signal and continue** ⇒ must use `while`
Multiplex / Barrier	higher-level patterns	built on the above

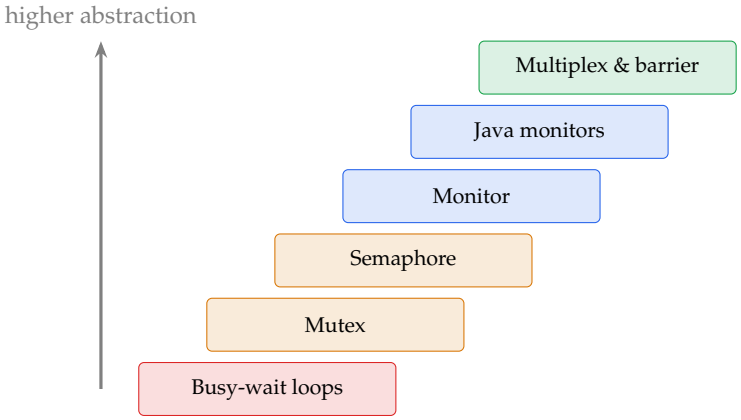


Figure 9.9: The ladder of synchronization constructs built in Lectures 5–9.

9.13 Summary

Lecture Summary

- **Busy waiting** wastes CPU time and can **deadlock a high-priority thread against a low-priority one**; the fix is **sleep and wakeup**.
- A **mutex** is a semaphore whose **counting ability is not needed** — pure mutual exclusion.
- Semaphores are **low-level, like `goto` and pointers**: easy to omit an `up`/`down`, to do one too many, to use the wrong one, or to miss a critical section. They are also **global**, so understanding one requires reading the **whole program**, and they **conflate** mutual exclusion with synchronization.
- A **monitor** encapsulates **data + service operations**; **only one process is active in it at a time**, others queue on the **service queue**.
- **Mutual exclusion comes free** with a monitor; **event synchronization** comes from **condition variables**.
- `Wait(C)` suspends on a **FIFO queue for** C and **releases the monitor**; `Signal(C)` moves the **head** of that queue to the **service queue**; `Non-empty(C)` tests the queue.
- Three **signalling disciplines**: **signal and exit** (signaller returns immediately), **signal**

and wait (signaller yields, resumes after), **signal and continue** (signaller keeps going, signalled thread has **no priority**).

- Under **signal and continue** the condition **may be false on resumption**, so you must write **`while (condition) wait();`** and never `if`.
- **Java** implements monitors with `private` data + **`synchronized`** methods, one **object lock**, and a **single unnamed wait queue** with **`wait()`, `notify()`, `notifyAll()`**.
- **Java uses signal and continue**, so a barging consumer can empty the buffer between `notify()` and resumption — hence `while` + preferably **`notifyAll()`**.
- A **multiplex** admits N **threads at once** (semaphore initialised to N); a **barrier** blocks every process until **the last one arrives**, then releases all.

Self-check Questions

1. Describe the priority-inversion scenario that makes busy waiting dangerous, and give the fix.
2. List the four coding mistakes that make semaphores error-prone, and the two structural disadvantages.
3. Define a monitor and state exactly how many processes may be active in one.
4. Give the three condition-variable operations and say precisely what `Wait(C)` does to the monitor lock.
5. Walk through the A/B/C scenario and explain what each of the three disciplines does next.
6. Under which disciplines is `if (condition) wait();` safe, and under which is it a bug?
7. Construct the two-consumer barging failure in Java step by step.
8. Why does `while` fix that failure, while `if` does not?
9. What does Java provide instead of named condition queues, and what problem does that create?
10. Explain the difference between `notify()` and `notifyAll()`, and say when you must prefer the latter.
11. How is a multiplex implemented, and what is it when $N = 1$?
12. Describe the behaviour of a barrier and name two constructs from earlier lectures that are barriers.

APPENDIX A

Quick Reference

A.1 Formula Sheet

Quantity	Formula	Lecture
Amdahl's Law	$\text{Speedup} = \dfrac{1}{(1-p)+p/n}, \quad \text{Speedup}_{\max} = \dfrac{1}{1-p}$	2
Speedup	$T_1/T_P \leq P$	2
Efficiency	$\dfrac{T_1}{P\,T_P}, \quad 0 < E \leq 1$	2
Work Law / Span Law	$T_P \geq T_1/P, \quad T_P \geq T_\infty$	2
Parallelism	$T_1/T_\infty = \text{Work}/\text{Span}$	2
Samples needed	$n = \left(\dfrac{100\,z\,s}{r\,\bar{x}}\right)^2$	2
Number of interleavings	$\dfrac{(nm)!}{(m!)^n}$ (n processes, m atomic actions each)	3
Filter lock	at most $n - L$ threads at level L	6
Bakery order	$(a,i) < (b,j) \iff a < b \vee (a = b \wedge i < j)$	6
GPU peak performance	cores $\times$ FLOPS/core $\times$ clock	7
CUDA global index	`blockIdx.x * blockDim.x + threadIdx.x`	7
Semaphore invariants	$S \geq 0, \quad S = S_0 + \#Up - \#Down$	8
Semaphore mutual exclusion	$\#CS + S = 1$	8

A.2 Correctness Properties at a Glance

Property	Meaning	Kind
Mutual exclusion	At most one process in its critical section	Safety
Deadlock-freedom	If some processes try to enter, one eventually succeeds	Liveness
Livelock-freedom	No infinite run where everyone is busy but no one enters	Liveness
Starvation-freedom	Every process that tries eventually enters	Liveness
Bounded waiting / FCFS	Bound on how often others may overtake a waiting process	Liveness (fairness)
No starvation w/o contention	A lone process gets in with minimal overhead	Liveness / efficiency

A.3 Optional Web Figures

These notes draw every explanatory diagram in TikZ, so they compile with no external images. A few spots also have an *optional* photo or reference chart from Wikipedia/Wikimedia Commons. To include one, download the image, save it in a `figures/` folder next to `main.tex` with the exact filename below, and recompile; the figure then appears automatically. Check the licence on each image's Commons page (most are CC BY or CC BY-SA, which require attribution) and update the caption credit if you share the PDF.

Filename in `figures/`	Lecture	Where to get it
`moores-law.png`	1	Lead chart of the Wikipedia article *Moore's law* (transistor counts 1970–2020, Our World in Data)
`amdahls-law.png`	2	Lead chart of the Wikipedia article *Amdahl's law* (export the SVG as PNG)
`dining-philosophers.png`	3	Lead illustration of the Wikipedia article *Dining philosophers problem*
`gpu-card.jpg`	7	Any photo of an NVIDIA Tesla / GeForce card from the Wikipedia article *Graphics processing unit* or Commons
`xeon-phi.jpg`	7	Photo from the Wikipedia article *Xeon Phi*
`railway-semaphore.jpg`	8	Photo from the Wikipedia article *Railway semaphore signal*

A.4 Building These Notes

```
pdflatex main.tex      # run twice so the contents and figure list fill in
pdflatex main.tex
# or simply:  latexmk -pdf main.tex
```

Only standard packages are used (TikZ, pgfplots, tcolorbox, listings, mathpazo, helvet, courier, pifont), all of which ship with TeX Live and MiKTeX. Overleaf: upload the whole folder and compile `main.tex`.